



Container Orchestration

컨테이너 오케스트레이션 (Container Orchestration)

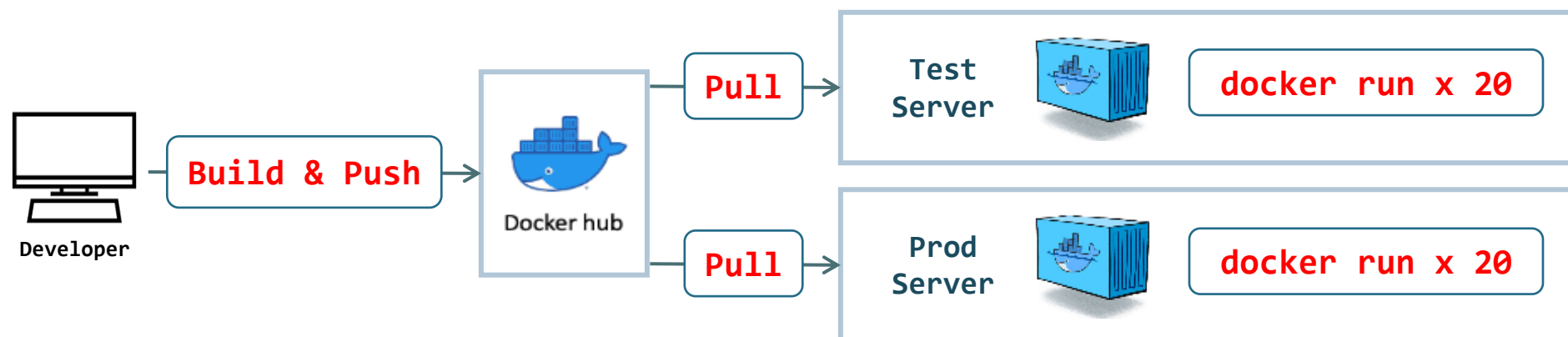
◎ Container Orchestration의 이해

- ▶ 복잡하게 구성되어있는 다양한 형태의 컨테이너를 서버 관리자 대신 관리작업 코드를 작성하여 관리작업을 수행하는 것을 말한다.
- ▶ 컨테이너 오케스트레이션은 서버 자원의 클러스터링, 컨테이너 배포관리, 서비스 탐색 및 접근, 부하 처리, 장애 복구등의 자동화를 지원한다.
- ▶ 대표적인 컨테이너 오케스트레이션 구현 도구로는 Docker Swarm, Kubernetes 등이 있다. (실질적인 표준: "**Kubernetes**")

◎ Container Orchestration을 사용하는 이유

- ▶ 일반적인 컨테이너 가상화의 경우 배포 및 운영관리 리소스 소모가 많아 효율적인 컨테이너 관리가 불가능하다.
- ▶ 컨테이너 오케스트레이션 도구를 사용 할 경우 클러스터링 된 서버들을 중앙 집중식 관리를 통해 관리 리소스를 대폭 감소 시킬 수 있다.
- ▶ 컨테이너 오케스트레이션 도구를 사용 할 경우 자동화 된 배포 및 스케일링, 롤아웃/롤백, 컨테이너 복구 작업등을 구현 할 수 있다.

▣ EX: 다수의 서버에 컨테이너 가상화만을 이용한 이미지 빌드 후 컨테이너 배포 과정



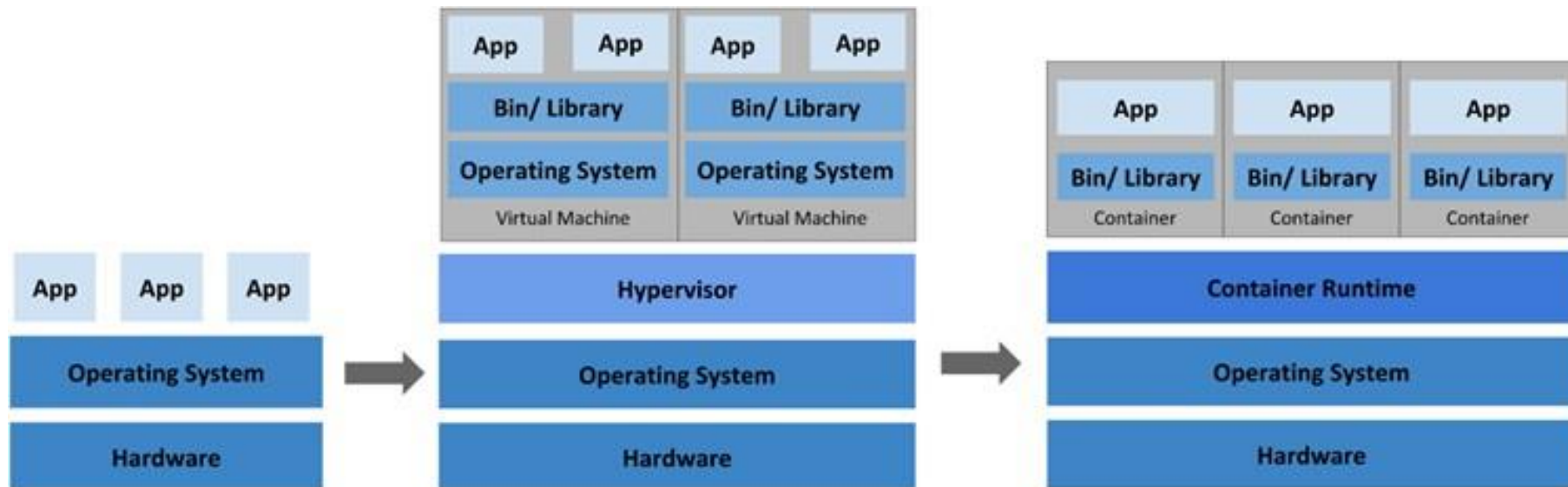
Container Virtualization Summary

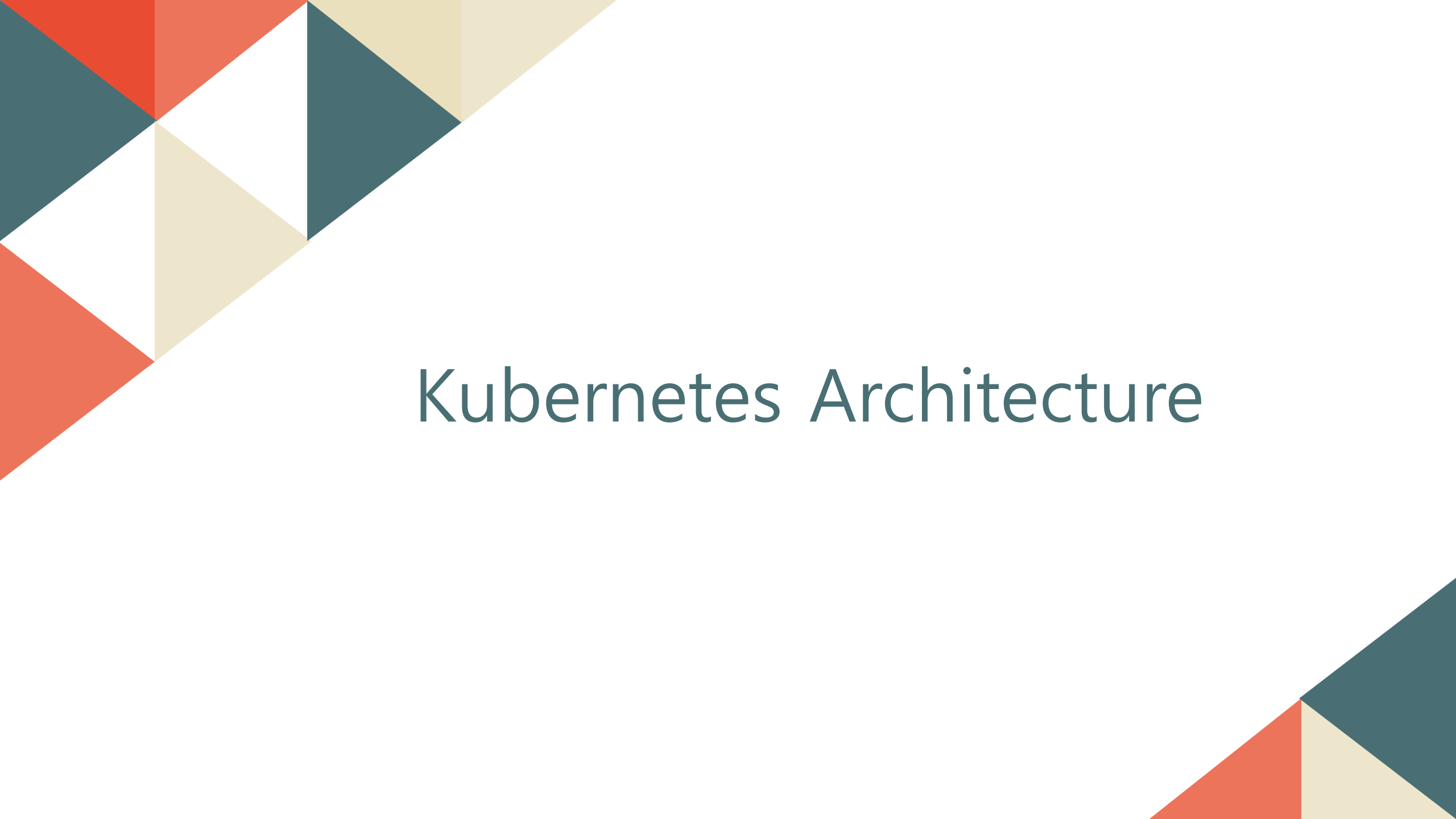
◎ 가상화의 발전

- ▶ 초기 Traditional 형태의 서비스 운영환경의 경우 각 APP의 Lib Version 충돌문제등의 독립성이 보장되지 않아 발생하는 문제점들이 있다.
- ▶ 위와 같은 문제들을 해결하기위해 가상화 서비스의 형태가 발전되었으며, 각 APP의 독립성을 보장하여 효율적인 서비스를 제공한다.
- ▶ 하이퍼 바이저 형식의 가상화의 경우 VM을 생성 후 Guest OS를 설치하여 독립 된 운영환경을 구축한다. (HW 자원의 비효율적 사용)
- ▶ 컨테이너 형식의 가상화의 경우 별도의 VM 및 Guest OS 설치없이 Host OS의 커널 기능을 활용하여 독립 된 운영환경을 구축한다.

◎ 컨테이너 가상화

- ▶ Container 가상화 구현기술은 도커, OpenVZ, LXC등 다양한 기술들이 존재하지만 사실상의 컨테이너 가상화 표준으로는 도커를 사용한다.
- ▶ Container 가상화와 함께 사용되는 다양한 컨테이너 활용 기술들은 도커를 기준으로 개발 및 업데이트되기 때문이다. (EX : 쿠버네티스)



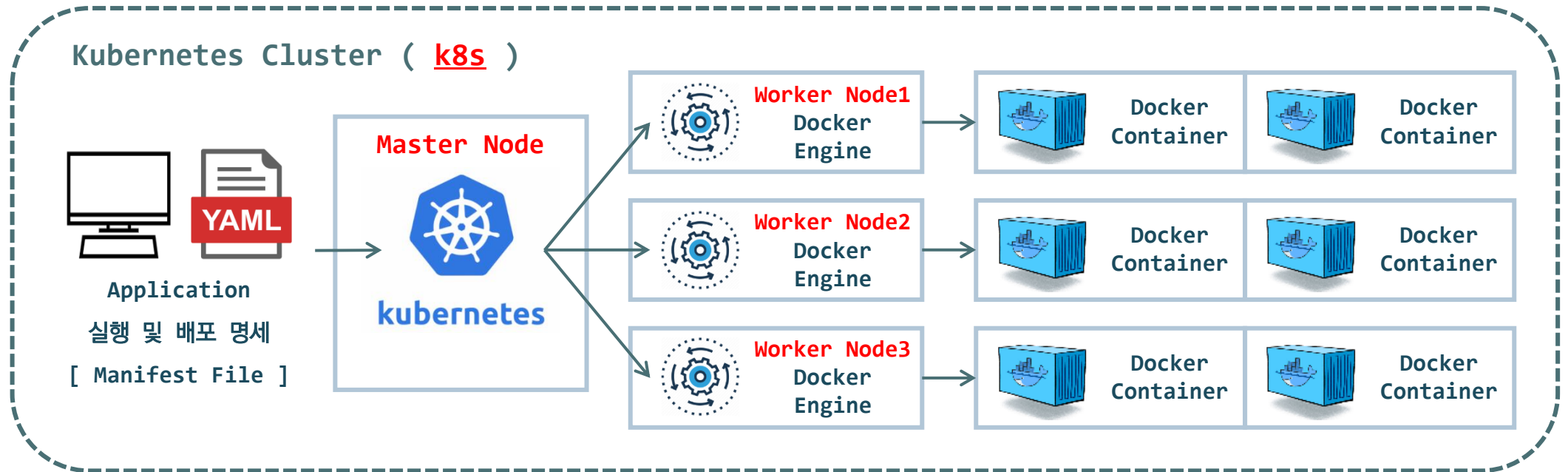


Kubernetes Architecture

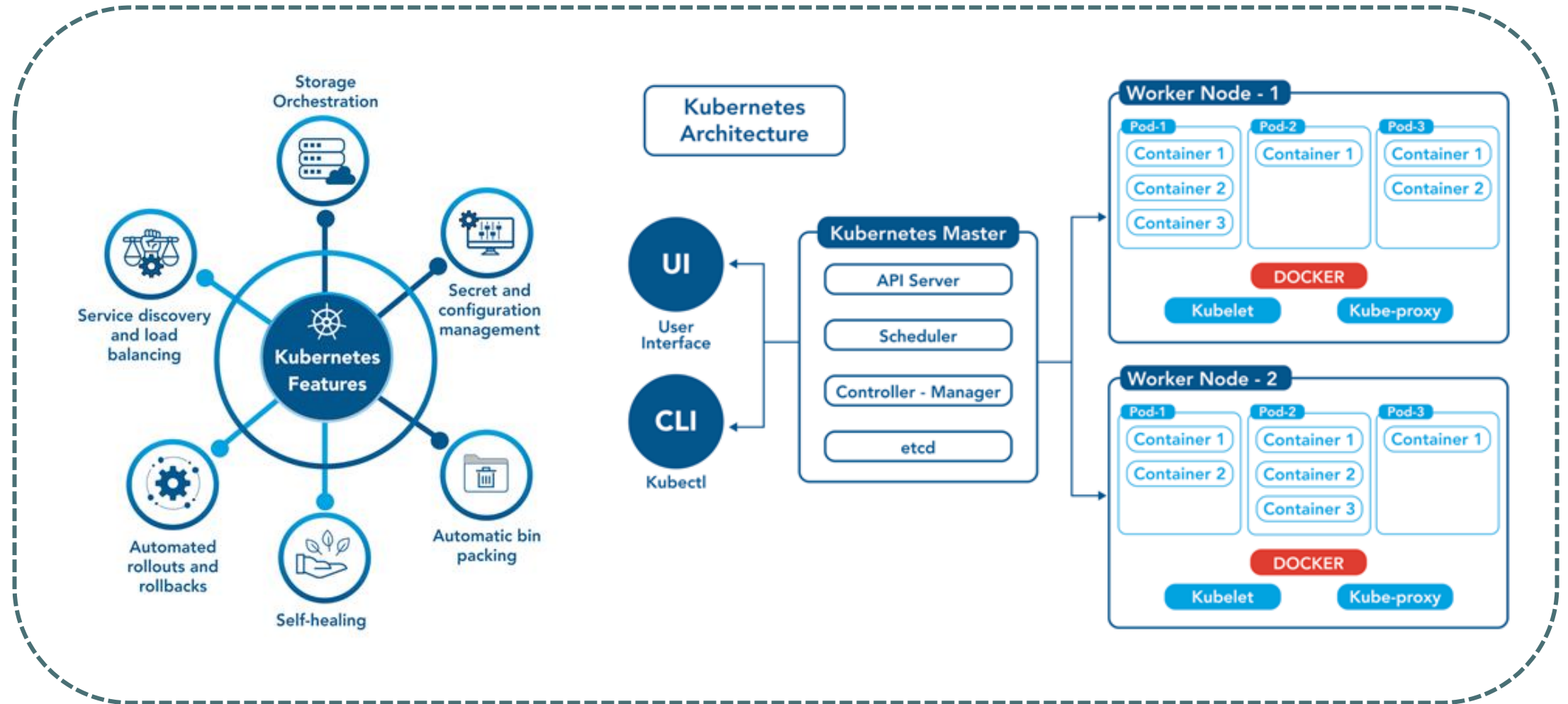
쿠버네티스 아키텍처 (Kubernetes Architecture)

◎ Kubernetes Container Orchestration Tool

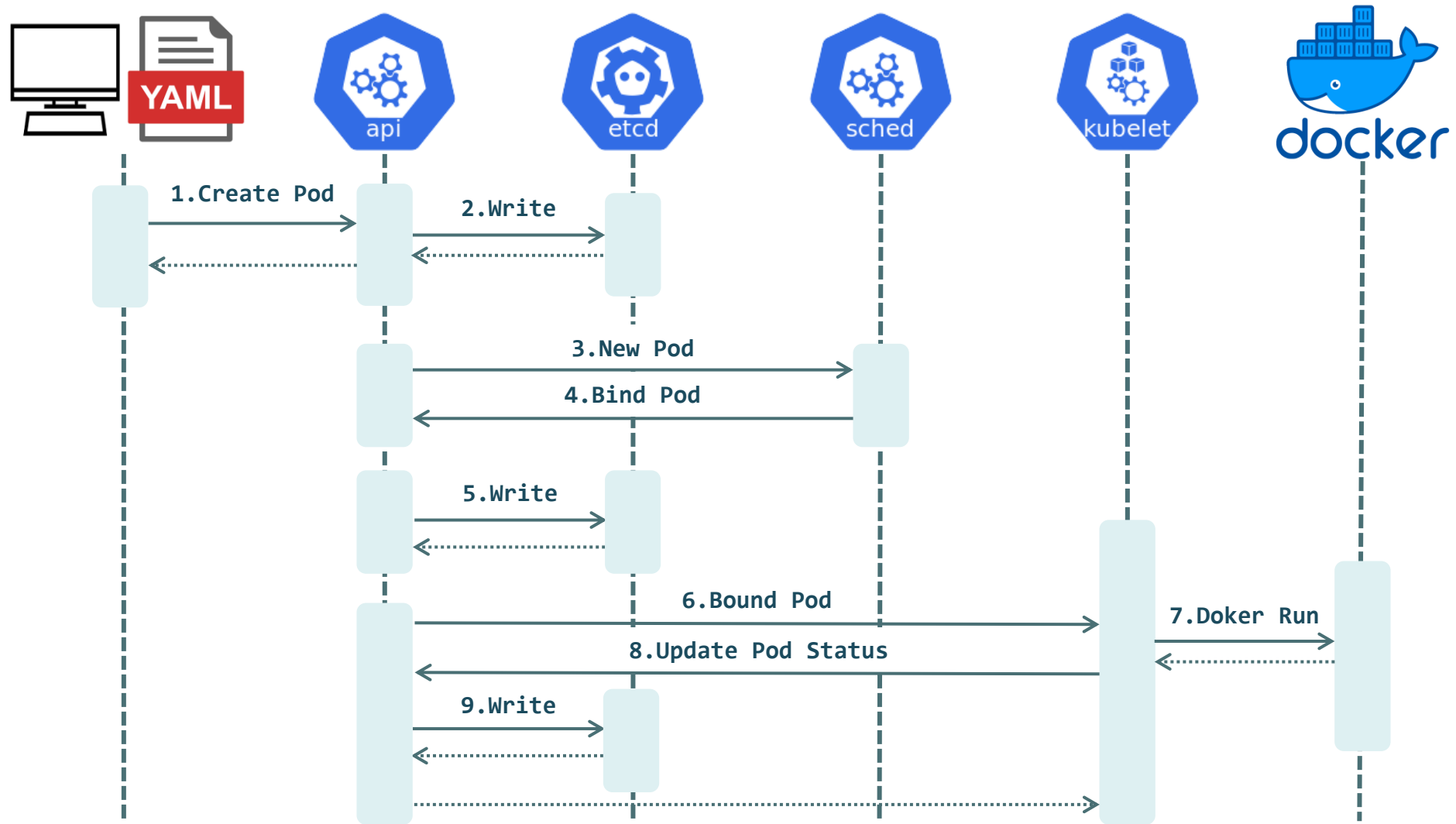
- ▶ Kubernetes : 구글에서 2014년에 오픈소스로 공개한 사실상의 표준 컨테이너 오케스트레이션 도구 (구글의 컨테이너 운영 노하우가 담겨있다.)
- ▶ kubernetes는 MSA 구조의 컨테이너 배포, 서비스 장애 복구등의 컨테이너 기반의 서비스 운영에 필요한 대부분의 기능을 지원한다.
- ▶ Kubernetes는 클라우드 환경에서 운영에 필요한 대부분의 기능과 컴포넌트를 지원하여 다른 클라우드 운영 도구와 쉽게 연동되어 확장성이 높다.
- ▶ 구글, 레드햇 등이 참여하는 오픈소스 프로젝트에서 Kubernetes를 구성하는 소스코드에 개발에 참여하고 있어 안정성 및 신뢰성이 높다.
- ▶ 컨테이너 오케스트레이션을 구성하는 다양한 컴포넌트들은 대부분 Kubernetes를 기준으로 개발 및 업데이트되고 있다.
- ▶ 현재 Kubernetes는 CNCF(Cloud Native Computing Foundation) 재단에서 관리하는 오픈소스 (URL: <https://landscape.cncf.io/>)



쿠버네티스 아키텍처 (Kubernetes Architecture)



쿠버네티스 아키텍처 (Kubernetes Architecture)



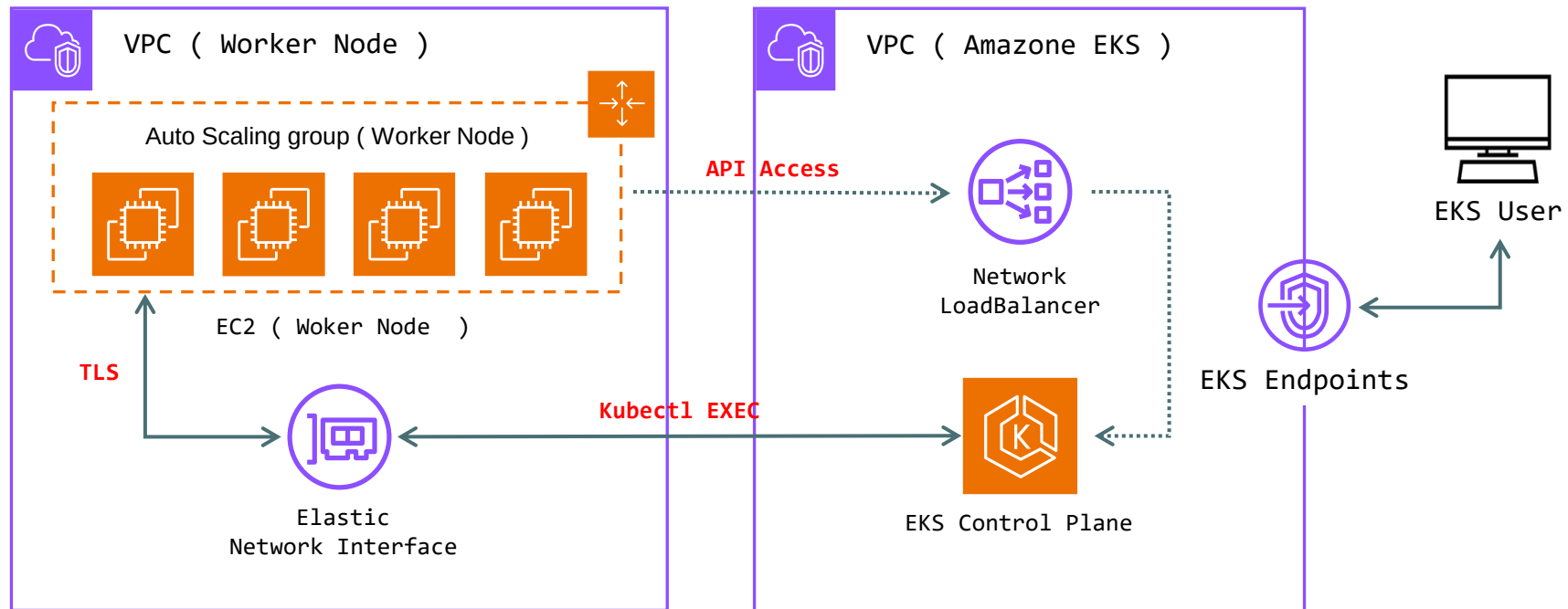


Elastic Kubernetes Service

AWS Elastic Kubernetes Service 개요 및 아키텍처

◎ AWS EKS

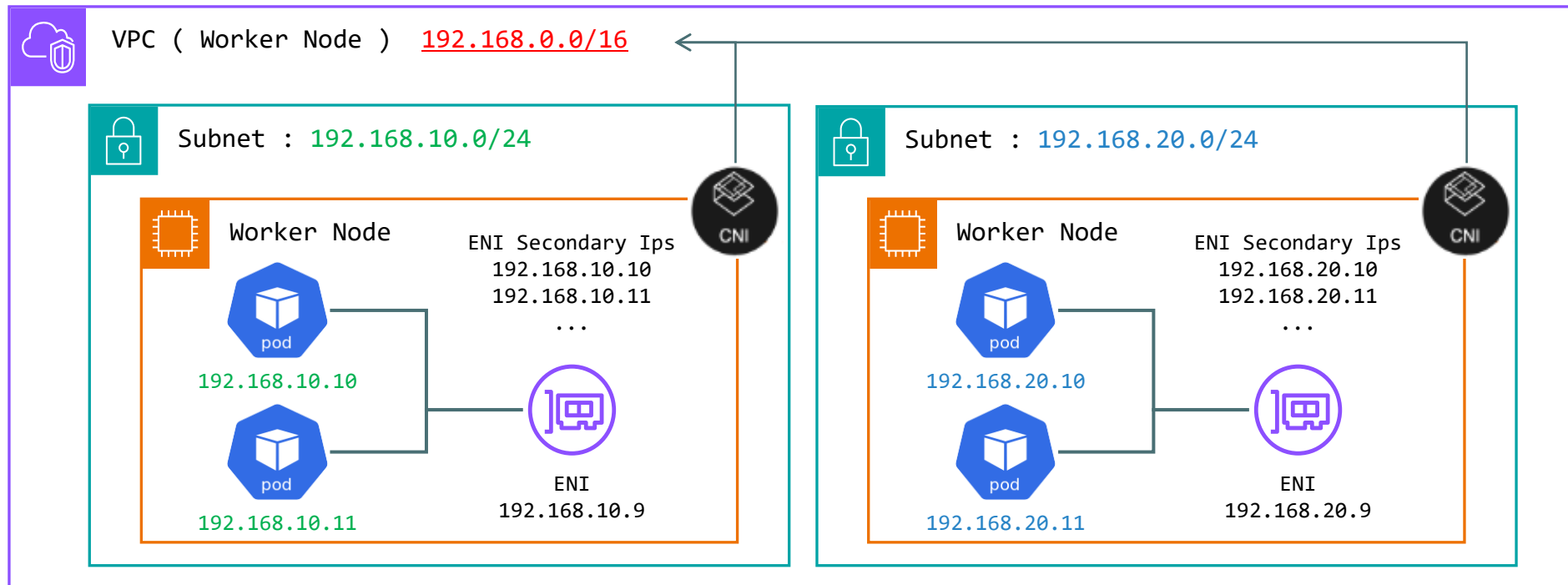
- ▶ EKS는 K8s의 Control Plane을 구성 및 관리할 필요없이, K8s환경을 쉽게 실행 할 수 있는 완전 관리형 서비스를 말한다.
- ▶ EKS를 사용할 경우 **사용자**는 **Data Plane**영역만 관리하고 모든 **Control Plane**영역은 **AWS에서 직접 관리**하게 된다.
- ▶ 사용자는 K8s에 접근가능한 EndPoint 주소를 제공받게되며, 해당 Endpoint 주소로 접근하여 K8s 관리작업을 수행한다.
- ▶ EKS 클러스터 비용 : 서울 리전 기준 (클러스터 x 시간당 0.10\$) / Instance 유형에 따른 이용 요금은 별도로 청구



AWS Elastic Kubernetes Service CNI 및 보안그룹

◎ AWS EKS VPC CNI (Container Network Interface)

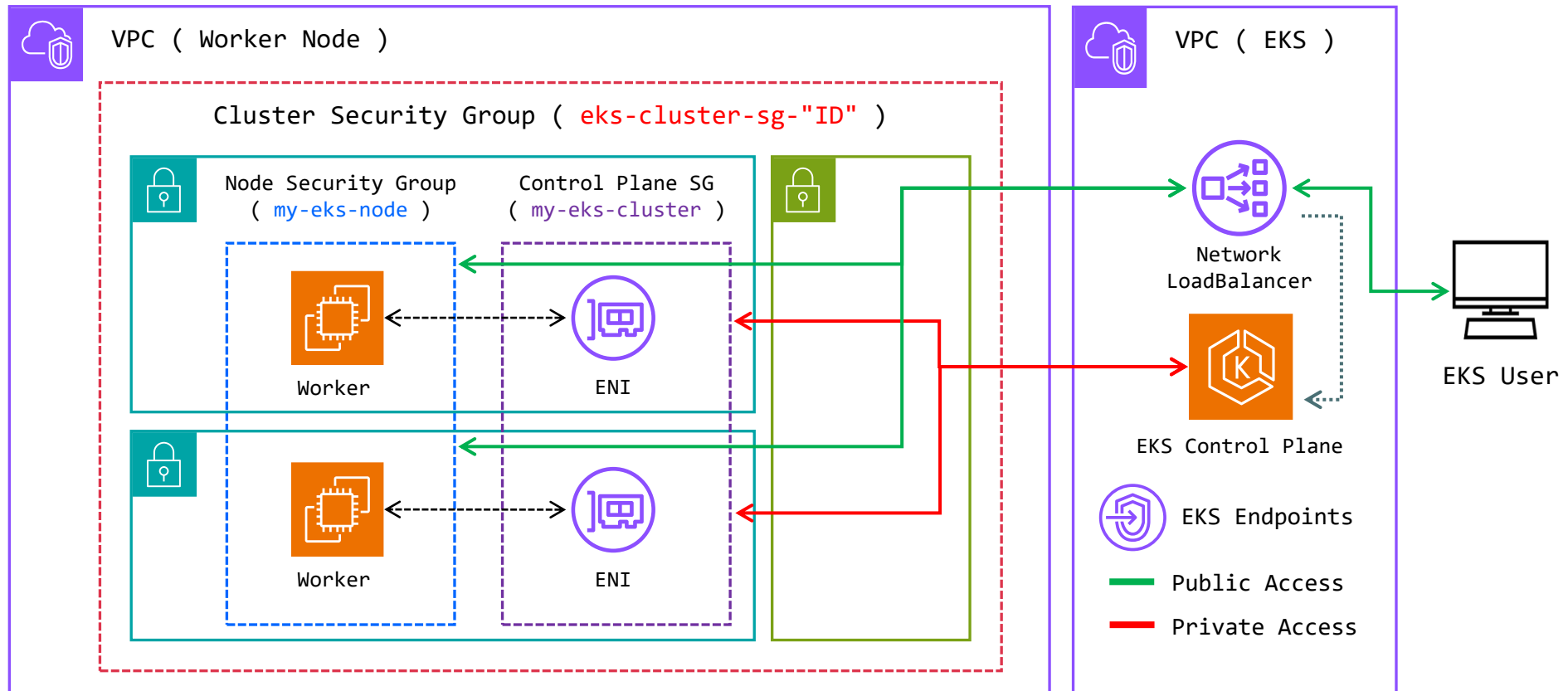
- ▶ CNI는 K8s내에서 Pod의 Network 연결을 담당하는 오픈소스 플러그인을 말한다. (EKS 구성 시 함께 자동으로 구성)
- ▶ EKS 환경에서는 VPC와 CNI가 통합되어 함께운영된다. (VPC Subnet의 IP대역 = Pod가 할당받는 IP대역)
- ▶ VPC와 CNI가 통합운영되어, 보안그룹, Routing Table 규칙, Network ACL 정책등을 생성하고 연결하는것이 편리해진다.
- ▶ VPC의 CNI 한계점으로는 Worker Node의 인스턴스 타입에 따라 최대 지원가능한 Pod에 부여 될 IP주소의 개수가 정해져 있다.



AWS Elastic Kubernetes Service CNI 및 보안그룹

© AWS EKS Security Group (Terraform)

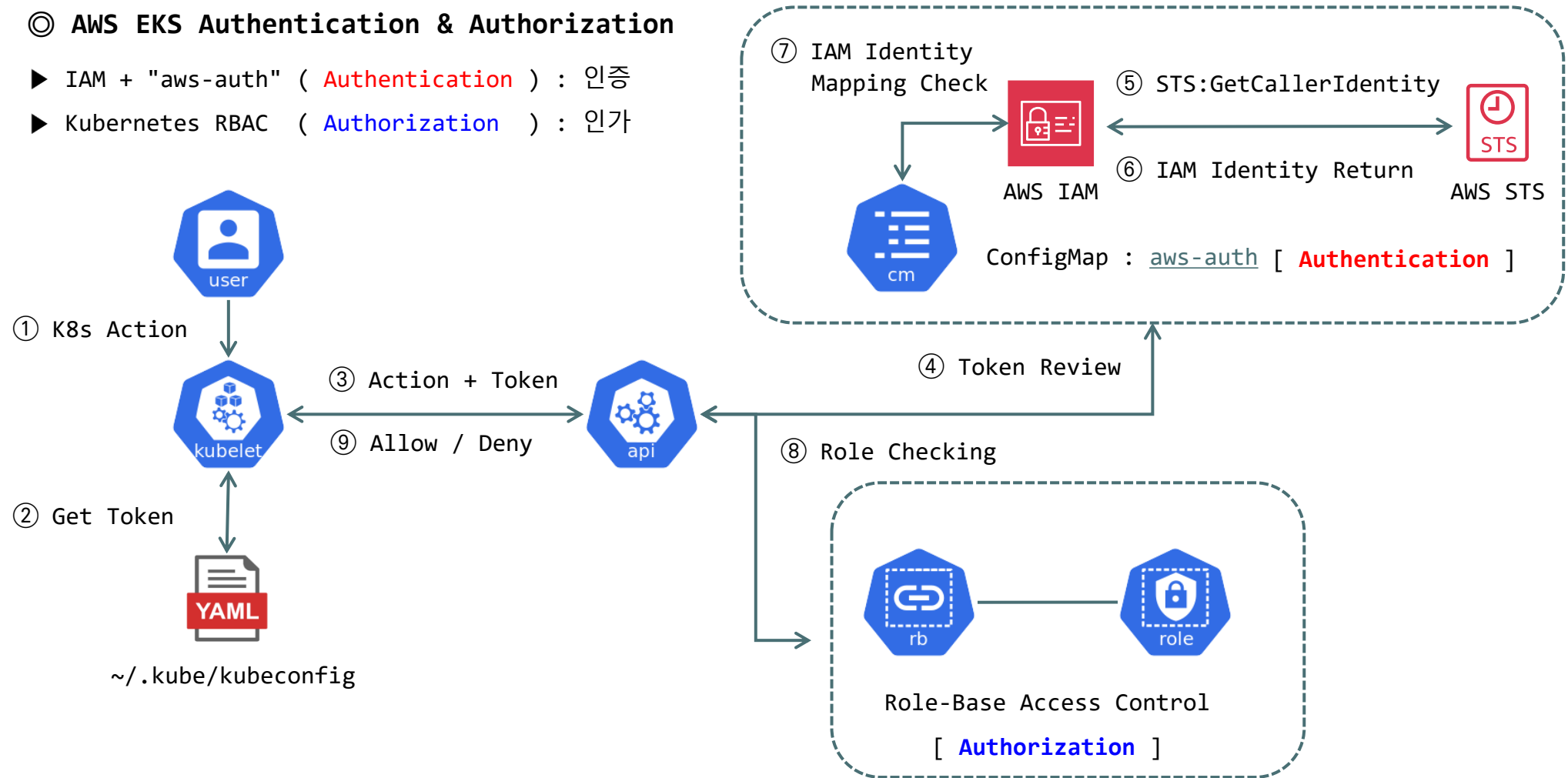
- ▶ Cluster Security Group : Control Plane 영역과 Worker Node 영역간 모든 통신을 허용하기 위해 생성되는 보안그룹
- ▶ Control Plane Security Group : Control Plane의 Cluster API와 Worker Node간 통신을 허용하기 위해 생성되는 보안그룹
- ▶ Node Security Group : Worker Node EC2 Instance에게 할당되는 보안그룹 (CoreDNS 및 Cluster API를 허용 정책 설정)



AWS Elastic Kubernetes Service 인증 및 인가

© AWS EKS Authentication & Authorization

- ▶ IAM + "aws-auth" (**Authentication**) : 인증
- ▶ Kubernetes RBAC (**Authorization**) : 인가





Kubernetes Pod & ReplicaSet

Kubernetes Pod & ReplicaSet

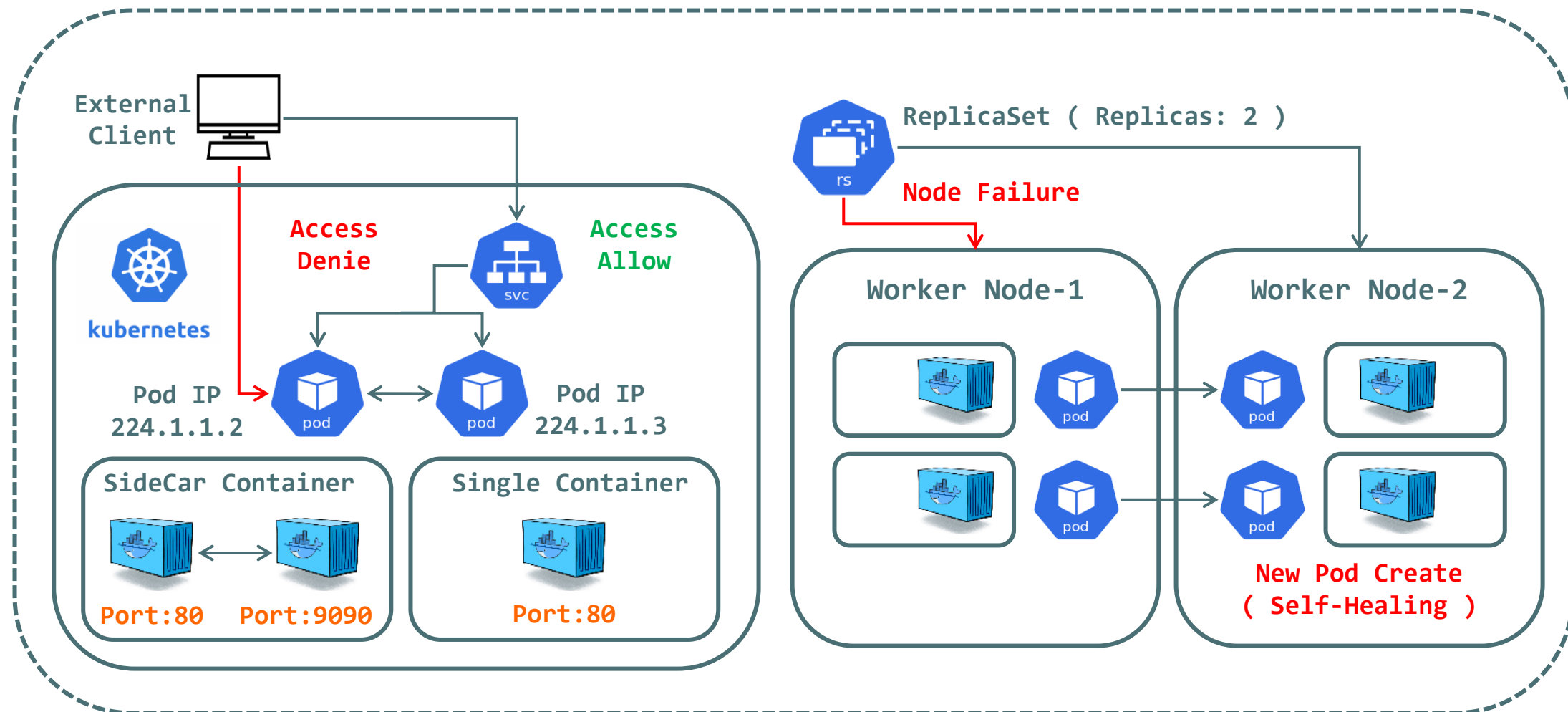
◎ Kubernetes Pod

- ▶ 쿠버네티스 클러스터 워커노드에서 애플리케이션 컨테이너 배포를 위한 기본 단위이다.
- ▶ Pod와 Container는 1 : 1 관계가 될 수도 있고 1 : N 관계가 될 수도 있다. (주로 하나의 Pod에 하나의 Container를 실행)
- ▶ Pod는 하나의 가상 서버로 동작하며 일회성 사용으로 설계되어있다. (Pod의 IP주소는 실행 될 때마다 변경 된다)
- ▶ Pod 내부 Container들은 Pod에 부여 된 네트워크 정보 (IP주소) 및 볼륨 정보등을 공유한다.
- ▶ Pod 내부 Container들간의 통신은 localhost로 통신이 가능하며 내부 Container간 Port번호 충돌에 주의해야 한다.
- ▶ Pod 내부 Container에는 동일 클러스터 내부에서만 접속이 가능하며, 외부접속을 허용하기위해 Service 오브젝트를 함께 사용한다.
- ▶ Pod는 단독으로 사용 될 수 있지만, 주로 쿠버네티스의 다른 오브젝트와 함께 사용된다. (Service, Controller, Volume)

◎ kubernetes ReplicaSet

- ▶ ReplicaSet은 Pod의 복제본(Replicas)을 생성하고 관리하는 오브젝트로 Pod의 Self-Healing을 구현 할 수 있다.
- ▶ ReplicaSet을 이용하여 Pod의 복제본 수를 정의 할 수 있으며, 정해진 수 만큼의 Pod가 항상 유지되도록 관리한다.
- ▶ ReplicaSet 오브젝트는 Replication Controller에 의해 관리되며 동작한다.
- ▶ ReplicaSet은 노드 장애가 발생 할 경우 장애가 발생 한 노드에 생성 되어있던 Pod의 수만큼 대체 노드에서 Pod를 재 생성한다.
- ▶ ReplicaSet을 이용하여 관리자 개입이 없는 Software 스스로 내결함성을 갖는 Fault Tolerance 환경을 구축 할 수 있다.
- ▶ ReplicaSet 오브젝트를 단독으로 생성하여 사용 할 수 있지만 주로 배포 Controller Deployment에 포함시켜 함께 사용된다.

Kubernetes Pod & ReplicaSet



Kubernetes Pod & ReplicaSet (EX.1 : Basic)

< 작업대상 : pod-basic.yml >

```
apiVersion: v1                # API Resource Version을 정의 ( 공식문서를 참조하여 작성 )
kind: Pod                     # Manifest File을 이용하여 쿠버네티스 클러스터에 생성 할 오브젝트 종류를 정의
metadata:                     # Label, Annotation, Object Name등과 같은 Meta Data 정보를 정의
  name: my-pod
spec:                          # Manifest File을 이용하여 생성 할 오브젝트의 상세 정보를 정의
  containers:                  # Pod의 Container 속성을 정의한다.
  - name: my-nginx-container   # Container의 이름 정의
    image: nginx:latest        # Container에서 사용 할 이미지 정의
    ports:
      - containerPort: 80      # Container에서 사용 할 포트번호 정의
        protocol: TCP          # Container 포트의 프로토콜 종류 정의
  resources:                   # 컨테이너의 자원 사용률등을 제어할 때 사용 ( 이후 생략 )
    limits:                    # RAM:100MB, CPU:0.5코어
      memory: "100Mi"
      cpu: "500Mi"
```


Kubernetes Pod & ReplicaSet (EX.1 : Basic)

```
$ kubectl apply -f pod-basic.yml
```

```
pod/my-pod created
```

```
$ kubectl get pod
```

NAME	READY	STATUS	RESTARTS	AGE
my-pod	1/1	Running	0	7s

```
$ kubectl get pod -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS GATES
my-pod	1/1	Running	0	37m	192.168.10.58	node-1	<none>	<none>

```
$ kubectl describe pod my-pod
```

```
Name:          my-pod
```

```
Namespace:     default
```

```
Node:          ip-192-168-10-147.ap-northeast-2.compute.internal/192.168.10.147
```

```
Start Time:    Wed, 13 Sep 2023 13:32:16 +0900
```

```
Annotations:   kubernetes.io/psp: eks.privileged
```

```
IP:            192.168.10.58
```

Kubernetes Pod & ReplicaSet (EX.1 : Basic)

```
$ kubectl run -i --rm --tty debug --image=busybox -- sh
```

```
/ # wget 192.168.10.58
```

```
/ # cat index.html
```

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<title>Welcome to nginx!</title>
```

```
</head>
```

```
<body>
```

```
<h1>Welcome to nginx!</h1>
```

```
... 생략 ...
```

```
/ # exit ( debug Pod Auto Delete )
```

```
$ kubectl delete pod my-pod
```

```
pod "my-pod" deleted
```

```
$ kubectl get pod
```

```
No resources found in default namespace.
```

Kubernetes Pod & ReplicaSet (EX.2 : ENV)

< 작업대상 : pod-env.yml >

```
apiVersion: v1
kind: Pod
metadata:
  name: env-pod # Pod 이름
  labels:
    app: env-pod # Pod 라벨 정의 ( 오브젝트 식별 정보 )
spec:
  containers:
  - name: env-con
    image: ubuntu:bionic
    env: # env 속성 : Container 환경변수 정의
      - name: MyName # 환경변수의 이름
        value: kube # 환경변수의 값
      - name: HelloMessage
        value: Hello $(MyName)
```

```
- name: NodeName
  valueFrom: # 환경변수의 값을 외부에서 참조
    fieldRef: # Pod 생성 시 정의되는 Pod의 정보를 불러와 값으로 사용
      fieldPath: spec.nodeName # Pod의 spec 정보를 참조
- name: Namespace
  valueFrom:
    fieldRef:
      fieldPath: metadata.namespace # Pod의 metadata 정보를 참조
- name: NodeIP
  valueFrom:
    fieldRef:
      fieldPath: status.hostIP # Pod의 status 정보를 참조
- name: PodIP
  valueFrom:
    fieldRef:
      fieldPath: status.podIP
command: ['sh', '-c', 'echo The app is running! && sleep 3600']
# 컨테이너 종료방지 ( 컨테이너는 실행 프로세스가 없을 경우 종료 )
```

Kubernetes Pod & ReplicaSet (EX.2 : ENV)

```
$ kubectl apply -f ./pod-env.yml
```

```
pod/env-pod created
```

```
$ kubectl get pod -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS GATES
env-pod	1/1	Running	0	12s	192.168.10.14	node-1	<none>	<none>

```
$ kubectl exec env-pod -- env
```

```
PodIP=192.168.10.14
```

```
NodeIP=192.168.10.147
```

```
MyName=Kube
```

```
HelloMessage=Hello Kube
```

```
$ kubectl exec -it env-pod -- bash
```

```
root@env-pod:/# env | grep IP
```

```
PodIP=192.168.10.14
```

```
$ kubectl delete pod env-pod
```

```
pod "env-pod" deleted
```

Kubernetes Pod & ReplicaSet (EX.3 : SideCar / Net)

< 작업대상 : pod-sidecar-net.yml >

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-sidecar # Pod 이름
  labels:
    name: pod-sidecar # Pod 라벨 정의
spec:
  containers: # 하나의 Pod에 2개의 Container를 정의
  - name: nginx-sidecar-app # 첫 번째 Container 정의
    image: nginx:latest
    ports:
      - containerPort: 80
  - name: busybox-sidecar-app # 두 번째 Container 정의
    image: busybox:latest # BusyBox: Debugging Linux System
    command: ['sh', '-c', 'echo The app is running! && sleep 3600']
```

< 작업대상 : pod-net.yml >

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-net # 네트워크 통신 테스트용 Pod
  labels:
    name: pod-net
spec:
  containers:
  - name: nginx-app
    image: nginx:latest
    ports:
      - containerPort: 80
```

Kubernetes Pod & ReplicaSet (EX.3 : SideCar / Net)

```
$ kubectl apply -f pod-sidecar-net.yml
```

```
pod/sidecar-pod created
```

```
$ kubectl apply -f pod-net.yml
```

```
pod/pod-net created
```

```
$ kubectl get pod -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED	NODE	READINESS	GATES
pod-net	1/1	Running	0	74m	192.168.20.181	node-1	<none>		<none>	
pod-sidecar	2/2	Running	0	74m	192.168.10.58	node-2	<none>		<none>	

■ pod-net의 경우 Pod IP를 192.168.20.181을 부여받았으며 Node-1(Worker Node-1)번에 생성 되었다.

■ pod-sidecar의 경우 Pod IP를 192.168.10.58을 부여받았으며 Node-2(Worker Node-2)번에 생성 되었다.

■ 생성 된 Pod의 정보를 확인 (pod-sidecar의 READY 영역을 확인 2/2)

■ 실제 노드이름 : "ip-192-168-10-147.ap-northeast-2.compute.internal"

Kubernetes Pod & ReplicaSet (EX.3 : SideCar / Net)

```
$ kubectl exec -it pod-sidecar -c busybox-sidecar-app -- sh
```

```
/ # wget -Sq localhost:80    ■ 동일 Pod-sidecar에 구성 된 nginx-sidecar-app Container와 통신
```

```
HTTP/1.1 200 OK
```

```
Server: nginx/1.23.0
```

```
Date: Tue, 12 Jul 2022 03:43:21 GMT
```

```
/ # rm -rf ./index.html    ■ 다음 통신 테스트를 위해 index.html 파일 삭제
```

```
/ # wget -Sq 192.168.20.181:80    ■ Pod-net에 구성 된 nginx-app Container와 통신 ( 다른 Pod와 통신 시 Pod IP 사용 )
```

```
HTTP/1.1 200 OK
```

```
Server: nginx/1.23.0
```

```
Date: Tue, 12 Jul 2022 04:09:44 GMT
```

```
$ kubectl logs pod-net nginx-app
```

```
192.168.10.58 - - [12/Jul/2022:04:09:44 +0000] "GET / HTTP/1.1" 200 615 "-" "Wget" "-"
```

```
$ kubectl delete pod --all --namespace default
```

```
pod "pod-net" deleted
```

```
pod "pod-sidecar" deleted
```

Kubernetes Pod & ReplicaSet (EX.4 : Label & Selector)

< 작업대상 : pod-label-test1.yml >

```
apiVersion: v1
kind: Pod
metadata:
  name: label-app-1 # Pod 이름
  labels:
    group: web # Pod 라벨 정의
    app: app-1
    version: v1
    env: prod
spec:
  containers:
    - name: webapp-1
      image: nginx:latest
      ports:
        - containerPort: 80
```

< 작업대상 : pod-label-test2.yml >

```
apiVersion: v1
kind: Pod
metadata:
  name: label-app-2 # Pod 이름
  labels:
    group: web # Pod 라벨 정의
    app: app-2
    version: v1
    env: stage
spec:
  containers:
    - name: webapp-2
      image: nginx:latest
      ports:
        - containerPort: 80
```


Kubernetes Pod & ReplicaSet (EX.4 : Label & Selector)

< 작업대상 : pod-label-test3.yml >

```
apiVersion: v1
kind: Pod
metadata:
  name: label-app-3 # Pod 이름
  labels:
    group: web # Pod 라벨 정의
    app: app-3
    version: v1
    env: test
spec:
  containers:
  - name: webapp-3
    image: nginx:latest
    ports:
      - containerPort: 80
```

< 작업대상 : pod-label-test4.yml >

```
apiVersion: v1
kind: Pod
metadata:
  name: label-app-4 # Pod 이름
spec:
  containers:
  - name: webapp-4
    image: nginx:latest
    ports:
      - containerPort: 80
```

Kubernetes Pod & ReplicaSet (EX.4 : Label & Selector)

\$ kubectl apply -f label ■ Manifest File 전체 선택 (Directory 지정)

pod/label-app-1 created pod / label-app-2 created

pod/label-app-3 created pod / label-app-4 created

■ [Label Select]

\$ kubectl get pod --show-labels ■ 전체 Label 목록 조회

NAME	READY	STATUS	RESTARTS	AGE	LABELS
label-app-1	1/1	Running	0	108s	app=app-1,env=prod,group=web,version=v1
label-app-2	1/1	Running	0	108s	app=app-2,env=stage,group=web,version=v1
label-app-3	1/1	Running	0	108s	app=app-3,env=test,group=web,version=v1
label-app-4	1/1	Running	0	108s	<none>

\$ kubectl get pod -L app,group,env ■ 특정 Label 목록 조회 ("-L" : --label-columns)

NAME	READY	STATUS	RESTARTS	AGE	APP	GROUP	ENV
label-app-1	1/1	Running	0	6m58s	app-1	web	prod
label-app-2	1/1	Running	0	6m58s	app-2	web	stage
label-app-3	1/1	Running	0	6m58s	app-3	web	test
label-app-4	1/1	Running	0	6m58s	<none>		

Kubernetes Pod & ReplicaSet (EX.4 : Label & Selector)

■ [Label Create, Update, Delete]

\$ kubectl label pod label-app-4 app=app-4 ■ Label Create

\$ kubectl get pod label-app-4 --show-labels

NAME	READY	STATUS	RESTARTS	AGE	LABELS
label-app-4	1/1	Running	0	3m27s	app=app-4

\$ kubectl label pod label-app-4 app=app-test --overwrite ■ Label Update

\$ kubectl get pod label-app-4 --show-labels

NAME	READY	STATUS	RESTARTS	AGE	LABELS
label-app-4	1/1	Running	0	5m48s	app=app-test

\$ kubectl label pod label-app-4 app- ■ Label Delete

\$ kubectl get pod label-app-4 --show-labels

NAME	READY	STATUS	RESTARTS	AGE	LABELS
label-app-4	1/1	Running	0	10m	<none>

Kubernetes Pod & ReplicaSet (EX.4 : Label & Selector)

■ [Label Selector]

```
$ kubectl get pod --selector env=prod
```

NAME	READY	STATUS	RESTARTS	AGE
label-app-1	1/1	Running	0	13m

```
$ kubectl get pod --selector env=stage
```

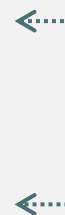
NAME	READY	STATUS	RESTARTS	AGE
label-app-2	1/1	Running	0	13m

```
$ kubectl get pod --selector env!=prod
```

NAME	READY	STATUS	RESTARTS	AGE
label-app-2	1/1	Running	0	15m
label-app-3	1/1	Running	0	15m
label-app-4	1/1	Running	0	15m

```
$ kubectl get pod --selector '!env'
```

NAME	READY	STATUS	RESTARTS	AGE
label-app-4	1/1	Running	0	19m



■ 지정 된 ENV Label의 값과 일치하는 Pod를 조회

■ 지정 된 ENV Label의 값이 아닌 Pod를 조회

■ 지정 된 ENV Label 갖지 않는 Pod를 조회

Kubernetes Pod & ReplicaSet (EX.4 : Label & Selector)

■ [Label Selector]

```
$ kubectl get pod --selector group=web,version=v1
```

NAME	READY	STATUS	RESTARTS	AGE
label-app-1	1/1	Running	0	21m
label-app-2	1/1	Running	0	21m
label-app-3	1/1	Running	0	21m

■ group의 값이 web이고, version의 값이 v1인 Pod 조회 (AND)

```
$ kubectl get pod --selector env!=prod,group=web
```

NAME	READY	STATUS	RESTARTS	AGE
label-app-2	1/1	Running	0	23m
label-app-3	1/1	Running	0	23m

■ env의 값이 prod가 아니고 group의 값은 web인 Pod 조회 (AND)

```
$ kubectl get pod --selector 'env in (test,stage)'
```

NAME	READY	STATUS	RESTARTS	AGE
label-app-2	1/1	Running	0	25m
label-app-3	1/1	Running	0	25m

■ env의 값이 test이거나 stage인 Pod 조회 (OR)

Kubernetes Pod & ReplicaSet (EX.4 : Label & Selector)

■ [Label Selector]

```
$ kubectl get pod --selector 'env notin (test,stage)'
```

NAME	READY	STATUS	RESTARTS	AGE	■ env의 값이 test이거나 stager가 아닌 Pod 조회 (OR)
label-app-1	1/1	Running	0	26m	
label-app-4	1/1	Running	0	26m	

```
$ kubectl get pod --selector 'group=web,version,env in (prod,stage)'
```

NAME	READY	STATUS	RESTARTS	AGE	■ group의 값이 web이고 version 키가 존재하며 env의 값은 prod이거나 stage인 Pod를 조회 (AND, OR 혼합)
label-app-1	1/1	Running	0	32m	
label-app-2	1/1	Running	0	32m	

```
$ kubectl delete pod --all
```

```
pod "label-app-1" deleted
```

```
pod "label-app-2" deleted
```

```
pod "label-app-3" deleted
```

```
pod "label-app-4" deleted
```

Kubernetes Pod & ReplicaSet (EX.5 : NodeSelector)

```
$ kubectl get node
```

NAME	STATUS	ROLES	AGE	VERSION
ip-192-168-10-147.ap-northeast-2.compute.internal	Ready	<none>	4h46m	v1.24.16-eks-8ccc7ba
ip-192-168-20-156.ap-northeast-2.compute.internal	Ready	<none>	4h46m	v1.24.16-eks-8ccc7ba

```
$ kubectl get node --show-labels
```

NAME	STATUS	ROLES	AGE	VERSION	LABELS
ip-192-168-10-147.ap-northeast-2.compute.internal	Ready	<none>	47h	v1.19.16	kubernetes.io/hostname=ip-192-168-10-147.ap-northeast-2.compute.internal ... 생략 ...
ip-192-168-20-156.ap-northeast-2.compute.internal	Ready	<none>	47h	v1.19.16	kubernetes.io/hostname=ip-192-168-20-156.ap-northeast-2.compute.internal ... 생략 ...

- Kubernetes Cluster를 구성하는 Node List를 확인 후 기본 Label 정보 확인
- Kubernetes Cluster 시 각 Node를 식별하는 Label이 이미 정의되어있는 것을 확인 할 수 있다.
- 실제 Label 정보는 Node를 구성하는 정보를 다양하게 포함하고 있다. (Instance Type, AMI, node-group 정보 등등)

Kubernetes Pod & ReplicaSet (EX.5 : NodeSelector)

```
$ kubectl label node ip-192-168-10-147.ap-northeast-2.compute.internal project=django
```

```
node/ip-192-168-10-147.ap-northeast-2.compute.internal labeled
```

```
$ kubectl label node ip-192-168-20-156.ap-northeast-2.compute.internal project=node
```

```
node/ip-192-168-20-156.ap-northeast-2.compute.internal labeled
```

```
$ kubectl get node -L project
```

NAME	STATUS	ROLES	AGE	VERSION	PROJECT
ip-192-168-10-147.ap-northeast-2.compute.internal	Ready	<none>	4h54m	v1.24.16-eks-8ccc7ba	django
ip-192-168-20-156.ap-northeast-2.compute.internal	Ready	<none>	4h55m	v1.24.16-eks-8ccc7ba	node

- 미리 정의 된 Label의 Key값은 사용하기 힘들어 별도의 사용자 지정 Label을 정의하여 각 Node를 식별 할 수 있도록 한다.
- Node-1에는 project=django Label을 정의하고, Node-2에는 project=node Label을 정의하여 각 노드를 식별할 수 있도록 한다.
- 위에서 정의 된 각 Node를 식별 할 수 있는 고유 Label을 활용하여 Pod가 생성 될 Node를 관리자가 직접 지정 할 수 있다.

Kubernetes Pod & ReplicaSet (EX.5 : NodeSelector)

```
$ kubectl run django-app-1 --labels="Framework=django" --image=nginx:latest ----  
overrides='{ "spec":{ "nodeSelector":{ "project":"django" }}}'
```

```
$ kubectl run django-app-2 --labels="Framework=django" --image=nginx:latest ----  
overrides='{ "spec":{ "nodeSelector":{ "project":"django" }}}'
```

```
$ kubectl run django-app-3 --labels="Framework=django" --image=nginx:latest ----  
overrides='{ "spec":{ "nodeSelector":{ "project":"django" }}}'
```

pod/django-app-1 created / pod/django-app-2 created / pod/django-app-3 created

```
$ kubectl run node-app-1 --labels="Framework=node" --image=nginx:latest ----  
overrides='{ "spec":{ "nodeSelector":{ "project":"node" }}}'
```

```
$ kubectl run node-app-2 --labels="Framework=node" --image=nginx:latest ----  
overrides='{ "spec":{ "nodeSelector":{ "project":"node" }}}'
```

```
$ kubectl run node-app-3 --labels="Framework=node" --image=nginx:latest ----  
overrides='{ "spec":{ "nodeSelector":{ "project":"node" }}}'
```

pod/node-app-1 created / pod/node-app-2 created / pod/node-app-3 created

▣ nodeSelector가 지정 된 Pod 생성 (--overrides: JSON 형식의 오브젝트 속성값 정의)

Kubernetes Pod & ReplicaSet (EX.5 : NodeSelector)

```
$ kubectl get pod -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED	NODE	READINESS	GATES
django-app-1	1/1	Running	0	92s	192.168.10.149	node-1	<none>		<none>	
django-app-2	1/1	Running	0	92s	192.168.10.156	node-1	<none>		<none>	
django-app-3	1/1	Running	0	92s	192.168.10.58	node-1	<none>		<none>	
node-app-1	1/1	Running	0	55s	192.168.20.181	node-2	<none>		<none>	
node-app-2	1/1	Running	0	55s	192.168.20.169	node-2	<none>		<none>	
node-app-3	1/1	Running	0	54s	192.168.20.179	node-2	<none>		<none>	

```
$ kubectl delete pod --all
```

```
pod "django-app-1" deleted
```

```
pod "django-app-2" deleted
```

```
pod "django-app-3" deleted
```

```
pod "node-app-1" deleted
```

```
pod "node-app-2" deleted
```

```
pod "node-app-3" deleted
```

▣ nodeSelector로 지정 된 노드에서 Pod를 생성한것을 확인 (django-app : node-1 / node-app : node-2) 후 Pod 오브젝트를 삭제한다.

Kubernetes Pod & ReplicaSet (EX.5 : NodeSelector)

< 작업대상 : nodeselector-test1.yml >

```
apiVersion: v1
kind: Pod
metadata:
  name: django-app # Pod 이름
  labels:
    framework: django # Pod 라벨 정의
spec:
  nodeSelector:
    project: django # 해당 Pod가 생성 될 Node의 Label 정의
  containers:
  - name: django-app
    image: nginx:latest
```

< 작업대상 : nodeselector-test2.yml >

```
apiVersion: v1
kind: Pod
metadata:
  name: node-app # Pod 이름
  labels:
    framework: node # Pod 라벨 정의
spec:
  nodeSelector:
    project: node # 해당 Pod가 생성 될 Node의 Label 정의
  containers:
  - name: node-app
    image: nginx:latest
```

Kubernetes Pod & ReplicaSet (EX.5 : NodeSelector)

```
$ kubectl apply -f pod/nodeselector
```

```
$ kubectl get pod -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS GATES
django-app	1/1	Running	0	32s	192.168.10.156	node-1	<none>	<none>
node-app	1/1	Running	0	32s	192.168.20.117	node-2	<none>	<none>

```
$ kubectl delete pod --all
```

```
$ kubectl apply -f pod/nodeselector
```

■ 항상 동일한 Node에 Pod가 생성되는 것을 확인한다.

```
$ kubectl get pod -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS GATES
django-app	1/1	Running	0	32s	192.168.10.58	node-1	<none>	<none>
node-app	1/1	Running	0	32s	192.168.20.179	node-2	<none>	<none>

```
$ kubectl delete pod --all
```

■ 각 Node에 정의한 project Label을 삭제한다.

```
$ kubectl label node ip-192-168-10-147.ap-northeast-2.compute.internal project-
```

```
node/node-1 labeled
```

```
$ kubectl label node ip-192-168-20-156.ap-northeast-2.compute.internal project-
```

```
node/node-2 labeled
```

Kubernetes Pod & ReplicaSet (EX.6 : NodeAffinity)

■ Node Affinity

>. requiredDuringSchedulingIgnoredDuringExecution (**Hard**)

>. preferredDuringSchedulingIgnoredDuringExecution (**Soft**)

- Required : 제약조건을 반드시 만족하는 Node에 Pod를 할당
- Preferred : 제약조건을 만족하는 Node를 우선, 반드시 만족하지않아도 Pod를 할당
- DOCS URL: <https://kubernetes.io/docs/concepts/scheduling-eviction/assign-pod-node>

\$ kubectl get nodes

NAME	STATUS	ROLES	AGE	VERSION
ip-192-168-10-147.ap-northeast-2.compute.internal	Ready	<none>	8d	v1.24.16-eks-8ccc7ba
ip-192-168-20-156.ap-northeast-2.compute.internal	Ready	<none>	8d	v1.24.16-eks-8ccc7ba

\$ kubectl get nodes --show-labels | grep failure-domain.beta.kubernetes.io/zone

ip-192-168-10-147	Ready	<none>	7d9h	v1.24.16-eks-8ccc7ba	topology.kubernetes.io/zone=ap-northeast-2a
ip-192-168-20-156	Ready	<none>	7d9h	v1.24.16-eks-8ccc7ba	topology.kubernetes.io/zone=ap-northeast-2c

■ topology.kubernetes.io/zone=ap-northeast-2a Label 활용 (192-168-10-147)

■ topology.kubernetes.io/zone=ap-northeast-2c Label 활용 (192-168-20-156)

Kubernetes Pod & ReplicaSet (EX.6 : NodeAffinity)

< 작업대상 : nodeaffinity-test1.yml >

spec: # 전체 내용은 TXT 교안 참조

affinity:

nodeAffinity:

requiredDuringSchedulingIgnoredDuringExecution: # Required (Hard)

nodeSelectorTerms:

- matchExpressions:

- key: topology.kubernetes.io/zone

operator: In # Operator Option : In, NotIn, Exists, DoesNotExist, Gt, Lt

values:

- ap-northeast-2a

\$ kubectl apply -f pod/nodeaffinity/nodeaffinity-test1.yml && kubectl get pod -o wide

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS GATES
node-app	1/1	Running	0	14s	192.168.10.231	ip-192-168-10-147	<none>	<none>

\$ kubectl delete pod node-app

▣ "topology.kubernetes.io/zone=ap-northeast-2a"를 만족하는 Node에서 Pod가 배포 된 것을 확인 후 삭제

Kubernetes Pod & ReplicaSet (EX.6 : NodeAffinity)

< 작업대상 : nodeaffinity-test1.yml >

spec: # 전체 내용은 TXT 교안 참조

affinity:

nodeAffinity:

requiredDuringSchedulingIgnoredDuringExecution: # Required (Hard)

nodeSelectorTerms:

- matchExpressions:

- key: topology.kubernetes.io/zone

operator: In # Operator Option : In, NotIn, Exists, DoesNotExist, Gt, Lt

values:

- ap-northeast-2b # ap-northeast-2a -> ap-northeast-2b 수정

\$ kubectl apply -f pod/nodeaffinity/nodeaffinity-test1.yml && kubectl get pod -o wide

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS GATES
node-app	0/1	Pending	0	4s	<none>	<none>	<none>	<none>

\$ kubectl delete pod node-app

▣ "topology.kubernetes.io/zone=ap-northeast-2b"를 만족하는 Node가 존재하지 않을 경우 Pod는 배포되지 않는다.

Kubernetes Pod & ReplicaSet (EX.6 : NodeAffinity)

< 작업대상 : nodeaffinity-test2.yml >

spec: # 전체 내용은 TXT 교안 참조

affinity:

nodeAffinity:

preferredDuringSchedulingIgnoredDuringExecution: # Preferred (Soft)

nodeSelectorTerms:

- matchExpressions:

- key: topology.kubernetes.io/zone

operator: In # Operator Option : In, NotIn, Exists, DoesNotExist, Gt, Lt

values:

- ap-northeast-2c

\$ kubectl apply -f pod/nodeaffinity/nodeaffinity-test2.yml && kubectl get pod -o wide

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS GATES
node-app	1/1	Running	0	14s	192.168.20.185	ip-192-168-20-156	<none>	<none>

\$ kubectl delete pod node-app

▣ "topology.kubernetes.io/zone=ap-northeast-2c"를 만족하는 Node에서 Pod가 배포 된 것을 확인 후 삭제

Kubernetes Pod & ReplicaSet (EX.6 : NodeAffinity)

< 작업대상 : nodeaffinity-test2.yml >

spec: # 전체 내용은 TXT 교안 참조

affinity:

nodeAffinity:

preferredDuringSchedulingIgnoredDuringExecution: # Preferred (Soft)

nodeSelectorTerms:

- matchExpressions:

- key: topology.kubernetes.io/zone

operator: In # Operator Option : In, NotIn, Exists, DoesNotExist, Gt, Lt

values:

- ap-northeast-2b # ap-northeast-2c -> ap-northeast-2b 수정

\$ kubectl apply -f pod/nodeaffinity/nodeaffinity-test2.yml && kubectl get pod -o wide

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS GATES
node-app	1/1	Running	0	14s	192.168.10.169	ip-192-168-10-147	<none>	<none>

\$ kubectl delete pod node-app

▣ "topology.kubernetes.io/zone=ap-northeast-2b"를 만족하는 Node가 존재하지 않아도 다른 Node에 Pod가 배포된다.

Kubernetes Pod & ReplicaSet (EX.7 : Taints & Tolerations)

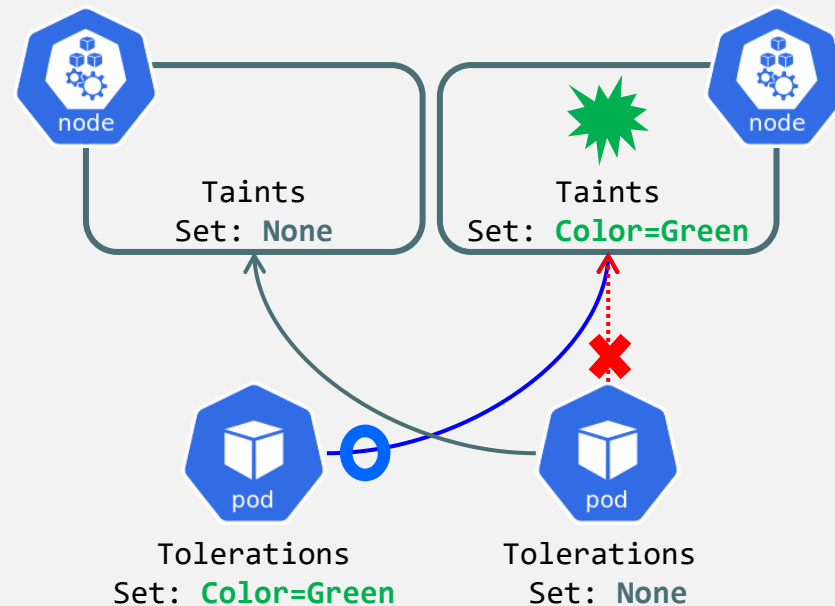
■ Taints (얼룩)

- >. Taint가 정의 된 Node에는 Pod를 배포 할 수 없다.
- >. Taint는 Node에 정의하며 "**<KEY>=<VALUE>:<EFFECT>**" 형식으로 정의된다.
- >. Taint EFFECT Option 3가지
 - # "**NoSchedule**" : Node에 Pod를 Scheduling 하지 않음
 - # "**NoExecute**" : Node에 Pod의 실행조차 허용하지 않음
 - # "**PreferNoSchedule**" : Node에 가능하면 Scheduling 하지 않음
- >. "NoSchedule" Option의 경우 기존에 실행중인 Pod에는 영향을 주지 않는다.
- >. "NoExecute" Option의 경우 기존에 실행중인 Pod를 강제로 종료시킨다.

■ Tolerations (용인)

- >. 특정 Taint가 정의 된 Node에서 Pod를 배포하기 위해서는 Tolerations를 설정해야한다.
- >. Tolerations는 Pod에 정의하며 "**<KEY>/<VALUE>/<EFFECT>/<OPERATOR>**"를 정의할 수 있다.
- >. "**<KEY>/<VALUE>/<EFFECT>**"는 Taint가 정의 된 Node를 지정하기위해 사용된다.
- >. "**<OPERATOR>**"는 "**Equal**"(Taint 정보와 일치), "**Exists**"(와일드카드)를 사용 할 수 있다.
 - # "**Equal**" : "<KEY>/<VALUE>/<EFFECT>"에 지정 된 정보와 일치하는 Taint를 갖는 Node를 지정
 - # "**Exists**" : "<KEY>/<VALUE>/<EFFECT>"에 대한 와일드 카드로 활용

EX:) KEY(X)/value(X)/EFFECT(NoSchedule)/Exists : Taint effect "NoSchedule"를 갖는 모든 Node를 지정



Kubernetes Pod & ReplicaSet (EX.7 : Taints & Tolerations)

```
$ kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
ip-192-168-10-147.ap-northeast-2.compute.internal	Ready	<none>	8d	v1.24.16-eks-8ccc7ba
ip-192-168-20-156.ap-northeast-2.compute.internal	Ready	<none>	8d	v1.24.16-eks-8ccc7ba

```
$ kubectl taint nodes ip-192-168-20-156.ap-northeast-2.compute.internal nodename=node2:NoSchedule
node/ip-192-168-20-156.ap-northeast-2.compute.internal tainted
```

```
$ kubectl describe nodes ip-192-168-20-156.ap-northeast-2.compute.internal | grep Taints
```

Name: ip-192-168-20-156.ap-northeast-2.compute.internal

CreationTimestamp: Wed, 13 Sep 2023 01:07:38 +0000

Taints: **nodename=node2:NoSchedule**

Unschedulable: false

Addresses:

InternalIP: 192.168.20.156

Hostname: ip-192-168-20-156.ap-northeast-2.compute.internal

InternalDNS: ip-192-168-20-156.ap-northeast-2.compute.internal

▣ ip-192-168-20-156.ap-northeast-2.compute.internal Node에 Taints 설정 및 확인

Kubernetes Pod & ReplicaSet (EX.7 : Taints & Tolerations)

< 작업대상 : taints-test1.yml >

```
apiVersion: v1
kind: Pod
metadata:
  name: node-app1 # Pod 이름
  labels:
    framework: node-app1 # Pod 라벨 정의
spec:
  containers:
  - name: node-app1
    image: nginx:latest
    resources:
      limits:
        memory: "100Mi"
        cpu: "500m"
```

< 작업대상 : taints-test2.yml >

```
apiVersion: v1
kind: Pod
metadata:
  name: node-app # Pod 이름
  labels:
    framework: node # Pod 라벨 정의
spec:
  containers:
  - name: node-app
    image: nginx:latest
    resources:
      limits:
        memory: "100Mi"
        cpu: "500m"
```

Kubernetes Pod & ReplicaSet (EX.7 : Taints & Tolerations)

```
$ kubectl apply -f pod/taints
```

```
pod/node-app1 created
```

```
pod/node-app2 created
```

```
$ kubectl get pod -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS GATES
node-app1	1/1	Running	0	14s	192.168.10.169	ip-192-168-10-147	<none>	<none>
node-app2	1/1	Running	0	14s	192.168.10.156	ip-192-168-10-147	<none>	<none>

■ 2개의 Pod가 전부 Taints 설정이 없는 첫번째 Worker Node에 배포 된 것을 확인한다.

```
$ kubectl delete -f pod/taints
```

```
pod "node-app1" deleted
```

```
pod "node-app2" deleted
```

■ 다음 TEST를 위해 생성 한 Pod 오브젝트를 삭제

Kubernetes Pod & ReplicaSet (EX.7 : Taints & Tolerations)

< 작업대상 : taints-test1.yml >

```
apiVersion: v1
kind: Pod
metadata:
  name: node-app1 # Pod 이름
  labels:
    framework: node-app1 # Pod 라벨 정의
spec:
  tolerations:
    - key: nodename
      value: node2
      effect: NoSchedule
      operator: Equal
  containers:
    - name: node-app1
      image: nginx:latest
```

< 작업대상 : taints-test2.yml >

```
apiVersion: v1
kind: Pod
metadata:
  name: node-app # Pod 이름
  labels:
    framework: node # Pod 라벨 정의
spec:
  tolerations:
    - key: nodename
      value: node2
      effect: NoSchedule
      operator: Equal
  containers:
    - name: node-app
      image: nginx:latest
```

Kubernetes Pod & ReplicaSet (EX.7 : Taints & Tolerations)

```
$ kubectl apply -f pod/taints
```

```
pod/node-app1 created
```

```
pod/node-app2 created
```

```
$ kubectl get pod -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS GATES
node-app1	1/1	Running	0	14s	192.168.20.117	ip-192-168-20-156	<none>	<none>
node-app2	1/1	Running	0	14s	192.168.20.169	ip-192-168-20-156	<none>	<none>

▣ Tolerations 영역에서 지정 된 Taint 조건에 맞는 Worker Node에 Pod가 배포되는 것을 확인

```
$ kubectl delete -f pod/taints
```

```
pod "node-app1" deleted
```

```
pod "node-app2" deleted
```

▣ 다음 TEST를 위해 생성 한 Pod 오브젝트를 삭제

Kubernetes Pod & ReplicaSet (EX.7 : Taints & Tolerations)

< 작업대상 : taints-test1.yml >

```
apiVersion: v1
kind: Pod
metadata:
  name: node-app1 # Pod 이름
  labels:
    framework: node-app1 # Pod 라벨 정의
spec:
  tolerations:
    - effect: NoSchedule
      operator: Exists
  containers:
    - name: node-app1
      image: nginx:latest
```

< 작업대상 : taints-test2.yml >

```
apiVersion: v1
kind: Pod
metadata:
  name: node-app # Pod 이름
  labels:
    framework: node # Pod 라벨 정의
spec:
  tolerations:
    - effect: NoSchedule
      operator: Exists
  containers:
    - name: node-app
      image: nginx:latest
```


Kubernetes Pod & ReplicaSet (EX.7 : Taints & Tolerations)

```
$ kubectl apply -f pod/taints
```

```
pod/node-app1 created
```

```
pod/node-app2 created
```

```
$ kubectl get pod -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS GATES
node-app1	1/1	Running	0	14s	192.168.20.137	ip-192-168-20-156	<none>	<none>
node-app2	1/1	Running	0	14s	192.168.20.185	ip-192-168-20-156	<none>	<none>

■ Tolerations 영역 수정 (operator "Exists" / key,value 삭제)

■ Effect: NoSchedule Taint 정보를 갖는 Worker Node에 Pod를 배포

```
$ kubectl delete -f pod/taints
```

```
pod "node-app1" deleted
```

```
pod "node-app2" deleted
```

```
$ kubectl taint nodes ip-192-168-20-156.ap-northeast-2.compute.internal nodename=node2:NoSchedule-
```

```
node/ip-192-168-20-156.ap-northeast-2.compute.internal untainted
```

■ 다음 TEST를 위해 생성 한 Pod 오브젝트 및 Worker Node에 지정한 Taints 정보를 삭제

Kubernetes Pod & ReplicaSet (EX.8 : cordon & drain)

■ Cordon (Node Pod Scheduling Disable)

>. Node의 Scheduling이 중단 되므로 해당 Node에는 Pod를 추가 배포 할 수 없다.

>. 해당 Node에서 이미 배포되어있던 Pod는 정상동작

\$ kubectl get nodes

NAME	STATUS	ROLES	AGE	VERSION
ip-192-168-10-147.ap-northeast-2.compute.internal	Ready	<none>	8d	v1.24.16-eks-8ccc7ba
ip-192-168-20-156.ap-northeast-2.compute.internal	Ready	<none>	8d	v1.24.16-eks-8ccc7ba

\$ kubectl cordon ip-192-168-20-156.ap-northeast-2.compute.internal

node/ip-192-168-20-156.ap-northeast-2.compute.internal cordoned

\$ kubectl get nodes

NAME	STATUS	ROLES	AGE	VERSION
ip-192-168-10-147.ap-northeast-2.compute.internal	Ready	<none>	8d	v1.24.16-eks-8ccc7ba
ip-192-168-20-156.ap-northeast-2.compute.internal	Ready, SchedulingDisabled	<none>	8d	v1.24.16-eks-8ccc7ba

\$ kubectl uncordon ip-192-168-20-156.ap-northeast-2.compute.internal

node/ip-192-168-20-156.ap-northeast-2.compute.internal uncordoned

Kubernetes Pod & ReplicaSet (EX.8 : cordon & drain)

■ Drain (Node Disable)

- >. Node의 상태를 비활성화 했으므로 해당 Node에는 Pod를 배포 할 수 없다.
- >. Multiple Pod 환경의 경우 비활성화 된 Node에 존재하던 Pod는 다른 활성 상태의 Node로 이동된다. (Replicas : 2)
- >. Single Pod 환경의 경우 비활성화 된 Node에 존재하던 Pod는 다른 활성 상태의 Node로 이동되지 않는다. (Replicas : 1)

```
$ kubectl drain ip-192-168-20-156.ap-northeast-2.compute.internal \
--ignore-daemonsets --delete-emptydir-data
```

```
evicting pod kube-system/efs-csi-controller-5cf794bfc4-t9vdx
```

```
evicting pod kube-system/aws-load-balancer-controller-8684f5455c-jps7h
```

```
evicting pod kube-system/coredns-dc4979556-59xzb
```

```
evicting pod kube-system/coredns-dc4979556-75lhc
```

```
$ kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
ip-192-168-10-147.ap-northeast-2.compute.internal	Ready	<none>	8d	v1.24.16-eks-8ccc7ba
ip-192-168-20-156.ap-northeast-2.compute.internal	Ready,SchedulingDisabled	<none>	8d	v1.24.16-eks-8ccc7ba

```
$ kubectl uncordon ip-192-168-20-156.ap-northeast-2.compute.internal
```

```
node/ip-192-168-20-156.ap-northeast-2.compute.internal uncordoned
```

Kubernetes Pod & ReplicaSet (EX.1 : ReplicaSet Basic)

< 작업대상 : rs-basic.yml >

```
apiVersion: apps/v1 # ReplicaSet은 apps/v1 버전을 사용
kind: ReplicaSet    # 오브젝트 종류 정의 ( ReplicaSet )
metadata:
  name: rs-basic     # ReplicaSet 이름
spec:
  replicas: 5
  selector:
    matchLabels:
      app: nginx-app
```

replicas 속성을 이용하여 복제 Pod의 수를 정의한다.
selector 속성에서 지정 된 Pod Template을 사용하여 Pod를 생성한다.
selector 속성은 template의 Label 정보를 사용하여 template을 선택한다.
matchLabel의 Label 정보와 template Label 정보는 반드시 일치해야한다.

template: # Pod Template

metadata:

name: nginx-template

labels:

app: nginx-app

Template Container

spec:

containers:

- name: nginx-con

image: nginx:latest

ports:

- containerPort: 80

Kubernetes Pod & ReplicaSet (EX.1 : ReplicaSet Basic)

```
$ kubectl apply -f replicaset/rs-basic.yml
```

```
replicaset.apps/rs-basic created
```

```
$ kubectl get rs rs-basic -o wide
```

NAME	DESIRED	CURRENT	READY	AGE	CONTAINERS	IMAGES	SELECTOR
rs-basic	5	5	5	94s	nginx-con	nginx:latest	app=nginx-app

```
$ kubectl get pod -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS GATES
rs-basic-c8w45	1/1	Running	0	36s	192.168.20.93	node-2	<none>	<none>
rs-basic-sfmtb	1/1	Running	0	36s	192.168.10.156	node-1	<none>	<none>
rs-basic-6zmhw	1/1	Running	0	36s	192.168.10.58	node-1	<none>	<none>
rs-basic-z6ns9	1/1	Running	0	36s	192.168.10.22	node-1	<none>	<none>
rs-basic-8pqhw	1/1	Running	0	36s	192.168.20.130	node-2	<none>	<none>

- Kubernetes에서는 Resource Name을 줄여서 표현 할 수 있다. (SHORTNAMES 확인 : `kubectl api-resources`)
- ReplicaSet 오브젝트 정보 확인 (DESIRED : 목표치 / CURRENT : 현재상태 / READY : 준비상태)
- replicas 속성에서 정의 한 수 만큼 Pod가 실행중인 것을 확인한다. (ReplicaSet Pod Name : ReplicaSet Name + Hash Value)

Kubernetes Pod & ReplicaSet (EX.1 : ReplicaSet Basic)

```
$ kubectl describe rs rs-basic
```

```
Name:          rs-basic
Selector:      app=nginx-app
Replicas:      5 current / 5 desired
Pods Status:   5 Running / 0 Waiting / 0 Succeeded / 0 Failed
Pod Template:
  Labels:  app=nginx-app
```

Events:

Type	Reason	Age	From	Message
----	-----	----	----	-----
Normal	SuccessfulCreate	5m26s	replicaset-controller	Created pod: rs-basic-c8w45
Normal	SuccessfulCreate	5m26s	replicaset-controller	Created pod: rs-basic-sfmtb
Normal	SuccessfulCreate	5m26s	replicaset-controller	Created pod: rs-basic-6zmhw
Normal	SuccessfulCreate	5m26s	replicaset-controller	Created pod: rs-basic-z6ns9
Normal	SuccessfulCreate	5m26s	replicaset-controller	Created pod: rs-basic-8pqhw

```
$ kubectl delete rs rs-basic
```

```
replicaset.apps "rs-basic" deleted
```

- ReplicaSet 오브젝트의 상세정보를 확인한다.
- 생략 된 정보이며 현재 작업에서 중요한 부분만 확인
- 오브젝트의 상세정보 확인 (kubectl describe)

Kubernetes Pod & ReplicaSet (EX.1 : ReplicaSet 관리영역)

```
$ kubectl run nginx-pod --labels="app=nginx-app" --image=nginx:latest
```

```
pod/nginx-pod created
```

```
$ kubectl get pod --show-labels
```

NAME	READY	STATUS	RESTARTS	AGE	LABELS
nginx-pod	1/1	Running	0	41s	app=nginx-app

▣ ReplicaSet 오브젝트 selector 속성에서 지정하는 Label가 동일한 Label 정보를 갖는 Pod를 생성한다. (단독 실행)

```
$ kubectl apply -f replicaset/rs-basic.yml
```

```
$ kubectl get pod -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS GATES
nginx-pod	1/1	Running	0	3m43s	192.168.10.240	node-2	<none>	<none>
rs-basic-ffw5m	1/1	Running	0	49s	192.168.20.89	node-1	<none>	<none>
rs-basic-mk8x4	1/1	Running	0	49s	192.168.10.58	node-2	<none>	<none>
rs-basic-pbxnz	1/1	Running	0	49s	192.168.20.169	node-1	<none>	<none>
rs-basic-tg5rr	1/1	Running	0	49s	192.168.10.156	node-1	<none>	<none>

▣ ReplicaSet 오브젝트 생성 후 전체 Pod 목록을 조회 (replicas 속성에서 정의 된 5개의 Pod가 정상적으로 유지되고 있다.)

▣ ReplicaSet 오브젝트를 생성하기전에 단독으로 실행 된 Pod가 포함되어있는 것을 확인한다.

Kubernetes Pod & ReplicaSet (EX.1 : ReplicaSet 관리영역)

```
$ kubectl describe rs rs-basic
```

Name: **rs-basic**

Selector: **app=nginx-app**

Replicas: **5 current / 5 desired**

Pods Status: **5 Running / 0 Waiting / 0 Succeeded / 0 Failed**

Pod Template:

Labels: **app=nginx-app**

Events:

Type	Reason	Age	From	Message
----	-----	----	----	-----
Normal	SuccessfulCreate	99s	replicaset-controller	Created pod: rs-basic-ffw5m
Normal	SuccessfulCreate	99s	replicaset-controller	Created pod: rs-basic-mk8x4
Normal	SuccessfulCreate	99s	replicaset-controller	Created pod: rs-basic-pbxnz
Normal	SuccessfulCreate	99s	replicaset-controller	Created pod: rs-basic-tg5rr

- ReplicaSet 오브젝트의 Events 기록 확인 (4개의 새로운 Pod만 생성한 것을 알 수 있다.)
- ReplicaSet 오브젝트는 Selector 속성에 지정 된 Label과 동일한 Label을 갖는 Pod는 자동으로 ReplicaSet 관리대상으로 포함한다.
- ReplicaSet 오브젝트 구성 시 주의할점 : 기존 운영중인 Pod의 Label 정보를 반드시 확인 후 ReplicaSet 오브젝트를 구성

Kubernetes Pod & ReplicaSet (EX.1 : ReplicaSet Pod 복구)

```
$ kubectl delete pod nginx-pod
```

```
$ kubectl describe rs rs-basic
```

Name: **rs-basic**

Selector: **app=nginx-app**

Replicas: **5 current / 5 desired**

Pods Status: **5 Running / 0 Waiting / 0 Succeeded / 0 Failed**

Pod Template:

Labels: **app=nginx-app**

Events:

Type	Reason	Age	From	Message
----	-----	----	----	-----
Normal	SuccessfulCreate	99s	replicaset-controller	Created pod: rs-basic-ffw5m
Normal	SuccessfulCreate	99s	replicaset-controller	Created pod: rs-basic-mk8x4
Normal	SuccessfulCreate	99s	replicaset-controller	Created pod: rs-basic-pbxnz
Normal	SuccessfulCreate	99s	replicaset-controller	Created pod: rs-basic-tg5rr
Normal	SuccessfulCreate	49m	replicaset-controller	Created pod: rs-basic-dcrkn

▣ 단독으로 생성 된 Pod를 삭제 할 경우 ReplicaSet 오브젝트는 새로운 Pod를 생성하여 5개의 Pod 수를 유지하는 것을 확인한다.

▣ 만약 Worker Node에 장애가 발생 한 경우 정상 Worker Node에서 요구치에 부족한 Pod 수만큼 새로 생성하여 요구치의 Pod 수를 유지하게 된다.

Kubernetes Pod & ReplicaSet (EX.1 : ReplicaSet 삭제)

```
$ kubectl delete rs rs-basic
```

```
replicaset.apps "rs-basic" deleted
```

```
$ kubectl get rs
```

```
No resources found in default namespace.
```

```
$ kubectl get pod
```

```
No resources found in default namespace.
```

▣ ReplicaSet 오브젝트가 삭제 될 경우 해당 ReplicaSet 오브젝트가 관리하는 Pod도 함께 삭제되는 것을 확인 할 수 있다.

```
$ kubectl apply -f replicaset/rs-basic.yml
```

```
replicaset.apps/rs-basic created
```

```
$ kubectl get rs
```

NAME	DESIRED	CURRENT	READY	AGE
rs-basic	5	5	5	7s

▣ 두 번째 삭제 방법 테스트를 위해 ReplicaSet 오브젝트를 재 생성한다.

Kubernetes Pod & ReplicaSet (EX.1 : ReplicaSet 삭제)

```
$ kubectl delete rs rs-basic --cascade=false
```

```
replicaset.apps "rs-basic" deleted
```

```
$ kubectl get rs
```

```
No resources found in default namespace.
```

```
$ kubectl get pod
```

NAME	READY	STATUS	RESTARTS	AGE
rs-basic-bthv6	1/1	Running	0	40s
rs-basic-cgqvt	1/1	Running	0	40s
rs-basic-gdhmg	1/1	Running	0	40s
rs-basic-k78bn	1/1	Running	0	40s
rs-basic-n7zh9	1/1	Running	0	40s

▣ cascade=false 옵션을 사용하여 ReplicaSet 오브젝트만 삭제 후 Pod 오브젝트는 삭제되지 않은 것을 확인한다.

```
$ kubectl delete pod --all
```

```
pod "rs-basic-bthv6" deleted / pod "rs-basic-cgqvt" deleted / pod "rs-basic-gdhmg" deleted
```

```
pod "rs-basic-k78bn" deleted / pod "rs-basic-n7zh9" deleted
```

Kubernetes Pod & ReplicaSet (EX.1 : ReplicaSet 삭제)

```
$ kubectl apply -f replicaset/rs-basic.yml
```

```
replicaset.apps/rs-basic created
```

```
$ kubectl get rs
```

NAME	DESIRED	CURRENT	READY	AGE
rs-basic	5	5	5	7s

```
$ kubectl get pod
```

NAME	READY	STATUS	RESTARTS	AGE
rs-basic-8g6df	1/1	Running	0	6s
rs-basic-8grsk	1/1	Running	0	6s
rs-basic-9n7fx	1/1	Running	0	6s
rs-basic-cx92w	1/1	Running	0	6s
rs-basic-rncrc	1/1	Running	0	6s

```
$ kubectl scale rs rs-basic --replicas 3
```

```
replicaset.apps/rs-basic scaled
```

▣ kubectl scale 명령어를 이용하여 ReplicaSet 오브젝트의 replicas 속성의 값을 변경 할 수 있다.

▣ ReplicaSet 삭제 시 Pod를 한번에 삭제하는 것이 아닌, 점차 수를 감소시키는 형태의 삭제 작업을 진행한다. (GraceFully 삭제)

Kubernetes Pod & ReplicaSet (EX.1 : ReplicaSet 삭제)

```
$ kubectl get pod
```

NAME	READY	STATUS	RESTARTS	AGE
rs-basic-8grsk	1/1	Running	0	25s
rs-basic-9n7fx	1/1	Running	0	25s
rs-basic-rncrc	1/1	Running	0	25s

■ Pod 오브젝트의 수가 5개에서 3개로 감소 된 것을 확인 할 수 있다.

```
$ kubectl edit rs rs-basic
```

```
  replicas: 2
```

```
replicaset.apps/rs-basic edited
```

```
$ kubectl get pod
```

NAME	READY	STATUS	RESTARTS	AGE
rs-basic-9n7fx	1/1	Running	0	76s
rs-basic-rncrc	1/1	Running	0	76s

■ kubectl edit 명령어를 이용하여 현재 생성 된 오브젝트의 정보를 변경 할 수 있다. (vim 편집기)

■ 현재 생성 된 ReplicaSet 오브젝트의 replicas 속성의 값을 3개에서 2개로 변경 후 Pod 오브젝트의 수를 확인한다.

■ 하나의 Pod 오브젝트가 추가로 Terminating 된 것을 확인 할 수 있다.

Kubernetes Pod & ReplicaSet (EX.1 : ReplicaSet 삭제)

```
$ kubectl scale rs rs-basic --replicas 0
```

```
replicaset.apps/rs-basic scaled
```

```
$ kubectl get pod
```

NAME	READY	STATUS	RESTARTS	AGE
rs-basic-rncrc	0/1	Terminating	0	8m23s

```
$ kubectl get pod
```

```
No resources found in default namespace.
```

```
$ kubectl get rs
```

NAME	DESIRED	CURRENT	READY	AGE
rs-basic	0	0	0	8m57s

▣ Replicaset 오브젝트의 replicas 속성의 값을 0으로 변경 할 경우 Pod 오브젝트는 전체 삭제 되지만 ReplicaSet 오브젝트는 남아있다.

```
$ kubectl delete rs rs-basic
```

```
replicaset.apps "rs-basic" deleted
```

```
$ kubectl get rs
```

```
No resources found in default namespace.
```

Kubernetes Pod & ReplicaSet (EX.1 : Template Update)

< 작업대상 : rs-update-rollback.yml >

apiVersion: apps/v1

kind: ReplicaSet

metadata:

name: rs-rollback # ReplicaSet 이름

spec:

replicas: 3

selector:

matchLabels:

app: node-app

Template Update & Rollback Test를 위한 Manifest File을 정의한다.

replicas => "3", Template selector => "node-app"

node:16-alpine3.15 Image를 사용한다.

Container 종료방지를 위한 command를 함께사용

template: # Pod Template

metadata:

name: node-template

labels:

app: node-app

spec: # Template Container

containers:

- name: node-con

image: node:16-alpine3.15

command: ['sh', '-c', 'tail -f /dev/null']

Kubernetes Pod & ReplicaSet (EX.1 : Template Update)

```
$ kubectl apply -f replicaset/rs-update-rollback.yml
```

```
replicaset.apps/rs-rollback created
```

```
$ kubectl get rs -o wide
```

NAME	DESIRED	CURRENT	READY	AGE	CONTAINERS	IMAGES	SELECTOR
rs-rollback	3	3	3	15s	node-con	node:16-alpine3.15	app=node-app

```
$ kubectl get pod -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS GATES
rs-rollback-pqnlb	1/1	Running	0	32s	192.168.10.58	node-2	<none>	<none>
rs-rollback-rlczz	1/1	Running	0	32s	192.168.10.248	node-1	<none>	<none>
rs-rollback-znkmg	1/1	Running	0	32s	192.168.20.169	node-1	<none>	<none>

▣ ReplicaSet 오브젝트 정보 확인 (DESIRED : 목표치 / CURRENT : 현재상태 / READY : 준비상태)

▣ replicas 속성에서 정의 한 수 만큼 Pod가 실행중인 것을 확인한다. (ReplicaSet Pod Name : ReplicaSet Name + Hash Value)

Kubernetes Pod & ReplicaSet (EX.1 : Template Update)

< 작업대상 : rs-update-rollback.yml >

apiVersion: apps/v1

kind: ReplicaSet

metadata:

name: rs-rollback

spec:

replicas: 3

selector:

matchLabels:

app: node-app

Template Update Test를 위해 새로운 Label 정보를 추가한다

template: # Pod Template

metadata:

name: node-template

labels:

app: node-app

version: v1

env: prod

spec: # Template Container

containers:

- name: node-con

image: node:16-alpine3.15

command: ['sh', '-c', 'tail -f /dev/null']

Kubernetes Pod & ReplicaSet (EX.1 : Template Update)

```
$ kubectl apply -f replicaset/rs-update-rollback.yml
```

```
replicaset.apps/rs-rollback configured
```

```
$ kubectl get pod --show-labels
```

NAME	READY	STATUS	RESTARTS	AGE	LABELS
rs-rollback-pqnlb	1/1	Running	0	3m53s	app=node-app
rs-rollback-rlczz	1/1	Running	0	3m53s	app=node-app
rs-rollback-znkmg	1/1	Running	0	3m53s	app=node-app

```
$ kubectl delete pod rs-rollback-znkmg
```

```
pod "rs-rollback-f9j2x" deleted
```

```
$ kubectl get pod --show-labels
```

NAME	READY	STATUS	RESTARTS	AGE	LABELS
rs-rollback-pqnlb	1/1	Running	0	5m41s	app=node-app
rs-rollback-rlczz	1/1	Running	0	5m41s	app=node-app
rs-rollback-sxb46	1/1	Running	0	52s	app=node-app,env=prod,version=v1

▣ Pod Template이 수정되어도 기존 Pod에는 변경사항이 반영되지 않는 것을 확인 할 수 있다.

▣ 기존 Pod를 삭제 후 재 생성 되는 Pod의 정보만 Pod Template 변경사항이 반영되었다.

Kubernetes Pod & ReplicaSet (EX.1 : Template Update)

< 작업대상 : rs-update-rollback.yml >

apiVersion: apps/v1

kind: ReplicaSet

metadata:

name: rs-rollback

spec:

replicas: 3

selector:

matchLabels:

app: node-app

Template Update Test를 위해 Image 정보를 변경한다.

개발팀에서 새로운 버전의 Image를 배포한 상황

image: node:16-alpine3.15 -> node:18-alpine3.15

template: # Pod Template

metadata:

name: node-template

labels:

app: node-app

version: v1

env: prod

spec: # Template Container

containers:

- name: node-con

image: node:18-alpine3.15

command: ['sh', '-c', 'tail -f /dev/null']

Kubernetes Pod & ReplicaSet (EX.1 : Template Update)

```
$ kubectl apply -f replicaset/rs-update-rollback.yml
```

```
replicaset.apps/rs-rollback configured
```

```
$ kubectl get rs -o wide
```

NAME	DESIRED	CURRENT	READY	AGE	CONTAINERS	IMAGES	SELECTOR
rs-rollback	3	3	3	5m21s	node-con	node:18-alpine3.15	app=node-app

▣ ReplicaSet 오브젝트에서 사용하는 Images 정보가 Update 된 것을 확인 할 수 있다.

```
$ kubectl describe pod
```

```
Image: node:16-alpine3.15
```

```
Image: node:16-alpine3.15
```

```
Image: node:16-alpine3.15
```

▣ kubectl describe pod (전체 Pod의 상세정보 확인)

▣ 기존에 생성되었던 Pod의 Image는 이전 버전의 Image가 사용되고 있는 것을 확인 할 수 있다.

▣ Pod에 새로운 Image 정보를 Update하기 위해 기존 Pod를 삭제 후 새로운 Pod가 생성되도록 한다.

Kubernetes Pod & ReplicaSet (EX.1 : Template Update)

```
$ kubectl scale rs rs-rollback --replicas=0
```

```
replicaset.apps/rs-rollback scaled
```

```
$ kubectl get pod -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS GATES
rs-rollback-pqn1b	1/1	Terminating	0	21m	192.168.10.58	node-2	<none>	<none>
rs-rollback-rlczz	1/1	Terminating	0	21m	192.168.10.240	node-1	<none>	<none>
rs-rollback-sxb46	1/1	Terminating	0	19m	192.168.20.185	node-2	<none>	<none>

▣ kubectl scale 명령어를 이용하여 전체 Pod에 대한 Scale Down을 진행한다. (기존 Pod 전부 Terminating)

```
$ kubectl scale rs rs-rollback --replicas=3
```

```
$ kubectl get pod -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS GATES
rs-rollback-6gq7t	0/1	ContainerCreating	0	3s	<none>	node-1	<none>	<none>
rs-rollback-drbx7	0/1	ContainerCreating	0	3s	<none>	node-2	<none>	<none>
rs-rollback-vhtj8	0/1	ContainerCreating	0	3s	<none>	node-1	<none>	<none>

▣ 새로운 Image를 사용하는 Pod생성을 위해 kubectl scale 명령어를 이용하여 Scale UP을 진행한다. (replicas = 3)

Kubernetes Pod & ReplicaSet (EX.1 : Template Update)

```
$ kubectl get pod -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS GATES
rs-rollback-6gq7t	1/1	Running	0	52s	192.168.10.132	node-1	<none>	<none>
rs-rollback-drbx7	1/1	Running	0	52s	192.168.20.169	node-2	<none>	<none>
rs-rollback-vhtj8	1/1	Running	0	52s	192.168.20.179	node-1	<none>	<none>

```
$ kubectl describe pod
```

Image: node:18-alpine3.15

Image: node:18-alpine3.15

Image: node:18-alpine3.15

■ 새로운 Pod가 생성 된 후 각 Pod의 Container Image 정보를 확인

■ [Template Update 정리]

1. ReplicaSet 오브젝트의 Manifest File의 내용을 변경하더라도 기존에 배포 된 Pod에는 영향을주지 않는다.
2. 새로운 변경사항을 Pod에 적용하기 위해서는 새로운 Pod를 배포 해야한다.

Kubernetes Pod & ReplicaSet (EX.1 : Rollback Test)

```
$ kubectl get rs -o wide
```

NAME	DESIRED	CURRENT	READY	AGE	CONTAINERS	IMAGES	SELECTOR
rs-rollback	3	3	3	40m	node-con	node:18-alpine3.15	app=node-app

```
$ kubectl set image rs/rs-rollback node-con=node:16-alpine3.15
```

```
replicaset.apps/rs-rollback image updated
```

```
$ kubectl get rs -o wide
```

NAME	DESIRED	CURRENT	READY	AGE	CONTAINERS	IMAGES	SELECTOR
rs-rollback	3	3	3	40m	node-con	node:16-alpine3.15	app=node-app

```
$ kubectl describe pod
```

```
Image:          node:18-alpine3.15
```

```
Image:          node:18-alpine3.15
```

```
Image:          node:18-alpine3.15
```

▣ Manifest File 편집이 아닌 kubectl set image 명령어를 이용하여 ReplicaSet 오브젝트의 Image 정보를 변경한다.

▣ ReplicaSet 오브젝트에만 변경사항이 적용되며 기존 Pod의 정보는 변하지 않는다.

Kubernetes Pod & ReplicaSet (EX.1 : Rollback Test)

```
$ kubectl get pod --show-labels
```

NAME	READY	STATUS	RESTARTS	AGE	LABELS
rs-rollback-6gq7t	1/1	Running	0	5m30s	app=node-app,env=prod,version=v1
rs-rollback-drbx7	1/1	Running	0	5m30s	app=node-app,env=prod,version=v1
rs-rollback-vhtj8	1/1	Running	0	5m30s	app=node-app,env=prod,version=v1

```
$ kubectl label pod rs-rollback-6gq7t app=delay-app --overwrite
```

```
$ kubectl label pod rs-rollback-drbx7 app=delay-app --overwrite
```

```
$ kubectl label pod rs-rollback-vhtj8 app=delay-app --overwrite
```

```
$ kubectl get pod --show-labels
```

NAME	READY	STATUS	RESTARTS	AGE	LABELS
rs-rollback-6gq7t	1/1	Running	0	7m7s	app=delay-app,env=prod
rs-rollback-drbx7	1/1	Running	0	7m7s	app=delay-app,env=prod
rs-rollback-vhtj8	1/1	Running	0	7m7s	app=delay-app,env=prod
rs-rollback-qjbfl	1/1	Running	0	3s	app=node-app,env=prod
rs-rollback-swrjc	1/1	Running	0	8s	app=node-app,env=prod
rs-rollback-tt5rh	1/1	Running	0	12s	app=node-app,env=prod

- ReplicaSet에 의해 새로 생성 된 Pod 목록
- 이전 버전의 Image 정보를 갖는다.

- 문제 발생 Pod의 Label 정보를 변경한다.
- ReplicaSet 관리 대상에서 제외된다.

Kubernetes Pod & ReplicaSet (EX.1 : Rollback Test)

```
$ kubectl delete pod rs-rollback-6gq7t
```

```
$ kubectl delete pod rs-rollback-drbx7
```

```
$ kubectl delete pod rs-rollback-vhtj8
```

▣ 기존 Image 정보를 갖는 Pod 오브젝트에 대한 삭제 작업을 진행

```
$ kubectl describe pod
```

```
Image:          node:16-alpine3.15
```

```
Image:          node:16-alpine3.15
```

```
Image:          node:16-alpine3.15
```

```
$ kubectl delete rs rs-rollback
```

```
replicaset.apps "rs-rollback" deleted
```

```
$ kubectl get rs
```

```
No resources found in default namespace.
```

```
$ kubectl get pod
```

```
No resources found in default namespace.
```

▣ 새롭게 배포 된 Pod의 Image 정보를 확인 후 Test에 사용한 ReplicaSet 오브젝트를 삭제

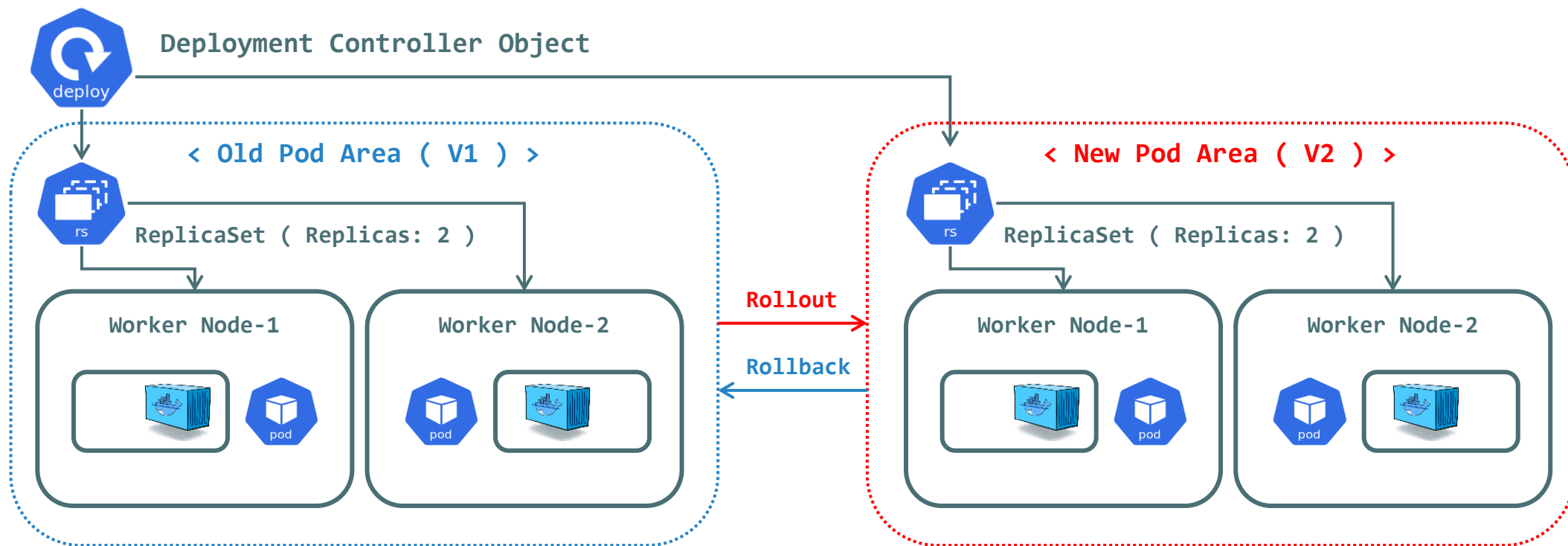


Kubernetes Deployment

Kubernetes Deployment Controller

◎ Kubernetes Deployment

- ▶ Deployment는 Pod 배포 자동화를 위한 Controller의 역할을 수행한다.
- ▶ Deployment는 Pod의 Rollout(Update), Rollback 작업 시 사용되는 ReplicaSet 오브젝트를 관리한다.
- ▶ Deployment 오브젝트는 ReplicaSet의 상위 오브젝트로 Deployment를 이용하여 ReplicaSet 오브젝트를 컨트롤 한다.
- ▶ Deployment는 Revision 기능을 이용한 Rollback, 무중단 서비스 배포를 위한 Rolling-Update를 지원한다.



Kubernetes Deployment Controller (EX.1 : Basic)

< 작업대상 : deployment-basic.yml >

```
apiVersion: apps/v1
```

```
kind: Deployment
```

```
metadata:
```

```
  name: my-deployment # Depolyment 이름
```

```
spec:
```

```
  replicas: 3
```

```
  selector:
```

```
    matchLabels:
```

```
      app: my-app
```

기존 ReplicaSet Manifest File과 유사한 구조를 갖는다.

replicas 속성을 이용하여 복제 Pod의 수를 정의한다.

selector 속성에서 지정 된 Pod Template을 사용하여 Pod를 생성한다.

selector 속성은 template의 Label 정보를 사용하여 template을 선택한다.

matchLabel의 Label 정보와 template Label 정보는 반드시 일치해야한다.

Pod Template

```
template:
```

```
  metadata:
```

```
    labels:
```

```
      app: my-app
```

spec: # Template Container

```
  containers:
```

```
    - name: my-nginx
```

```
      image: nginx:1.22
```

```
      ports:
```

```
        - containerPort: 80
```

Kubernetes Deployment Controller (EX.1 : Basic)

```
$ kubectl apply -f ./deployment-basic.yml
```

```
$ kubectl get deploy -o wide
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE	CONTAINERS	IMAGES	SELECTOR
my-deployment	3/3	3	3	107s	my-nginx	nginx:1.22	app=my-app

```
$ kubectl describe deploy my-deployment
```

Events:

Type	Reason	Age	From	Message
Normal	ScalingReplicaSet	18m	deployment-controller	Scaled up replica set my-deployment-79b4f9588f to 3

```
$ kubectl get rs --show-labels
```

NAME	DESIRED	CURRENT	READY	AGE	LABELS
my-deployment-5c8448bb6b	3	3	3	8m29s	app=my-app, pod-template-hash=5c8448bb6b

```
$ kubectl get pod --show-labels
```

NAME	READY	STATUS	RESTARTS	AGE	LABELS
my-deployment-5c8448bb6b-9kz4k	1/1	Running	0	9m10s	app=my-app, pod-template-hash=5c8448bb6b
my-deployment-5c8448bb6b-qhwqb	1/1	Running	0	9m10s	app=my-app, pod-template-hash=5c8448bb6b
my-deployment-5c8448bb6b-xl85r	1/1	Running	0	9m10s	app=my-app, pod-template-hash=5c8448bb6b

Kubernetes Deployment Controller (EX.1 : Rollout)

```
$ kubectl scale deploy my-deployment --replicas=5
```

```
$ kubectl get rs --show-labels
```

NAME	DESIRED	CURRENT	READY	AGE	LABELS
my-deployment-5c8448bb6b	5	5	5	26m	app=my-app,pod-template-hash=5c8448bb6b

```
$ kubectl get pod --show-labels
```

NAME	READY	STATUS	RESTARTS	AGE	LABELS
my-deployment-5c8448bb6b-9kz4k	1/1	Running	0	4m22s	app=my-app,pod-template-hash=5c8448bb6b
my-deployment-5c8448bb6b-f72c4	1/1	Running	0	47s	app=my-app,pod-template-hash=5c8448bb6b
my-deployment-5c8448bb6b-kzndj	1/1	Running	0	47s	app=my-app,pod-template-hash=5c8448bb6b
my-deployment-5c8448bb6b-qhwqb	1/1	Running	0	4m22s	app=my-app,pod-template-hash=5c8448bb6b
my-deployment-5c8448bb6b-xl85r	1/1	Running	0	4m22s	app=my-app,pod-template-hash=5c8448bb6b

```
$ kubectl describe deploy my-deployment
```

Events:

Type	Reason	Age	From	Message
----	-----	----	----	-----
Normal	ScalingReplicaSet	4m43s	deployment-controller	Scaled up replica set my-deployment-5c8448bb6b to 3
Normal	ScalingReplicaSet	75s	deployment-controller	Scaled up replica set my-deployment-5c8448bb6b to 5

Kubernetes Deployment Controller (EX.1 : Rollout)

< 작업대상 : deployment-basic.yml >

apiVersion: apps/v1

kind: Deployment

metadata:

name: my-deployment # Depolyment 이름

spec:

replicas: 3

selector:

matchLabels:

app: my-app

Template Update Test를 위한 Manifest File을 정의한다.

Template 영역에 새로운 Label을 추가한다. (env: prod)

template: # Pod Template

metadata:

labels:

app: my-app

env: prod

spec: # Template Container

containers:

- name: my-nginx

image: nginx:1.22

ports:

- containerPort: 80

Kubernetes Deployment Controller (EX.1 : Rollout)

```
$ kubectl apply -f deployment/deployment-basic.yml
```

```
deployment.apps/my-deployment configured
```

```
$ kubectl get rs --show-labels
```

NAME	DESIRED	CURRENT	READY	AGE	LABELS
my-deployment-5c8448bb6b	0	0	0	7m25s	app=my-app, pod-template-hash=5c8448bb6b
my-deployment-74d6d7448d	3	3	3	10s	app=my-app, env=prod, pod-template-hash=74d6d7448d

- Deployment Manifest File의 변경사항을 적용 후 Deployment에서 관리하는 ReplicaSet 오브젝트의 변경사항을 확인한다.
- 기존 ReplicaSet의 목표 Pod의 수가 "0"으로 변경되며, 현재 동작중인 Pod의 정보 또한 "0"으로 변경된다.
- 새로운 ReplicaSet이 생성되고, 해당 ReplicaSet에는 새로운 pod-template-hash 값이 부여되어있다.

```
$ kubectl get pod --show-labels
```

NAME	READY	STATUS	RESTARTS	AGE	LABELS
my-deployment-74d6d7448d-kcbvb	1/1	Running	0	78s	app=my-app, env=prod, pod-template-hash=74d6d7448d
my-deployment-74d6d7448d-tt4dr	1/1	Running	0	75s	app=my-app, env=prod, pod-template-hash=74d6d7448d
my-deployment-74d6d7448d-vxxd7	1/1	Running	0	76s	app=my-app, env=prod, pod-template-hash=74d6d7448d

- 현재 운영중인 Pod의 정보를 출력하여 변경 된 pod-template-hash값을 사용하는지 확인한다.

Kubernetes Deployment Controller (EX.1 : Rollout)

```
$ kubectl describe deploy my-deployment
```

Events:

Type	Reason	Age	From	Message
----	-----	----	----	-----
Normal	ScalingReplicaSet	2m52s	deployment-controller	Scaled down replica set <code>my-deployment-5c8448bb6b</code> to 3
Normal	ScalingReplicaSet	2m52s	deployment-controller	Scaled up replica set <code>my-deployment-74d6d7448d</code> to 1
Normal	ScalingReplicaSet	2m51s	deployment-controller	Scaled down replica set <code>my-deployment-5c8448bb6b</code> to 2
Normal	ScalingReplicaSet	2m50s	deployment-controller	Scaled up replica set <code>my-deployment-74d6d7448d</code> to 2
Normal	ScalingReplicaSet	2m49s	deployment-controller	Scaled down replica set <code>my-deployment-5c8448bb6b</code> to 1
Normal	ScalingReplicaSet	2m49s	deployment-controller	Scaled up replica set <code>my-deployment-74d6d7448d</code> to 3
Normal	ScalingReplicaSet	2m47s	deployment-controller	Scaled down replica set <code>my-deployment-5c8448bb6b</code> to 0

■ OLD_ReplicaSet: `my-deployment-5c8448bb6b`

■ NEW_ReplicaSet: `my-deployment-74d6d7448d`

■ OLD[5->3] -> NEW[0->1] -> OLD[3->2] -> NEW[1->2] -> OLD[2->1] -> NEW[2->3] -> OLD[1->0]

Kubernetes Deployment Controller (EX.1 : Rollout)

< 작업대상 : deployment-basic.yml >

apiVersion: apps/v1

kind: Deployment

metadata:

name: my-deployment # Depolyment 이름

spec:

replicas: 3

selector:

matchLabels:

app: my-app

Template Update Test를 위한 Manifest File을 정의한다.

Template 영역에 Container Image를 변경한다. (image: nginx:latest)

template: # Pod Template

metadata:

labels:

app: my-app

env: prod

spec: # Template Container

containers:

- name: my-nginx

image: nginx:latest

ports:

- containerPort: 80

Kubernetes Deployment Controller (EX.1 : Rollout)

```
$ kubectl apply -f ./deployment-basic.yml
```

```
deployment.apps/my-deployment configured
```

```
$ kubectl get rs --show-labels
```

NAME	DESIRED	CURRENT	READY	AGE	LABELS
my-deployment-5c8448bb6b	0	0	0	26m	app=my-app,pod-template-hash=5c8448bb6b
my-deployment-6f5579bcb	3	3	3	9s	app=my-app,env=prod,pod-template-hash=6f5579bcb
my-deployment-74d6d7448d	0	0	0	19m	app=my-app,env=prod,pod-template-hash=74d6d7448d

■ Deployment Manifest File의 변경사항을 적용 후 Deployment에서 관리하는 ReplicaSet 오브젝트의 변경사항을 확인한다.

■ 기존 ReplicaSet의 목표 Pod의 수가 "0"으로 변경되며, 현재 동작중인 Pod의 정보 또한 "0"으로 변경된다.

■ 새로운 ReplicaSet이 생성되고, 해당 ReplicaSet에는 새로운 pod-template-hash 값이 부여되어있다.

```
$ kubectl get pod --show-labels
```

NAME	READY	STATUS	RESTARTS	AGE	LABELS
my-deployment-6f5579bcb-77lm8	1/1	Running	0	50s	app=my-app,env=prod,pod-template-hash=6f5579bcb
my-deployment-6f5579bcb-8m2vx	1/1	Running	0	51s	app=my-app,env=prod,pod-template-hash=6f5579bcb
my-deployment-6f5579bcb-vzmrđ	1/1	Running	0	48s	app=my-app,env=prod,pod-template-hash=6f5579bcb

■ 현재 운영중인 Pod의 정보를 출력하여 변경 된 pod-template-hash값을 사용하는지 확인한다.

Kubernetes Deployment Controller (EX.1 : Rollout)

```
$ kubectl describe deploy my-deployment
```

Events:

Type	Reason	Age	From	Message
----	-----	----	----	-----
Normal	ScalingReplicaSet	119s	deployment-controller	Scaled up replica set my-deployment-6f5579bcb to 1
Normal	ScalingReplicaSet	117s	deployment-controller	Scaled down replica set my-deployment-74d6d7448d to 2
Normal	ScalingReplicaSet	117s	deployment-controller	Scaled up replica set my-deployment-6f5579bcb to 2
Normal	ScalingReplicaSet	115s	deployment-controller	Scaled down replica set my-deployment-74d6d7448d to 1
Normal	ScalingReplicaSet	115s	deployment-controller	Scaled up replica set my-deployment-6f5579bcb to 3
Normal	ScalingReplicaSet	114s	deployment-controller	Scaled down replica set my-deployment-74d6d7448d to 0

■ OLD_ReplicaSet: **my-deployment-74d6d7448d**

■ NEW_ReplicaSet: **my-deployment-6f5579bcb**

■ NEW[**0->1**] -> OLD[**3->2**] -> NEW[**1->2**] -> OLD[**2->1**] -> NEW[**2->3**] -> OLD[**1->0**]

Kubernetes Deployment Controller (EX.1 : Revision)

```
$ kubectl rollout history deploy my-deployment
```

```
deployment.apps/my-deployment
```

```
REVISION  CHANGE-CAUSE
```

```
1          <none>
```

```
2          <none>
```

```
3          <none>
```

▣ kubectl rollout history 명령어를 이용하여 Deployment Revision 정보를 확인한다. (현재 총 3개의 Revision 정보가 존재)

▣ Revision 상세정보 확인 : kubectl rollout history deploy/my-deployment --revision=1

```
$ kubectl rollout undo deploy my-deployment
```

```
deployment.apps/my-deployment rolled back
```

```
$ kubectl get rs --show-labels
```

NAME	DESIRED	CURRENT	READY	AGE	LABELS
my-deployment-5c8448bb6b	0	0	0	32m	app=my-app,pod-template-hash=5c8448bb6b
my-deployment-6f5579bcb	0	0	0	6m16s	app=my-app,env=prod,pod-template-hash=6f5579bcb
my-deployment-74d6d7448d	3	3	3	25m	app=my-app,env=prod,pod-template-hash=74d6d7448d

▣ kubectl rollout undo 명령어를 이용하여 직전 Revision 정보로 Rollback 작업을 수행한다.

▣ Rollback을 진행 한 Revision에서 사용 한 ReplicaSet 오브젝트가 다시 사용되는 것을 확인 할 수 있다.

Kubernetes Deployment Controller (EX.1 : Revision)

```
$ kubectl annotate deploy my-deployment kubernetes.io/change-cause="image-rollback"
```

```
$ kubectl rollout history deploy my-deployment
```

```
deployment.apps/my-deployment
```

```
REVISION  CHANGE-CAUSE
```

```
1          <none>
```

```
3          <none>
```

```
4          image-rollback
```

▣ 현재 배포상태 Revision에 주석을 지정하여, 해당 Revision 정보를 기록 할 수 있다.

▣ 주로 Rollback 진행 시 Rollback 사유등을 명시하는 용도로 사용 된다.

```
$ kubectl rollout undo deploy my-deployment --to-revision=1
```

```
deployment.apps/my-deployment rolled back
```

```
$ kubectl get rs --show-labels
```

NAME	DESIRED	CURRENT	READY	AGE	LABELS
my-deployment-5c8448bb6b	3	3	3	38m	app=my-app, pod-template-hash=5c8448bb6b
my-deployment-6f5579bcb	0	0	0	11m	app=my-app, env=prod, pod-template-hash=6f5579bcb
my-deployment-74d6d7448d	0	0	0	30m	app=my-app, env=prod, pod-template-hash=74d6d7448d

▣ 특정 Revision으로 RollBack 시 "--to-revision=<Revision Num>" 옵션을 사용한다. (기본값 : 직전 Revision으로 Rollback)

Kubernetes Deployment Controller (EX.1 : Revision)

```
$ kubectl annotate deploy my-deployment kubernetes.io/change-cause="First-rollback"
```

```
$ kubectl rollout history deploy my-deployment
```

```
deployment.apps/my-deployment
```

```
REVISION  CHANGE-CAUSE
```

```
3          <none>
```

```
4          image-rollback
```

```
5          First-rollback
```

▣ 현재 배포상태 Revision에 "First-rollback" 주석을 지정 후 확인

```
$ kubectl delete deploy my-deployment
```

```
deployment.apps "my-deployment" deleted
```

```
$ kubectl get deploy my-deployment
```

```
Error from server (NotFound): deployments.apps "my-deployment" not found
```

```
$ kubectl get rs
```

```
No resources found in default namespace.
```

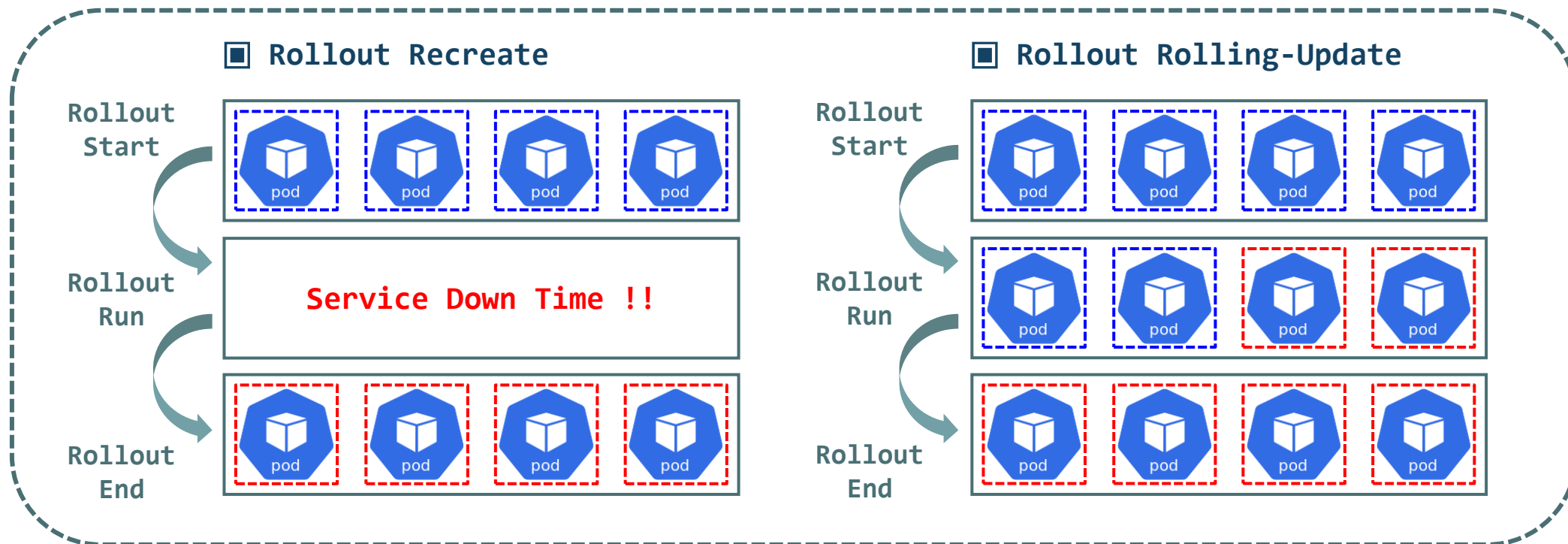
```
$ kubectl get pod
```

```
No resources found in default namespace.
```

Kubernetes Deployment Controller (Rolling-Update)

◎ Kubernetes Deployment Rollout (Recreate & Rolling-Update)

- ▶ Deployment는 2가지의 Rollout 방식을 제공한다. (Recreate, Rolling-Update)
- ▶ Recreate는 새로운 배포를 위해 이전 배포에 사용 된 모든 파드 오브젝트를 즉시 종료한다.
- ▶ Recreate는 새로운 배포 작업을 완료하기까지 서비스 중단의 문제가 발생한다. (개발 및 테스트 환경에 적합한 배포전략)
- ▶ Rolling-Update는 새로운 배포를 진행하면서 동시에 이전 배포에 사용 된 파드 오브젝트를 순차적으로 종료한다.
- ▶ Rolling-Update는 서비스 중단을 최소화하며 새로운 배포가 진행되므로 실제 운영환경에서 사용하기 적합한 배포전략



Kubernetes Deployment Controller (Rolling-Update)

◎ Kubernetes Deployment Rolling-Update (maxUnavailable & maxSurge)

- ▶ Deployment는 배포 속도제어 속성인 "**maxUnavailable**", "**maxSurge**"를 이용하여 서비스 무중단 업데이트 배포를 구현한다.
- ▶ maxUnavailable, maxSurge 속성의 값으로는 "**숫자값**" 혹은 전체 replicas의 "%"로 표현이 가능하다.
- ▶ maxUnavailable, maxSurge 속성을 이용하여 새로운 배포 버전으로 **전환하는 시간을 최소화** 할 수 있다.
- ▶ 속도 제어를 통해 새로운 배포 버전을 한번에 배포 할 경우 컴퓨팅 자원이 부족할 수 있는 문제를 해결 할 수 있다.

■ maxUnavailable : Rolling-Update 작업 시 유지해야 할 최소 Pod의 수를 정의

- . EX: replicas(10), maxUnavailable(30%)일 경우 Rolling-Update 작업 중 최소 7개의 Pod는 유지되어야 한다.
- . 이전 배포 버전의 Pod는 한번에 최대 30%(3개)까지 **Scale Down**이 가능하다.
- . 신규 배포 버전의 Pod와 이전 배포 버전의 Pod를 최소 7개를 유지하며 업데이트를 진행한다.
- . 최소 유지 Pod 계산식 : **replicas(10)** - **maxUnavailable(30%[3])** = 최소 7개의 Pod를 유지

■ maxSurge : Rolling-Update 작업 중 유지할 수 있는 최대 Pod의 수를 정의

- . EX: replicas(10), maxSurge(30%)일 경우 Rolling-Update 작업 중 최대 13개의 Pod를 유지 할 수 있다.
- . 신규 배포 버전의 Pod를 최대 3개까지 즉시 **Scale UP**이 가능하다.
- . 신규 배포 버전의 Pod와 이전 배포 버전의 Pod를 최대 13개까지 유지하며 업데이트를 진행한다.
- . 최대 유지 Pod 계산식 : **replicas(10)** + **maxSurge(30%[3])** = 최대 13개의 Pod를 유지

Kubernetes Deployment Controller (EX.2 : Rolling-Update)

< 작업대상 : deploy-rollingupdate.yml >

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-deployment # Depolyment 이름
spec:
  replicas: 5
  selector:
    matchLabels:
      app: my-app
  strategy:
    # 배포방식 정의 영역
    type: RollingUpdate # 배포방식 타입 지정
    rollingUpdate:
      # Rolling-Update 속성정의 영역
      maxUnavailable: 2 # 업데이트 중 최소로 유지 할 Pod 수 정의
      maxSurge: 1 # 업데이트 중 최대로 유지 할 수 있는 Pod 수 정의
```

Strategy 영역의 type 속성에 값을 Recreate로 정의도 가능하다.

Recreate를 사용 할 경우 추가 속성 정의는 진행하지 않는다.

template: # Pod Template

metadata:

labels:

app: my-app

spec: # Template Container

containers:

- name: my-nginx
- image: nginx:1.22
- ports:
- containerPort: 80

Kubernetes Deployment Controller (EX.2 : Rolling-Update)

```
$ kubectl apply -f deployment/deploy-rollingupdate.yml
```

```
deployment.apps/my-deployment created
```

```
$ kubectl get deploy my-deployment
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
my-deployment	5/5	5	5	7m57s

```
$ kubectl describe deploy my-deployment
```

```
Replicas:          5 desired | 5 updated | 5 total | 5 available | 0 unavailable
```

```
StrategyType:      RollingUpdate
```

```
MinReadySeconds:   0
```

```
RollingUpdateStrategy: 2 max unavailable, 1 max surge
```

Events:

Type	Reason	Age	From	Message
----	-----	----	----	-----
Normal	ScalingReplicaSet	9m47s	deployment-controller	Scaled up replica set my-deployment-5c8448bb6b to 5

■ maxUnavailable : replicas(5) - maxUnavailable(2) = 최소 3개의 Pod를 유지

■ maxSurge : replicas(5) + maxSurge(1) = 최대 6개의 Pod를 유지

Kubernetes Deployment Controller (EX.2 : Rolling-Update)

< 작업대상 : deploy-rollingupdate.yml >

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-deployment # Depolyment 이름
spec:
  replicas: 5
  selector:
    matchLabels:
      app: my-app
  strategy:
    # 배포방식 정의 영역
    type: RollingUpdate # 배포방식 타입 지정
    rollingUpdate:
      # Rolling-Update 속성정의 영역
      maxUnavailable: 2 # 업데이트 중 최소로 유지 할 Pod 수 정의
      maxSurge: 1 # 업데이트 중 최대로 유지 할 수 있는 Pod 수 정의

# Pod Template Label 추가 ( env: prod )
# Container Image 변경 ( nginx: latest )
```

Pod Template

```
template:
  metadata:
    labels:
      app: my-app
      env: prod
```

Template Container

```
spec:
  containers:
    - name: my-nginx
      image: nginx:latest
      ports:
        - containerPort: 80
```

Kubernetes Deployment Controller (EX.2 : Rolling-Update)

```
$ kubectl apply -f deployment/deploy-rollingupdate.yml
```

```
$ kubectl describe deploy my-deployment
```

Events:

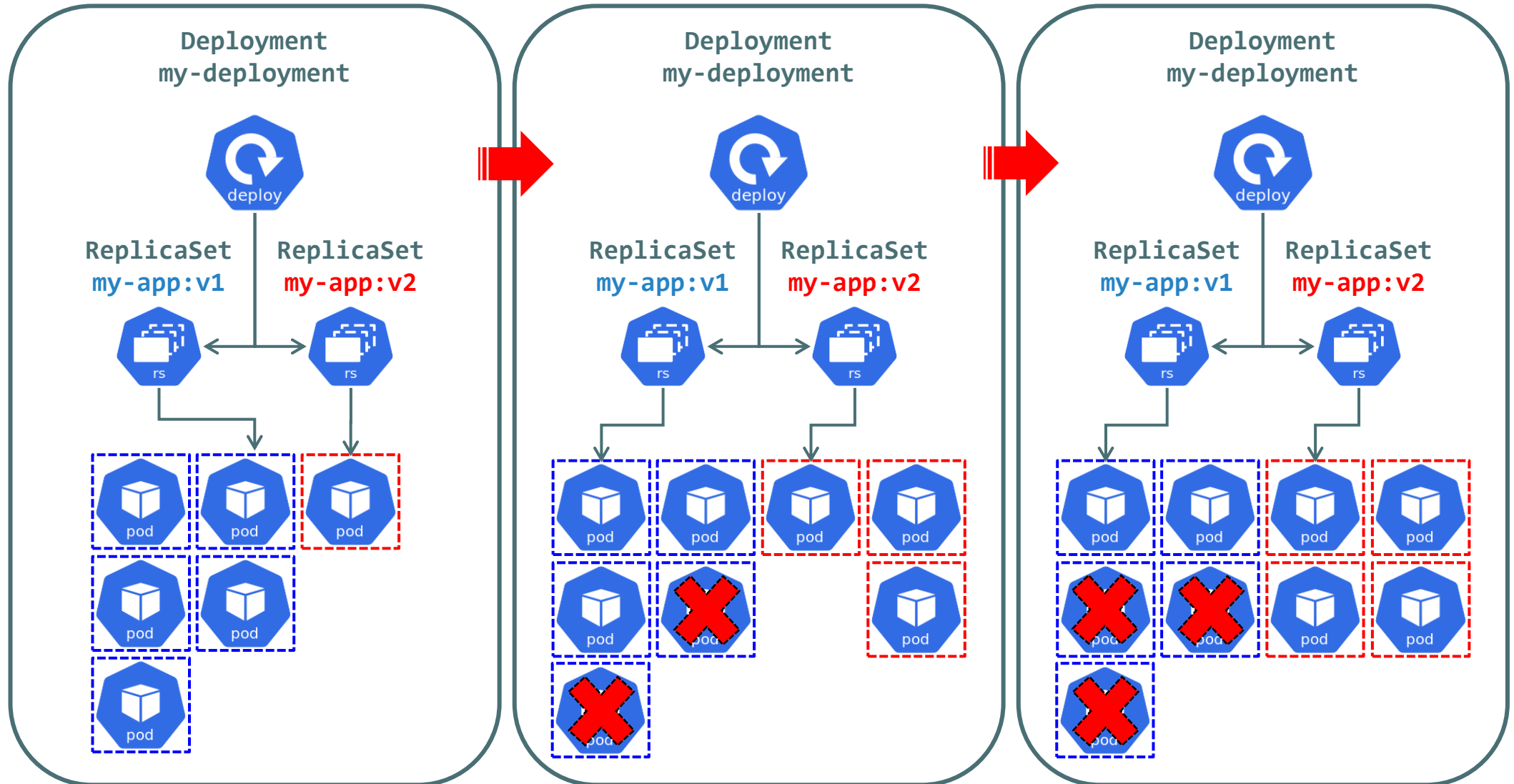
Type	Reason	Age	From	Message
----	-----	----	----	-----
Normal	ScalingReplicaSet	2m8s	deployment-controller	Scaled up replica set my-deployment-5c8448bb6b to 5
Normal	ScalingReplicaSet	10s	deployment-controller	Scaled up replica set my-deployment-6f5579bcb to 1
Normal	ScalingReplicaSet	10s	deployment-controller	Scaled down replica set my-deployment-5c8448bb6b to 3
Normal	ScalingReplicaSet	10s	deployment-controller	Scaled up replica set my-deployment-6f5579bcb to 3
Normal	ScalingReplicaSet	8s	deployment-controller	Scaled down replica set my-deployment-5c8448bb6b to 2
Normal	ScalingReplicaSet	8s	deployment-controller	Scaled up replica set my-deployment-6f5579bcb to 4
Normal	ScalingReplicaSet	7s	deployment-controller	Scaled down replica set my-deployment-5c8448bb6b to 1
Normal	ScalingReplicaSet	7s	deployment-controller	Scaled up replica set my-deployment-6f5579bcb to 5
Normal	ScalingReplicaSet	6s	deployment-controller	Scaled down replica set my-deployment-5c8448bb6b to 0

■ OLD_ReplicaSet: my-deployment-5c8448bb6b

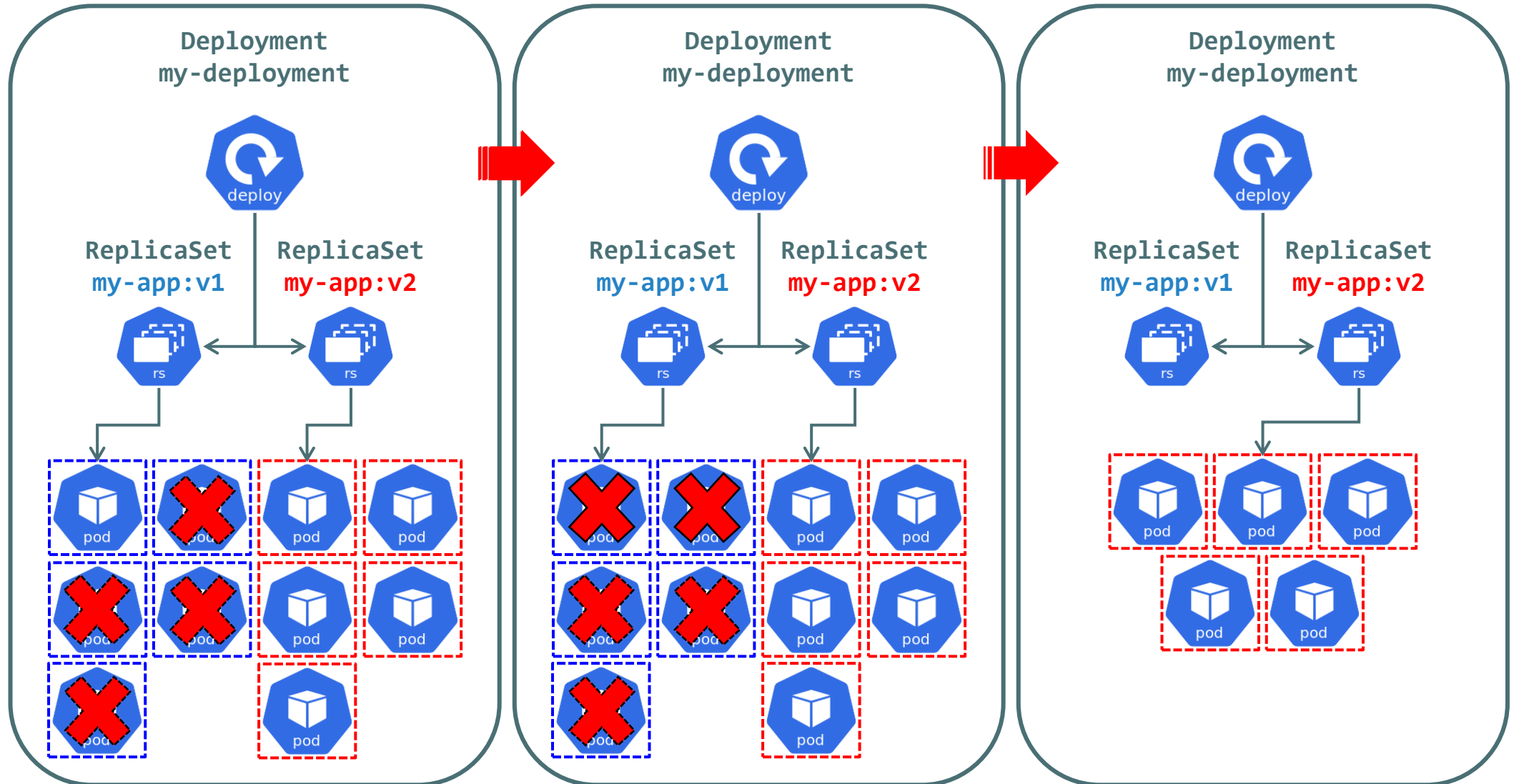
■ NEW_ReplicaSet: my-deployment-6f5579bcb

■ NEW[0->1] -> OLD[5->3] -> NEW[1->3] -> OLD[3->2] -> NEW[3->4] -> OLD[2->1] -> NEW[4->5] -> OLD[1->0]

Kubernetes Deployment Controller (EX.2 : Rolling-Update)



Kubernetes Deployment Controller (EX.2 : Rolling-Update)



Kubernetes Deployment Controller (EX.2 : Rolling-Update)

< 작업대상 : deploy-rollingupdate.yml >

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-deployment # Depolyment 이름
spec:
  replicas: 3          # replicas 5 -> 3 변경
  selector:
    matchLabels:
      app: my-app
  strategy:
    type: RollingUpdate
    rollingUpdate:      # maxUnavailable 삭제
      maxSurge: 3      # maxSurge 1 -> 3 변경

# replicas 변경 / maxSurge 변경 / maxUnavailable 삭제
```

Pod Template

```
template:
  metadata:
    labels:
      app: my-app
      env: prod
```

Template Container

```
spec:
  containers:
    - name: my-nginx
      image: nginx:latest
      ports:
        - containerPort: 80
```


Kubernetes Deployment Controller (EX.2 : Rolling-Update)

```
$ kubectl delete deploy my-deployment
```

```
deployment.apps/my-deployment deleted
```

■ 기존 Deployment 오브젝트 삭제 후 재 배포 진행 (Event 기록 초기화)

```
$ kubectl apply -f deployment/deploy-rollingupdate.yml
```

```
deployment.apps/my-deployment created
```

```
$ kubectl describe deploy my-deployment
```

```
Replicas:          3 desired | 3 updated | 3 total | 3 available | 0 unavailable
```

```
StrategyType:      RollingUpdate
```

```
MinReadySeconds:   0
```

```
RollingUpdateStrategy: 25% max unavailable, 3 max surge
```

Events:

Type	Reason	Age	From	Message
----	-----	----	----	-----
Normal	ScalingReplicaSet	44s	deployment-controller	Scaled up replica set my-deployment-6c6b7bc57d to 3

■ maxUnavailable : 기본값 25%가 설정되어있는 것을 확인

■ maxSurge : replicas(3) + maxSurge(3) = 최대 6개의 Pod를 유지

Kubernetes Deployment Controller (EX.2 : Rolling-Update)

< 작업대상 : deploy-rollingupdate.yml >

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-deployment # Depolyment 이름
spec:
  replicas: 3          # replicas 5 -> 3 변경
  selector:
    matchLabels:
      app: my-app
  strategy:
    type: RollingUpdate
    rollingUpdate:      # maxUnavailable 삭제
      maxSurge: 3       # maxSurge 1 -> 3 변경

# Pod Template Label 추가 ( project: kube )
```

Pod Template

```
template:
  metadata:
    labels:
      app: my-app
      env: prod
      project: kube
```

Template Container

```
spec:
  containers:
    - name: my-nginx
      image: nginx:latest
      ports:
        - containerPort: 80
```

Kubernetes Deployment Controller (EX.2 : Rolling-Update)

```
$ kubectl apply -f deployment/deploy-rollingupdate.yml
```

```
deployment.apps/my-deployment configured
```

```
$ kubectl describe deploy my-deployment
```

Events:

Type	Reason	Age	From	Message
----	-----	----	----	-----
Normal	ScalingReplicaSet	3m36s	deployment-controller	Scaled up replica set my-deployment-6c6b7bc57d to 3
Normal	ScalingReplicaSet	7s	deployment-controller	Scaled up replica set my-deployment-6b6b584974 to 3
Normal	ScalingReplicaSet	4s	deployment-controller	Scaled down replica set my-deployment-6c6b7bc57d to 2
Normal	ScalingReplicaSet	4s	deployment-controller	Scaled down replica set my-deployment-6c6b7bc57d to 0

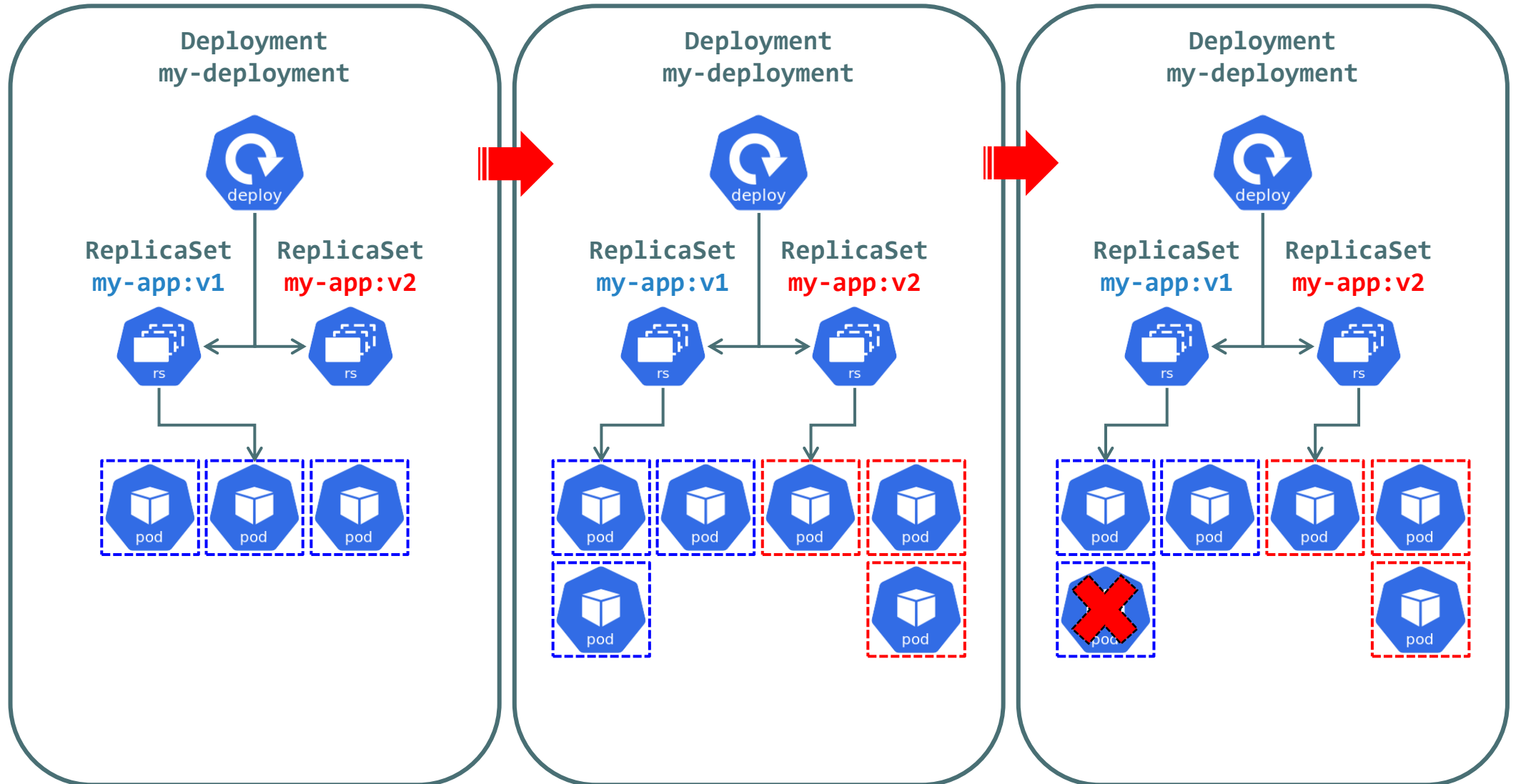
■ OLD_ReplicaSet: my-deployment-b4469cb67

■ NEW_ReplicaSet: my-deployment-7b4bd8d6b7

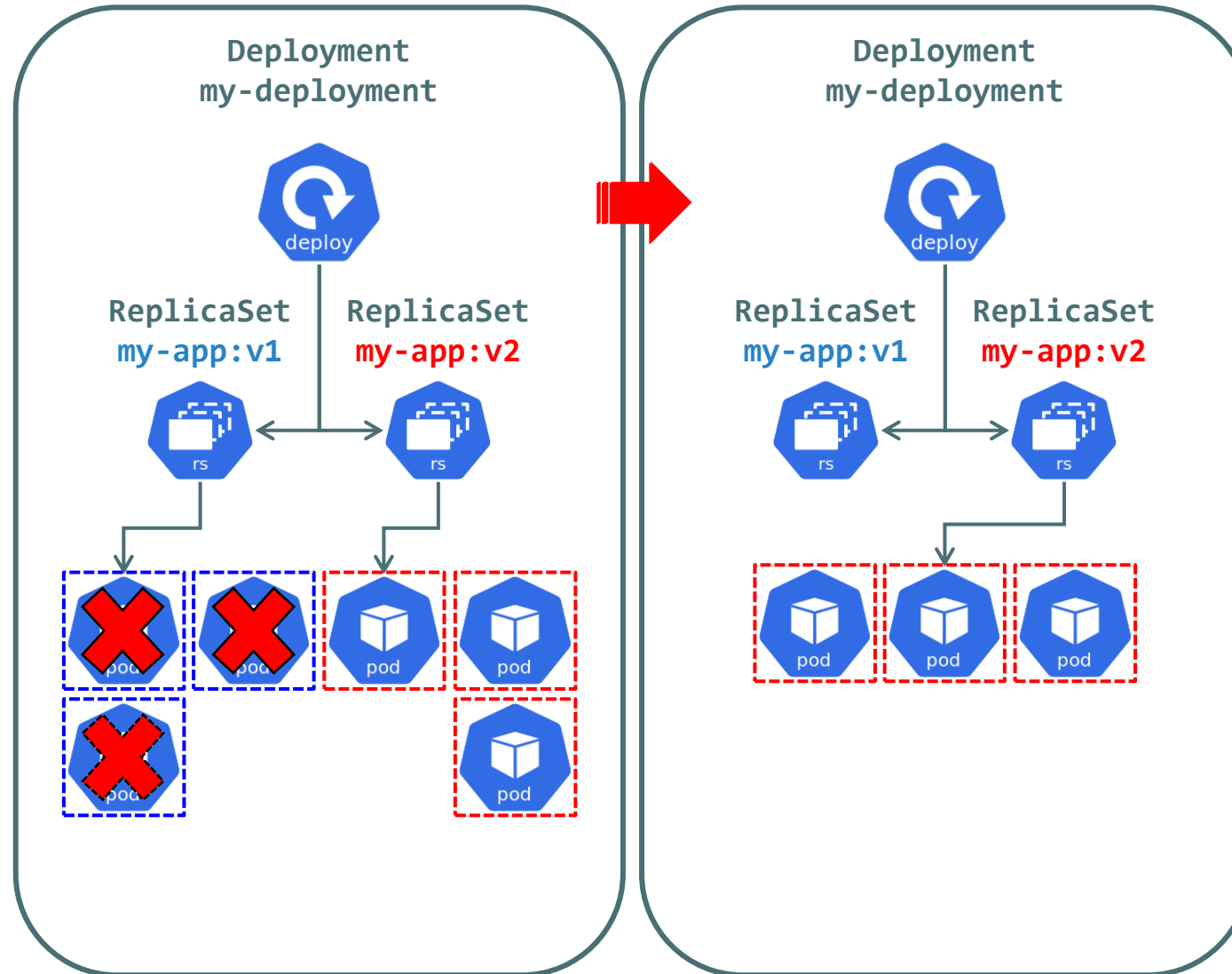
■ NEW[0->3] -> OLD[3->2] -> OLD[2->0]

■ maxSurge를 replicas 수 만큼 지정 할 경우 빠른 업데이트가 가능하나 컴퓨팅 자원 리소스가 많이 소모된다.

Kubernetes Deployment Controller (EX.2 : Rolling-Update)



Kubernetes Deployment Controller (EX.2 : Rolling-Update)



Kubernetes Deployment Controller (EX.2 : Rolling-Update)

```
$ kubectl delete deploy my-deployment
```

```
deployment.apps "my-deployment" deleted
```

```
$ kubectl get deploy my-deployment
```

```
Error from server (NotFound): deployments.apps "my-deployment" not found
```

```
$ kubectl get rs
```

```
No resources found in default namespace.
```

```
$ kubectl get pod
```

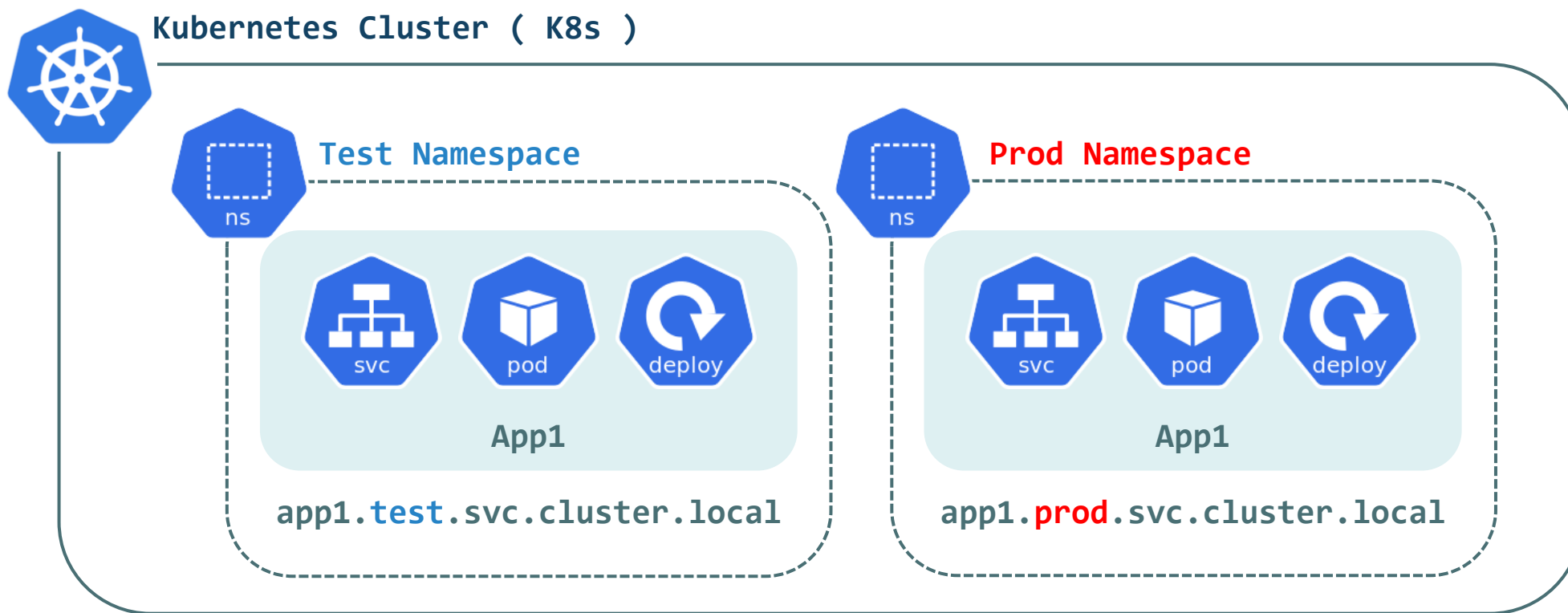
```
No resources found in default namespace.
```

- Deployment Controller 오브젝트를 삭제 할 경우 해당 Controller가 관리하는 오브젝트는 함께 삭제된다.
- Deployment Controller 삭제 후 ReplicaSet, Pod 오브젝트가 함께 삭제되었는지 확인
- 다음 테스트 작업 시 기존 오브젝트가 남아있을 경우 문제가 발생 할 수 있으므로 반드시 삭제 및 확인 작업을 수행한다.

Kubernetes Deployment Controller (Namespace)

◎ Kubernetes Namespace

- ▶ Namespace는 Kubernetes Cluster를 가상화하여 논리적인 작업영역을 생성 한다. (작업영역 분리 효과)
- ▶ Namespace의 활용: "테스트 환경"과 "운영 환경"의 분리 / "프로젝트 A"와 "프로젝트 B"를 분리
- ▶ Namespace는 명령어 적용 범위, 오브젝트, 컴퓨팅 자원 리소스 할당, 네트워크 영역 등을 구분한다.
- ▶ Namespace를 생성 후 Deployment Controller 오브젝트를 활용한 배포작업 테스트를 진행



Kubernetes Deployment Controller (EX.3 : Namespace)

```
$ kubectl create namespace prod
```

```
namespace/prod created
```

```
$ kubectl create namespace dev-stage
```

```
namespace/dev-stage created
```

```
$ kubectl get namespace
```

NAME	STATUS	AGE
default	Active	3d22h
dev-stage	Active	6s
kube-node-lease	Active	3d22h
kube-public	Active	3d22h
kube-system	Active	3d22h
prod	Active	8m48s

- ▣ kubectl create namespace 명령어를 이용하여 새로운 Namespace를 생성한다.
- ▣ 현재 Kubernetes Cluster에서 사용중인 전체 Namespace 목록을 확인한다.
- ▣ 기본 배포 Namespace인 default부터 Kubernetes Cluster 운영에 필요한 Namespace 목록이 출력된다.

Kubernetes Deployment Controller (EX.3 : Namespace)

< 작업대상 : deploy-prod.yml >

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: prod-deploy
  namespace: prod      # Namespace Prod 지정
spec:
  replicas: 3
  selector:
    matchLabels:
      app: prod-nginx
```

새롭게 생성 된 prod Namespace를 배포 Namespace로 정의한다.
해당 Deployment 오브젝트는 정의 된 Namespace에 배포작업을 수행하게 된다.

Pod Template

```
template:
  metadata:
    labels:
      app: prod-nginx
    env: production
```

Template Container

```
spec:
  containers:
    - name: my-nginx
      image: nginx:1.22
      ports:
        - containerPort: 80
```

Kubernetes Deployment Controller (EX.3 : Namespace)

< 작업대상 : deploy-dev-stage.yml >

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: dev-deploy
  namespace: dev-stage # Namespace Prod 지정
spec:
  replicas: 2
  selector:
    matchLabels:
      app: dev-apache
```

새롭게 생성 된 dev-stage Namespace를 배포 Namespace로 지정한다.
해당 Deployment 오브젝트는 지정 된 Namespace에 배포작업을 수행하게 된다.

Pod Template

```
template:
  metadata:
    labels:
      app: dev-apache
      env: development
```

Template Container

```
spec:
  containers:
    - name: dev-web
      image: httpd:latest
      ports:
        - containerPort: 8080
```

Kubernetes Deployment Controller (EX.3 : Namespace)

```
$ kubectl apply -f deployment/deploy-prod.yml
```

```
deployment.apps/prod-deploy created
```

```
$ kubectl apply -f deployment/deploy-dev-stage.yml
```

```
deployment.apps/dev-deploy created
```

```
$ kubectl get deploy -o wide
```

```
No resources found in default namespace.
```

```
$ kubectl get deploy -o wide -n prod
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE	CONTAINERS	IMAGES	SELECTOR
prod-deploy	3/3	3	3	7m19s	my-nginx	nginx:1.22	app=prod-nginx

```
$ kubectl get deploy -o wide -n dev-stage
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE	CONTAINERS	IMAGES	SELECTOR
dev-deploy	2/2	2	2	59s	dev-web	httpd:latest	app=dev-apache

- ▣ kubectl 명령어를 이용하여 오브젝트 조회 작업을 수행 할 경우 기본 default Namespace에서 조회 작업을 수행한다.
- ▣ default가 아닌 다른 Namespace에 존재하는 오브젝트를 조회 할 때에는 "--namespace or -n" 옵션을 사용한다.
- ▣ 현재 새로 생성한 prod, dev-stage Namespace에 Deployment Controller 오브젝트가 생성 된 것을 확인한다.

Kubernetes Deployment Controller (EX.3 : Namespace)

```
$ kubectl get rs -o wide -n prod
```

NAME	DESIRED	CURRENT	READY	AGE	CONTAINERS	IMAGES	SELECTOR
prod-deploy-bc4b64b5d	3	3	3	9m54s	my-nginx	nginx:1.22	app=prod-nginx

```
$ kubectl get pod -o wide -n prod
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS
prod-deploy-bc4b64b5d-mzzgz	1/1	Running	0	10m	192.168.20.185	node-2	<none>	<none>
prod-deploy-bc4b64b5d-wfq4k	1/1	Running	0	10m	192.168.10.58	node-2	<none>	<none>
prod-deploy-bc4b64b5d-x92gl	1/1	Running	0	10m	192.168.10.156	node-1	<none>	<none>

```
$ kubectl get rs -o wide -n dev-stage
```

NAME	DESIRED	CURRENT	READY	AGE	CONTAINERS	IMAGES	SELECTOR
dev-deploy-5d7dfcf975	2	2	2	3m42s	dev-web	httpd:latest	app=dev-apache

```
$ kubectl get pod -o wide -n dev-stage
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS
dev-deploy-5d7dfcf975-24jg6	1/1	Running	0	3m58s	192.168.20.169	node-1	<none>	<none>
dev-deploy-5d7dfcf975-h2j7k	1/1	Running	0	3m58s	192.168.10.112	node-2	<none>	<none>

▣ ReplicaSet, Pod 오브젝트 또한 서로 다른 Namespace 영역에 생성 된 것을 확인한다.

Kubernetes Deployment Controller (EX.3 : Namespace)

```
$ kubectl delete deploy prod-deploy -n prod
```

```
deployment.apps "prod-deploy" deleted
```

```
$ kubectl delete deploy dev-deploy -n dev-stage
```

```
deployment.apps "dev-deploy" deleted
```

```
$ kubectl delete namespace prod
```

```
namespace "prod" deleted
```

```
$ kubectl delete namespace dev-stage
```

```
namespace "dev-stage" deleted
```

```
$ kubectl get namespace
```

NAME	STATUS	AGE
default	Active	3d22h
kube-node-lease	Active	3d22h
kube-public	Active	3d22h
kube-system	Active	3d22h

▣ 다음 테스트를 위해 생성한 Deployment Controller 오브젝트를 삭제한다.

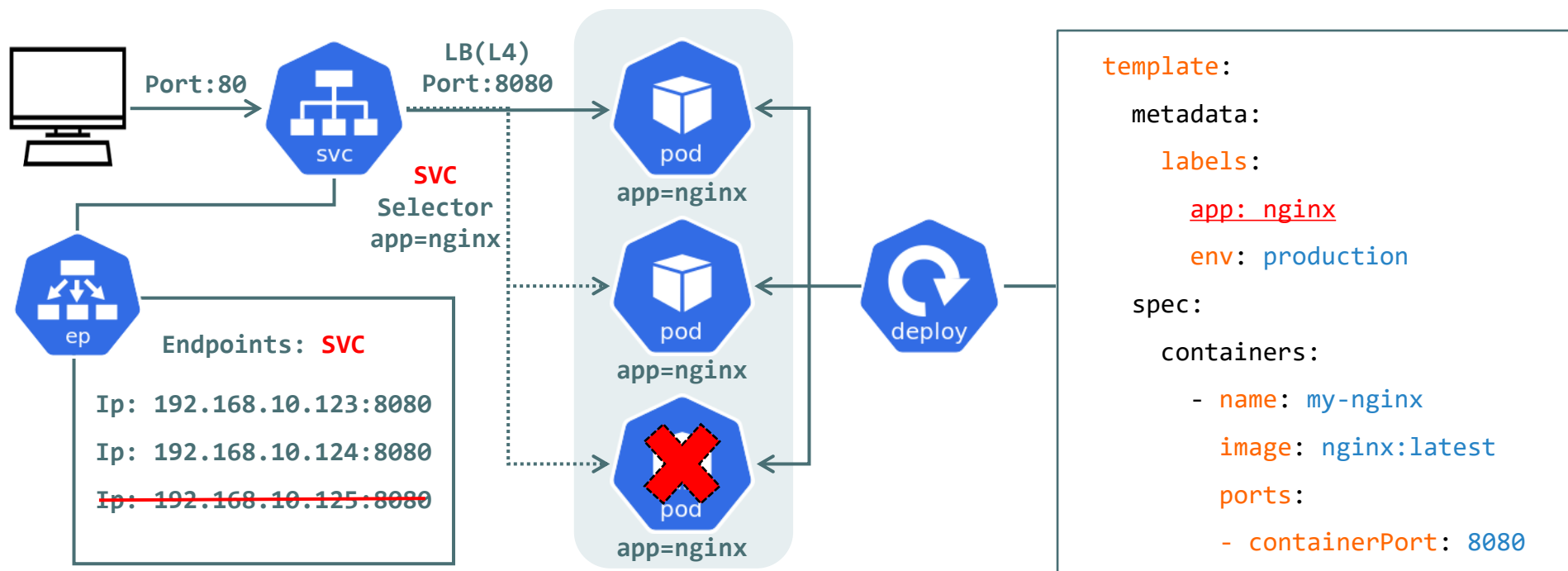


Kubernetes Service Object

Kubernetes Service Object

© Kubernetes Service Object

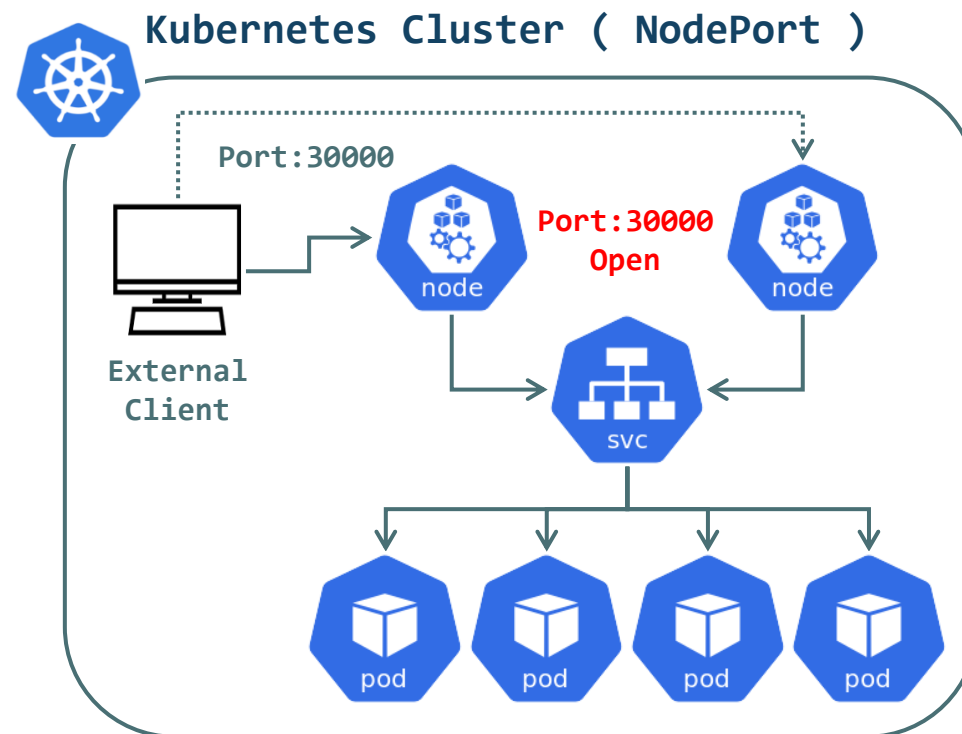
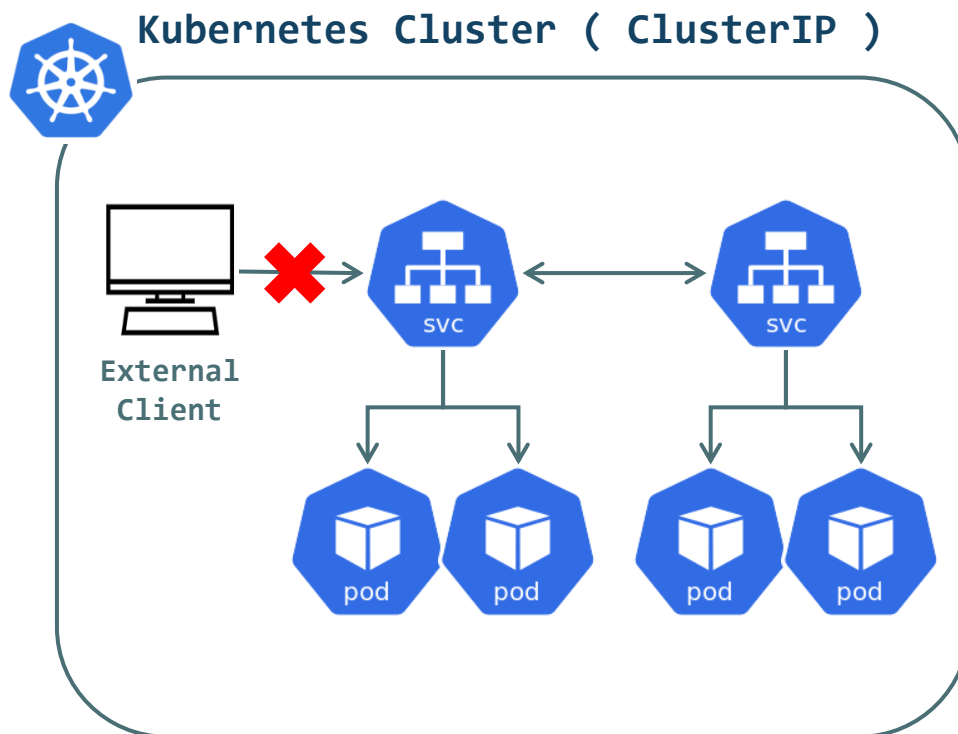
- ▶ Service 오브젝트는 Pod 집합에 대한 네트워크 접근 규칙을 정의하기 위해 사용되는 Kubernetes 필수 오브젝트이다.
- ▶ Kubernetes Cluster에 배포된 Pod의 IP는 영속적이지 않으며, 외부 접근이 불가능한 사설 IP주소 대역을 사용한다.
- ▶ Kubernetes Cluster에 배포된 Pod 집합을 외부 네트워크에 공개하기 위해서는 반드시 Service 오브젝트가 함께 구성되어야 한다.
- ▶ Service는 자신과 연결된 Pod 집합의 주소 목록을 Endpoints로 관리하며, Pod 집합의 변경사항 발생 시 함께 업데이트 된다.
- ▶ Service는 클라이언트 요청을 받아들여 해당 요청을 Endpoints에 등록된 Pod 중 하나로 리다이렉트 한다.
- ▶ Kubernetes Cluster 내부에 생성된 Service 오브젝트는 Kubernetes Cluster 내부 DNS에 자동등록 된다. (Service Discovery)



Kubernetes Service Type (ClusterIP, NodePort)

© Kubernetes Service Type (ClusterIP, NodePort)

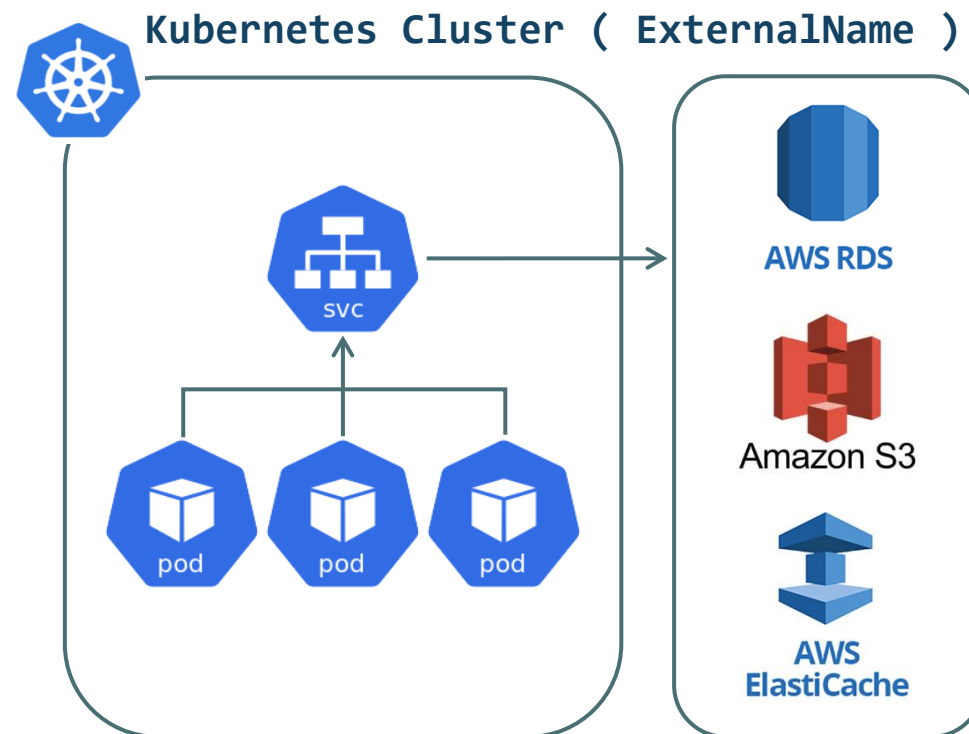
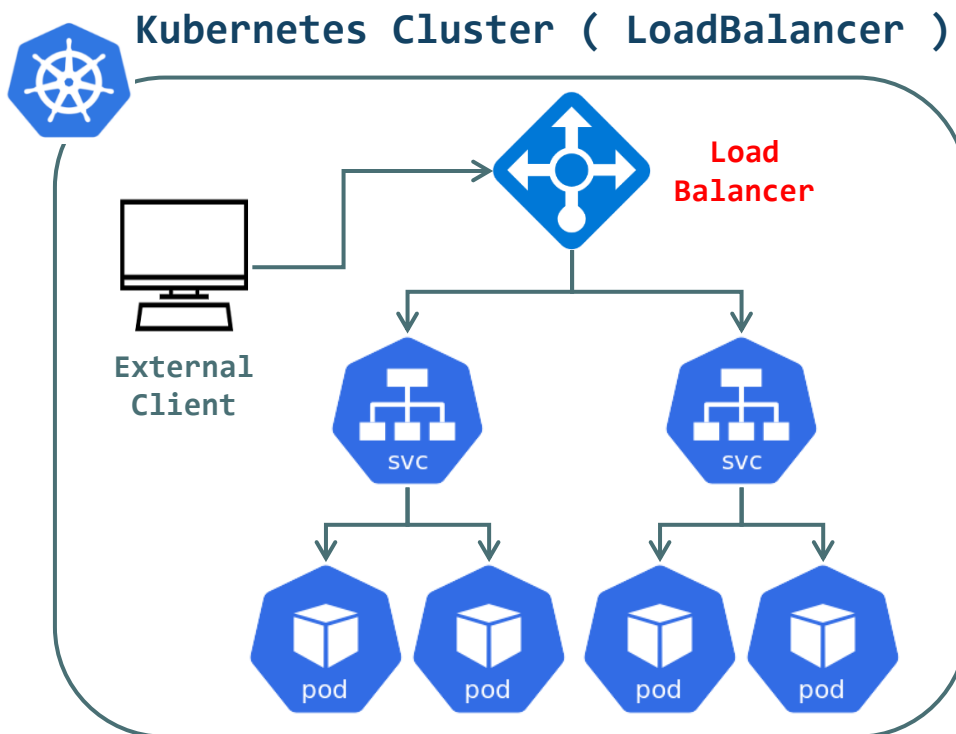
- ▶ ClusterIP: Kubernetes Cluster 내부에서만 접근을 허용한다. (외부 연결이 불가능한 Type)
- ▶ NodePort: Pod가 생성 되는 Node의 특정 Port를 외부에 Open하여 Pod를 외부에 공개한다. (모든 Node는 동일 Port를 Open)



Kubernetes Service Type (ExternalName, LoadBalancer)

© Kubernetes Service Type (LoadBalancer, ExternalName)

- ▶ LoadBalancer: 별도로 구성 된 LoadBalancer를 Pod와 연결하여 Pod를 외부에 공개한다. (운영 환경에 적합)
- ▶ ExternalName: Kubernetes Cluster 외부 Resource와 연결 시 사용 (Domain Name을 이용한 접근 [EX: Pod <-> AWS RDS])



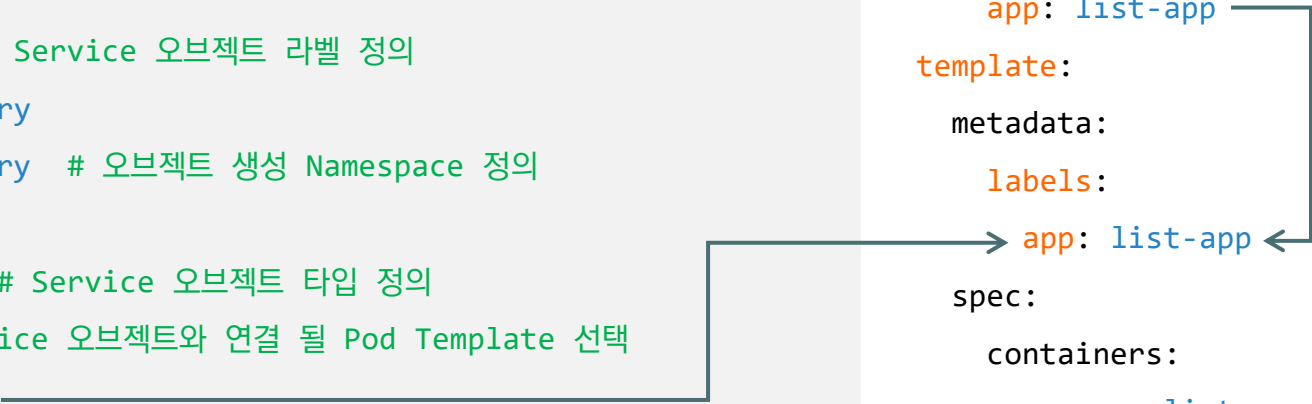
Kubernetes Service Type (EX.1 : ClusterIP, NodePort)

< 작업대상 : svc-clusterip-list.yml >

```
apiVersion: v1
kind: Service
metadata:
  name: list-svc # Service 오브젝트 이름
  labels:
    app: list-app # Service 오브젝트 라벨 정의
    project: delivery
  namespace: delivery # 오브젝트 생성 Namespace 정의
spec:
  type: ClusterIP # Service 오브젝트 타입 정의
  selector: # Service 오브젝트와 연결 될 Pod Template 선택
    app: list-app
  ports:
    - port: 80 # Service 오브젝트 Port 및 연결 될 Pod의 Port 정의
      targetPort: 8000
```

※ [Deployment 오브젝트 전체 내용은 TXT 교안을 참조]

```
spec:
  replicas: 2
  selector:
    matchLabels:
      app: list-app
  template:
    metadata:
      labels:
        app: list-app
    spec:
      containers:
        - name: list-app
          image: chlzzz/kube-image:list
          ports:
            - containerPort: 8000
```



The diagram illustrates the connection between the Service and the Deployment. A line from the 'app: list-app' label in the Service selector points to the 'app: list-app' label in the Deployment pod template metadata. Another line from the 'app: list-app' label in the Deployment pod template metadata points to the 'app: list-app' label in the Deployment spec containers, indicating that the pods created by the Deployment must have this label to be selected by the Service.

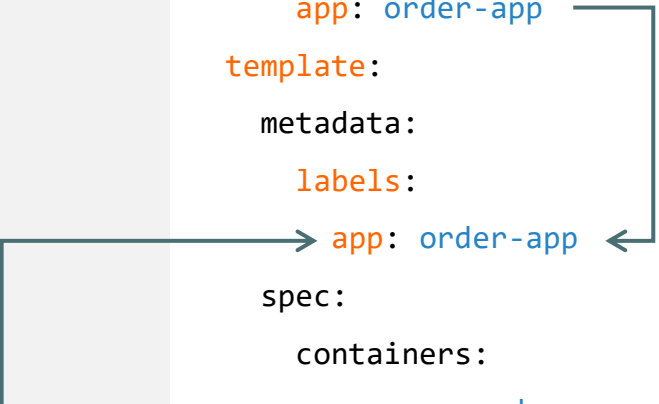
Kubernetes Service Type (EX.1 : ClusterIP, NodePort)

< 작업대상 : svc-clusterip-order.yml >

```
apiVersion: v1
kind: Service
metadata:
  name: order-svc # Service 오브젝트 이름
  labels:
    app: order-app # Service 오브젝트 라벨 정의
    project: delivery
  namespace: delivery # 오브젝트 생성 Namespace 정의
spec:
  type: ClusterIP # Service 오브젝트 타입 정의
  selector: # Service 오브젝트와 연결 될 Pod Template 선택
    app: order-app
  ports:
    - port: 80 # Service 오브젝트 Port 및 연결 될 Pod의 Port 정의
      targetPort: 8000
```

※ [Deployment 오브젝트 전체 내용은 TXT 교안을 참조]

```
spec:
  replicas: 2
  selector:
    matchLabels:
      app: order-app
  template:
    metadata:
      labels:
        app: order-app
    spec:
      containers:
        - name: order-app
          image: chlzzz/kube-image:order
          ports:
            - containerPort: 8000
```



Kubernetes Service Type (EX.1 : ClusterIP, NodePort)

```
$ kubectl create namespace delivery
```

```
namespace/delivery created
```

```
$ kubectl get ns
```

NAME	STATUS	AGE
default	Active	4d4h
delivery	Active	8s
kube-node-lease	Active	4d4h
kube-public	Active	4d4h
kube-system	Active	4d4h

```
$ kubectl apply -f service/
```

```
service/list-svc created
```

```
deployment.apps/list-deploy created
```

```
service/order-svc created
```

```
deployment.apps/order-deploy created
```

▣ 오브젝트가 생성 될 NameSpace를 생성 후 Manifest File을 이용하여 오브젝트를 생성한다.

Kubernetes Service Type (EX.1 : ClusterIP, NodePort)

```
$ kubectl get all -n delivery
```

NAME	READY	STATUS	RESTARTS	AGE
pod/list-deploy-549c9bbb7-fcs75	1/1	Running	0	12s
pod/list-deploy-549c9bbb7-rk5vs	1/1	Running	0	12s
pod/order-deploy-7d6794d495-8glr1	1/1	Running	0	8s
pod/order-deploy-7d6794d495-x2t7j	1/1	Running	0	8s

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
service/list-svc	ClusterIP	10.100.96.94	<none>	80/TCP	12s
service/order-svc	ClusterIP	10.100.32.93	<none>	80/TCP	8s

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/list-deploy	2/2	2	2	12s
deployment.apps/order-deploy	2/2	2	2	8s

NAME	DESIRED	CURRENT	READY	AGE
replicaset.apps/list-deploy-549c9bbb7	2	2	2	12s
replicaset.apps/order-deploy-7d6794d495	2	2	2	8s

▣ kubectl get all 명령어와 "-n" 옵션을 이용하여 지정 된 NameSpace에 생성 된 전체 오브젝트 목록을 확인 할 수 있다.

▣ kubectl get svc,pod 형식의 명령어를 작성 할 경우 지정 된 오브젝트 목록만 확인 할 수 있다.

Kubernetes Service Type (EX.1 : ClusterIP, NodePort)

```
$ kubectl get pod -o wide -n delivery
```

NAME	READY	STATUS	RESTARTS	AGE	IP
list-deploy-549c9bbb7-fcs75	1/1	Running	0	7m40s	192.168.20.93
list-deploy-549c9bbb7-rk5vs	1/1	Running	0	7m40s	192.168.10.58
order-deploy-7d6794d495-8glr1	1/1	Running	0	7m40s	192.168.10.240
order-deploy-7d6794d495-x2t7j	1/1	Running	0	7m40s	192.168.20.22

```
$ kubectl get svc -n delivery
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
service/list-svc	ClusterIP	10.100.96.94	<none>	80/TCP	12s
service/order-svc	ClusterIP	10.100.32.93	<none>	80/TCP	8s

▣ Service 오브젝트의 TYPE, ClusterIP, Port를 확인한다. ("--show-labels -o wide" 옵션 : Label 및 Selector 함께 확인)

```
$ kubectl get endpoints -n delivery
```

NAME	ENDPOINTS	AGE
list-svc	192.168.10.58:8000,192.168.20.93:8000	54s
order-svc	192.168.10.240:8000,192.168.20.22:8000	50s

▣ Service 오브젝트에 연결 된 Endpoints 목록을 확인한다. (배포 된 Pod의 IP주소 및 Port 정보를 등록)

▣ Endpoints 목록에 등록되지 않은 Pod는 Service 오브젝트를 이용한 접근이 불가능하다.

▣ ClusterIP 타입의 Service 오브젝트는 외부접근이 불가능하다. (내부통신만을 지원하기 위한 Service Type)

Kubernetes Service Type (EX.1 : ClusterIP, NodePort)

■ [동일 NameSpace에 생성되는 Pod의 **환경변수**를 이용 한 ClusterIP 통신]

```
$ kubectl run -it --rm debug --image=chlzzz/kube-image:debug --restart=Never -n delivery -- sh
```

```
/ # env | grep LIST
```

```
LIST_SVC_SERVICE_HOST=10.109.112.174
```

```
LIST_SVC_SERVICE_PORT=80
```

```
/ # env | grep ORDER
```

```
ORDER_SVC_SERVICE_HOST=10.110.132.242
```

```
ORDER_SVC_SERVICE_PORT=80
```

■ Kubernetes Cluster 내부 통신 테스트를 위해 통신 확인용 Pod를 생성 후 해당 Pod에 환경변수 목록을 확인한다.

■ 동일 NameSpace에 구성되어있는 Service 오브젝트의 IP주소와 Port번호가 환경변수로 자동 등록되어있는 것을 확인 할 수 있다.

```
/ $ curl $LIST_SVC_SERVICE_HOST:$LIST_SVC_SERVICE_PORT
```

```
/ $ curl $ORDER_SVC_SERVICE_HOST:$ORDER_SVC_SERVICE_PORT
```

■ 환경변수를 이용하여 Service 오브젝트가 연결중인 Pod 집합으로 통신 테스트를 진행한다.

■ Service 오브젝트는 L4기반의 Load Balancing을 수행하여 여러개의 Pod중 하나의 Pod를 선택 후 요청정보를 Redirect 한다.

Kubernetes Service Type (EX.1 : ClusterIP, NodePort)

▣ [Kubernetes Cluster 내부 DNS 서버를 이용한 Cluster IP 통신확인 (동일 Namespace)]

\$ `kubectl get service -n kube-system` # 새로운 터미널 창 활성화 후 명령어 입력

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
kube-dns	ClusterIP	10.100.0.10	<none>	53/UDP,53/TCP	6d23h

▣ ClusterIP TYPE으로 생성 된 kube-dns를 확인한다. (kube-system Namespace 영역에 자동 생성)

▣ Kubernetes Cluster DNS 서버의 IP주소는 10.96.0.10을 부여받아 동작하며, 내부 DNS Query를 처리하도록 구성되어있다. (DNS: UDP/53)

/ \$ `cat /etc/resolv.conf`

`nameserver 10.96.0.10`

/ \$ `curl list-svc.delivery.svc.cluster.local`

/ \$ `curl list-svc.delivery:80`

/ \$ `curl list-svc:80`

/ \$ `curl order-svc.delivery.svc.cluster.local`

/ \$ `curl order-svc.delivery:80`

/ \$ `curl order-svc:80`

/ \$ `exit`

▣ Kubernetes Cluster 내부 DNS 서버를 이용하여 각 Service 오브젝트가 연결중인 Pod 집합으로 통신 테스트를 진행한다.

Kubernetes Service Type (EX.1 : ClusterIP, NodePort)

▣ [Kubernetes Cluster 내부 DNS 서버를 이용한 Cluster IP 통신확인 (서로 다른 Namespace)]

```
$ kubectl run -it --rm debug --image=chlzzz/kube-image:debug --restart=Never -- sh
```

```
/ $ env | grep LIST
```

```
/ $ env | grep ORDER
```

▣ debug Pod를 동일 Namespace가 아닌 default Namespace에서 실행한다.

▣ Pod가 생성 된 Namespace가 아닌 다른 Namespace의 Service 오브젝트 항목은 환경변수로 등록되지 않는다.

```
/ $ curl list-svc.delivery:80
```

```
/ $ curl order-svc.delivery:80
```

```
/ $ exit
```

```
pod "debug" deleted
```

▣ Pod가 생성 된 Namespace가 아닌 다른 Namespace의 Service 오브젝트에 접근하기 위해서는 Cluster 내부 DNS 서버를 사용해야한다.

▣ Service 오브젝트의 Domain Name으로 접근하며, 반드시 해당 Service 오브젝트가 속한 Namespace를 함께 명시해야한다.

```
$ kubectl delete all -l project=delivery -n delivery
```

```
$ kubectl get all -n delivery
```

```
No resources found in delivery namespace.
```

▣ delivery Namespace 오브젝트 중 Label "project=delivery"를 포함하는 모든 오브젝트를 삭제한다.

Kubernetes Service Type (EX.1 : ClusterIP, NodePort)

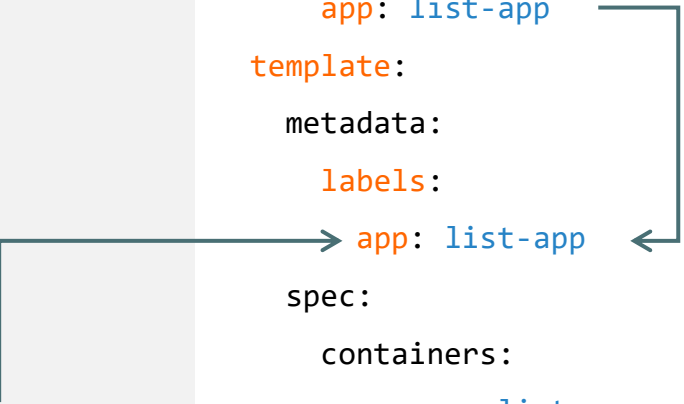
< 작업대상 : svc-nodeport-list.yml >

```
apiVersion: v1
kind: Service
metadata:
  name: list-svc # Service 오브젝트 이름
  labels:
    app: list-app # Service 오브젝트 라벨 정의
    project: delivery
  namespace: delivery # 오브젝트 생성 Namespace 정의
spec:
  type: NodePort # Service 오브젝트 타입 정의
  selector: # Service 오브젝트와 연결 될 Pod Template 선택
    app: list-app
  ports:
    - port: 80 # Service 오브젝트 Port 및 연결 될 Pod의 Port 정의
      targetPort: 8000

# Node Open Port 범위 ( 기본값 : 30000 ~ 32767 )
# Node Open Port 지정 ( ports 하위 속성 : nodePort: 30001 )
```

※ [Deployment 오브젝트 전체 내용은 TXT 교안을 참조]

```
spec:
  replicas: 2
  selector:
    matchLabels:
      app: list-app
  template:
    metadata:
      labels:
        app: list-app
    spec:
      containers:
        - name: list-app
          image: chlzzz/kube-image:list
          ports:
            - containerPort: 8000
```



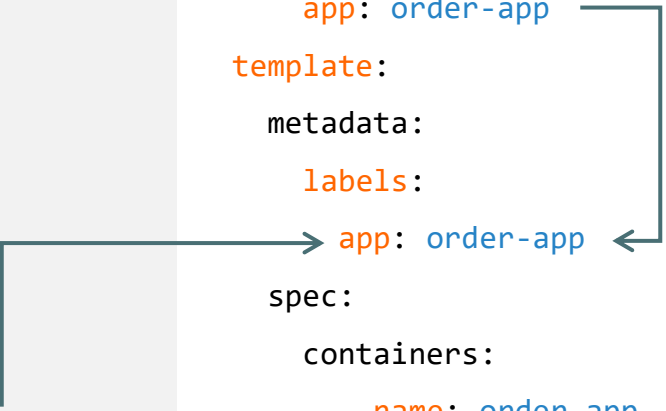
Kubernetes Service Type (EX.1 : ClusterIP, NodePort)

< 작업대상 : svc-nodeport-order.yml >

```
apiVersion: v1
kind: Service
metadata:
  name: order-svc # Service 오브젝트 이름
  labels:
    app: order-app # Service 오브젝트 라벨 정의
    project: delivery
  namespace: delivery # 오브젝트 생성 Namespace 정의
spec:
  type: NodePort # Service 오브젝트 타입 정의
  selector: # Service 오브젝트와 연결 될 Pod Template 선택
    app: order-app
  ports:
    - port: 80 # Service 오브젝트 Port 및 연결 될 Pod의 Port 정의
      targetPort: 8000
```

※ [Deployment 오브젝트 전체 내용은 TXT 교안을 참조]

```
spec:
  replicas: 2
  selector:
    matchLabels:
      app: order-app
  template:
    metadata:
      labels:
        app: order-app
    spec:
      containers:
        - name: order-app
          image: chlzzz/kube-image:order
          ports:
            - containerPort: 8000
```



Kubernetes Service Type (EX.1 : ClusterIP, NodePort)

```
$ kubectl apply -f service/svc-nodeport-list.yml
```

```
$ kubectl apply -f service/svc-nodeport-order.yml
```

```
$ kubectl get all -n delivery
```

NAME	READY	STATUS	RESTARTS	AGE
pod/list-deploy-549c9bbb7-jpfbn	1/1	Running	0	7s
pod/list-deploy-549c9bbb7-vcqkf	1/1	Running	0	7s
pod/order-deploy-7d6794d495-dbsdj	1/1	Running	0	4s
pod/order-deploy-7d6794d495-g78x5	1/1	Running	0	4s

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
service/list-svc	NodePort	10.100.62.50	<none>	80:30140/TCP	7s
service/order-svc	NodePort	10.100.80.181	<none>	80:32455/TCP	4s

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/list-deploy	2/2	2	2	7s
deployment.apps/order-deploy	2/2	2	2	4s

NAME	DESIRED	CURRENT	READY	AGE
replicaset.apps/list-deploy-549c9bbb7	2	2	2	7s
replicaset.apps/order-deploy-7d6794d495	2	2	2	4s

Kubernetes Service Type (EX.1 : ClusterIP, NodePort)

```
$ kubectl get pod -n delivery -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
list-deploy-549c9bbb7-jpfbn	1/1	Running	0	2m39s	192.168.20.169	
list-deploy-549c9bbb7-vcqkf	1/1	Running	0	2m39s	192.168.10.58	
order-deploy-7d6794d495-dbsdj	1/1	Running	0	2m36s	192.168.10.132	
order-deploy-7d6794d495-g78x5	1/1	Running	0	2m36s	192.168.20.181	

```
$ kubectl get endpoints -n delivery
```

NAME	ENDPOINTS	AGE
list-svc	192.168.10.58:8000,192.168.20.169:8000	100s
order-svc	192.168.10.132:8000,192.168.20.181:8000	97s

▣ Pod의 IP주소정보가 Endpoints로 등록되었는지 확인

```
$ kubectl get svc -n delivery
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
list-svc	NodePort	10.100.62.50	<none>	80:30140/TCP	2m19s
order-svc	NodePort	10.100.80.181	<none>	80:32455/TCP	2m16s

▣ Service 오브젝트 정보 확인 ([List APP Open Port:30140], [Order APP Open Port:32455])

Kubernetes Service Type (EX.1 : ClusterIP, NodePort)

■ [NodePort Type Service 오브젝트 외부 통신 테스트]

- . 정리 : List APP 접근 시 30140번 Port 사용 / Order APP 접근 시 32455번 Port 사용
- . 해당 Service 오브젝트가 연결 중인 Node의 Port번호에 맞는 방화벽 규칙이 활성화되어 있어야 한다.
- . Worker Node에 연결 된 보안그룹 규칙 인바운드 편집 -> 소스: BastionHost SG -> TCP/30000 - 32767, ICMP 허용 2개의 규칙을 추가
- . BastionHost에서 NodePort Service 오브젝트를 이용하여 각 Node에서 실행중인 Pod로 연결 테스트를 진행한다.
- . `curl 192.168.10.147:30140(node-1 : list )` , `curl 192.168.10.147:32455 (node-1 : order )`
- . `curl 192.168.20.156:30140(node-2 : list )` , `curl 192.168.20.156:32455 (node-2 : order )`
- . 연결 테스트 완료 후 보안그룹 규칙에 새로 추가한 인바운드 규칙은 제거한다.

```
$ kubectl delete all -l project=delivery -n delivery
```

```
service "list-svc" deleted
```

```
service "order-svc" deleted
```

```
deployment.apps "deploy-svc" deleted
```

```
deployment.apps "order-deploy" deleted
```

```
$ kubectl get all -n delivery
```

```
No resources found in delivery namespace.
```

■ delivery NameSpace 오브젝트 중 Label "project=delivery"를 포함하는 모든 오브젝트를 삭제한다.

Kubernetes Service Type (EX.2 : ExternalName)

■ [ExternalName Test 시나리오]

- . External System(EX:RDS) 및 DNS(EX:Route53)가 구축되어있어야 완벽한 테스트가 가능하므로 가상의 시나리오를 바탕으로 테스트를 진행한다.
- . **delivery** NameSpace는 Kubernetes Cluster **외부 시스템** / **default** NameSpace는 Kubernetes Cluster **내부 시스템**
- . ExternalName Service 오브젝트 TEST를 위해 기존 ClusterIP Service 오브젝트를 먼저 생성 후 확인한다.

```
$ kubectl apply -f service/svc-clusterip-list.yml
```

```
service/list-svc created / deployment.apps/list-deploy created
```

```
$ kubectl apply -f service/svc-clusterip-order.yml
```

```
service/order-svc created / deployment.apps/order-deploy created
```

```
$ kubectl get svc -n delivery
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
list-svc	ClusterIP	10.100.241.80	<none>	80/TCP	12s
order-svc	ClusterIP	10.100.195.147	<none>	80/TCP	8s

```
$ kubectl get endpoints -n delivery
```

NAME	ENDPOINTS	AGE
list-svc	192.168.10.22:8000,192.168.20.89:8000	53s
order-svc	192.168.10.58:8000,192.168.20.179:8000	51s

Kubernetes Service Type (EX.2 : ExternalName)

< 작업대상 : svc-externalname.yml >

```
apiVersion: v1
```

```
kind: Service
```

```
metadata:
```

```
  name: externalname-list-svc
```

```
# 기본 NameSpace default에 ExternalName 오브젝트를 생성
```

```
# ExternalName 오브젝트 이름정의 ( 통신 시 사용 )
```

```
spec:
```

```
  type: ExternalName
```

```
# Service Type 정의 ( ExternalName )
```

```
  externalName: list-svc.delivery.svc.cluster.local
```

```
# ExternalName 오브젝트가 연결 할 외부 시스템 Domain Name 정의
```

```
--- # 하나의 Manifest File에 여러 개의 오브젝트 정의 시 오브젝트 구분을 위해 사용
```

```
apiVersion: v1
```

```
kind: Service
```

```
metadata:
```

```
  name: externalname-order-svc
```

```
# 기본 NameSpace default에 ExternalName 오브젝트를 생성
```

```
# ExternalName 오브젝트 이름정의 ( 통신 시 사용 )
```

```
spec:
```

```
  type: ExternalName
```

```
# Service Type 정의 ( ExternalName )
```

```
  externalName: order-svc.delivery.svc.cluster.local
```

```
# ExternalName 오브젝트가 연결 할 외부 시스템 Domain Name 정의
```


Kubernetes Service Type (EX.2 : ExternalName)

```
$ kubectl apply -f service/svc-externalname.yml
```

```
service/externalname-list-svc created
```

```
service/externalname-order-svc created
```

```
$ kubectl get svc -n default
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
externalname-list-svc	ExternalName	<none>	list-svc.delivery.svc.cluster.local	<none>	66s
externalname-order-svc	ExternalName	<none>	order-svc.delivery.svc.cluster.local	<none>	66s
kubernetes	ClusterIP	10.100.0.1	<none>		

■ ExternalName 오브젝트가 생성 된 것을 확인한다. (EXTERNAL-IP : 외부 시스템 Domain Name)

■ TEST 환경 : deilvery Namespace에 생성 된 ClusterIP Service 오브젝트의 이름이 정의되어있다. (내부 DNS서버에 등록)

```
# kubectl run -it --rm debug --image=chlzzz/kube-image:debug --restart=Never -- sh
```

```
/ $ curl externalname-list-svc
```

```
/ $ curl externalname-order-svc
```

■ ExternalName 오브젝트의 이름인 externalname-<list or order>-svc를 이용하여 서비스 요청 작업을 수행한다.

■ 요청처리 순서 : Debug Pod -> ExternalName -> Delivery Namespcae Pod

Kubernetes Service Type (EX.2 : ExternalName)

```
$ kubectl delete all -l project=delivery -n delivery
```

```
service "list-svc" deleted
```

```
service "order-svc" deleted
```

```
deployment.apps "deploy-svc" deleted
```

```
deployment.apps "order-deploy" deleted
```

```
$ kubectl get all -n delivery
```

```
No resources found in delivery namespace.
```

▣ delivery NameSpace 오브젝트 중 Label "project=delivery"를 포함하는 모든 오브젝트를 삭제한다.

```
$ kubectl delete svc externalname-list-svc
```

```
service "externalname-list-svc" deleted
```

```
$ kubectl delete svc externalname-order-svc
```

```
service "externalname-order-svc" deleted
```

```
$ kubectl get all
```

```
No resources found in default namespace.
```

▣ default NameSpace에 생성되어있는 ExternalName 오브젝트를 삭제한다.

Kubernetes Service Type (EX.3 : LoadBalancer)

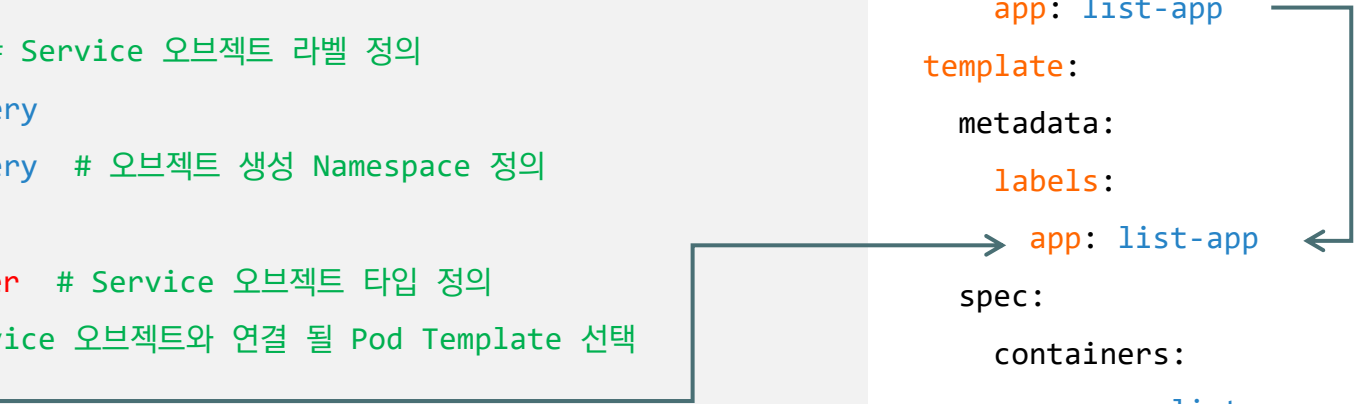
< 작업대상 : svc-loadbalancer.yml >

```
apiVersion: v1
kind: Service
metadata:
  name: list-svc # Service 오브젝트 이름
  labels:
    app: list-app # Service 오브젝트 라벨 정의
    project: delivery
  namespace: delivery # 오브젝트 생성 Namespace 정의
spec:
  type: LoadBalancer # Service 오브젝트 타입 정의
  selector: # Service 오브젝트와 연결 될 Pod Template 선택
    app: list-app
  ports:
    - port: 80 # Service 오브젝트 Port 및 연결 될 Pod의 Port 정의
      targetPort: 8000

# 기존 NodePort Manifest을 복사하여 LoadBalancer Manifest File 구성
# Service Type: LoadBalancer 변경 / Pod Template Replicas: 3 변경
```

※ [Deployment 오브젝트 전체 내용은 TXT 교안을 참조]

```
spec:
  replicas: 3
  selector:
    matchLabels:
      app: list-app
  template:
    metadata:
      labels:
        app: list-app
    spec:
      containers:
        - name: list-app
          image: chlzzz/kube-image:list
          ports:
            - containerPort: 8000
```



The diagram illustrates the connection between the Service and Deployment manifests. A line from the 'app: list-app' label in the Service selector points to the 'app: list-app' label in the Deployment pod template metadata. Another line from the 'app: list-app' label in the Deployment pod template metadata points back to the 'app: list-app' label in the Service selector, forming a loop that indicates the selector must match the pod labels.

Kubernetes Service Type (EX.3 : LoadBalancer)

```
$ kubectl apply -f service/svc-loadbalancer.yml
```

```
$ kubectl get pod -n delivery -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP
list-deploy-549c9bbb7-rjqh4	1/1	Running	0	33m	192.168.10.187
list-deploy-549c9bbb7-sdrcl	1/1	Running	0	33m	192.168.10.58
list-deploy-549c9bbb7-v29zg	1/1	Running	0	33m	192.168.20.205

```
$ kubectl get svc -n delivery
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
service/list-svc	LoadBalancer	10.100.217.107	ap-northeast-2.elb.amazonaws.com	80:31186/TCP	11s

- 완전 관리형(EKS) K8s환경에서는 Service Type을 LoadBalancer로 지정하면 LoadBalancer가 자동으로 생성되고 연결된다.
- Service 오브젝트의 TYPE, EXTERNAL-IP 주소를 확인한다. (EXTERNAL-IP: 외부연결이 가능 한 AWS ELB의 Endpoint 주소)
- 로컬 PC에서 "크롬" 웹 브라우저에서 접속 테스트를 진행
- ELB Endpoint 주소로 접속 / 캐시 지우기 -> 새탭 -> 반복 -> 접속 Pod 변경)
- ELB Endpoint 주소로 반복적인 요청을 진행 (접속 Pod의 IP주소 정보가 바뀌는 것을 확인)
- 웹 서비스 운영 시 접속 Pod의 정보가 변경되는 경우 Session 정보가 유지되지 않아 많은 문제가 발생하게 된다.
- EX: A Pod에 접속 후 로그인 수행 / 재 연결 시 B Pod와 연결 될 경우 재 로그인 (Cokie Data : 장바구니 정보 초기화 등)
- Session 변경 문제를 해결하기 위해 "Session Affinty" 설정을 진행한다. (Session 유지 기능)

Kubernetes Service Type (EX.3 : LoadBalancer)

■ [Session Affinity]

- . Session Affinity 설정 방법은 Service 오브젝트에서 진행하는 방법과 Ingress 오브젝트에서 진행하는 방법 2가지로 구분된다.
- . Service 오브젝트에서 지원하는 Session Affinity는 **L3를 기반**으로 동작한다. (접속 Client IP주소 정보를 활용)
- . Ingress 오브젝트에서 지원하는 Session Affinity는 **L7을 기반**으로 동작한다. (Client Cookie 정보를 활용)

```
$ kubectl get svc -n delivery
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
service/list-svc	LoadBalancer	10.100.217.107	ap-northeast-2.elb.amazonaws.com	80:31186/TCP	11s

```
$ kubectl edit svc list-svc -n delivery
```

```
selector:
```

```
  app: list-app
```

```
sessionAffinity: ClientIP
```

```
sessionAffinityConfig:
```

```
  clientIP:
```

```
    timeoutSeconds: 10800
```

```
configmap/kube-proxy edited
```

- kubectl edit 명령어를 이용하여 현재 동작중인 list-svc 오브젝트에 SessionAffinity 속성값을 추가로 정의한다.
- Session Affinity Type 및 Client IP 주소 정보를 유지할 시간을 설정한다. (기본값 : 10800초)

Kubernetes Service Type (EX.3 : LoadBalancer)

POD CONTAINER INFORMATION

POD IP Address	192.168.10.58
POD HostName	list-deploy-549c9bbb7-sdrcm
POD SVC Infomation	10.100.217.107:80

접속 Pod의 정보가 Session Affinity에 의해 변경되지 않는다.

▣ 로컬 PC에서 "크롬" 웹 브라우저에서 접속 테스트를 진행 (접속 Pod의 IP주소 정보가 바뀌지 않는 것을 확인)

▣ 지정 된 Session Timeout 시간까지 Client IP주소와 접속 Pod 정보를 기억한다.

```
$ kubectl delete -f service/svc-loadbalancer.yml
```

```
service "list-svc" deleted
```

```
deployment.apps "list-deploy" deleted
```

```
$ kubectl get all -n delivery
```

```
No resources found in delivery namespace.
```

▣ 다음 테스트를 위하여 오브젝트 생성에 사용한 Manifest File을 이용하여 오브젝트 삭제 작업을 수행한다.

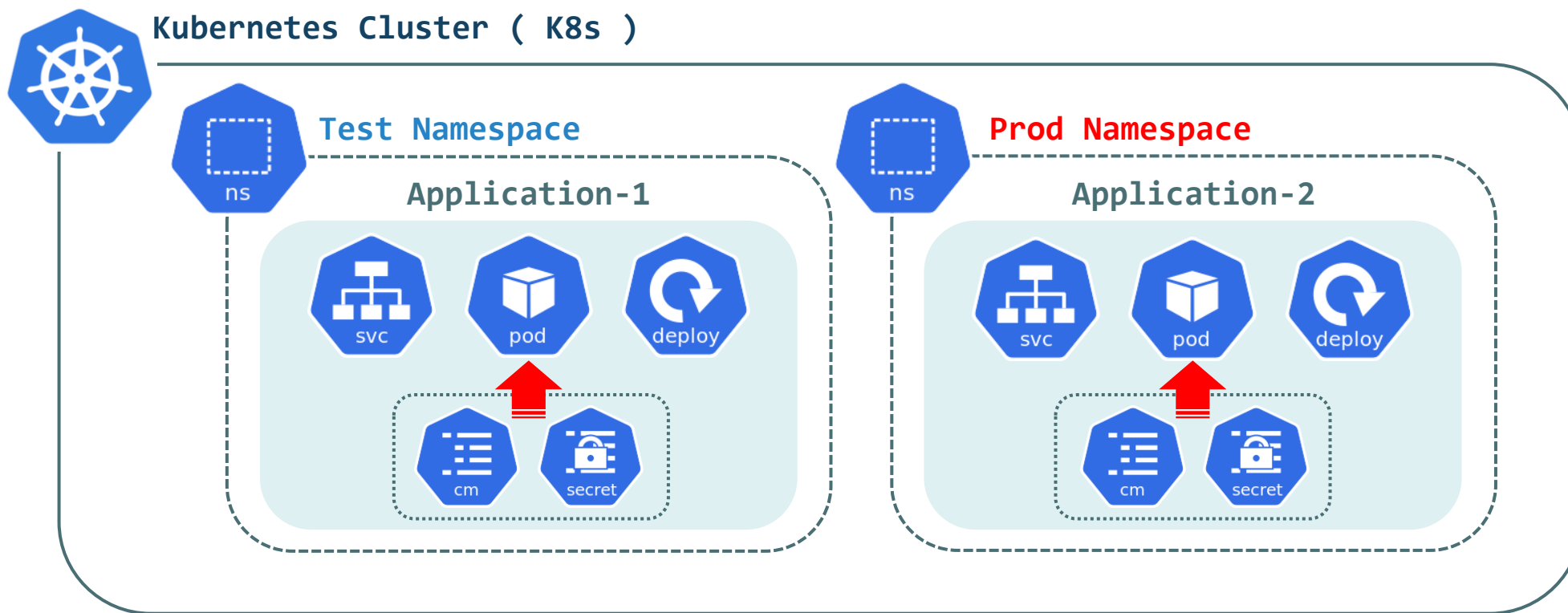


Kubernetes ConfigMap & Secret

Kubernetes ConfigMap & Secret Object

© Kubernetes ConfigMap & Secret Object

- ▶ Kubernetes Cluster 운영 환경에서는 App 설정 정보나 인증 정보와 같은 민감 정보를 Pod 내부에서 보관하거나 관리하지 않는다.
- ▶ App 설정 정보를 저장하는 ConfigMap 오브젝트를 사용하고, 인증정보와 같은 민감 정보는 Secret 오브젝트를 사용하여 관리된다.
- ▶ ConfigMap 오브젝트와 Secret 오브젝트는 NameSpace 단위로 적용되며 동일 NameSpace에 배포 된 Pod에서 참조하여 사용된다.
- ▶ 참조 방법으로는 Pod에서 ConfigMap, Secret 오브젝트를 FileSystem으로 Mount하거나 환경변수(ENV)를 이용하여 참조할 수 있다.



Kubernetes ConfigMap & Secret Object

◎ Kubernetes ConfigMap Object 특징

- ▶ Application이 사용하는 다양한 설정 정보를 **KEY = Value** 형식으로 정의한다.
- ▶ ConfigMap 설정 정보값은 컨테이너 내부에서 키값으로 참조되므로 설정 정보에 해당하는 Value값을 자유롭게 변경 할 수 있다.
- ▶ ConfigMap 오브젝트에는 DB 접속정보, 모니터링 서비스 연결정보, 다른 App의 IP주소 및 Port정보 등을 주로 정의한다.
- ▶ ConfigMap 오브젝트는 Namespace 단위로 나누어 관리 할 수 있다. (EX: 개발 환경 DB 접속 정보 / 운영 환경 DB 접속 정보)
- ▶ ConfigMap 오브젝트 정의 시 Directory 혹은 File을 지정하여 데이터로 정의 할 수 있다. (파일이름: KEY, 파일내용: Value)
- ▶ ConfigMap 오브젝트 정의 시 **서버 프로그램의 설정파일 전체를 데이터로 정의** 할 수 있다. (EX: Apache, Nginx 설정 파일)
- ▶ ConfigMap 오브젝트를 이용하여 Image의 설정값이 변경되었을 경우 해당 **Image를 다시 Build해야하는 문제를 해결** 할 수 있다.
- ▶ ConfigMap 오브젝트 업데이트 시 환경변수를 이용한 참조를 사용하는 경우 Pod에 해당 변경사항은 자동 적용되지 않는다.
- ▶ ConfigMap 오브젝트 업데이트 시 **Volume Mount를 이용한 참조**를 사용하는 경우 Pod에 해당 **변경사항을 자동적용** 한다.

◎ Kubernetes Secret Object 특징

- ▶ SSH Key, SSL/TLS 인증서, Service Account Token 정보와 같은 **민감 정보를 Namespace 단위로 관리** 할 수 있다.
- ▶ Secret 오브젝트는 Secret 오브젝트가 생성 된 Namespace에서만 사용 가능하다. (다른 Namespace에서는 참조 할 수 없다.)
- ▶ Secret 오브젝트의 데이터는 **Base64로 인코딩되어 저장**되며, Pod에서 참조 시 데이터를 디코딩하여 메모리에 저장하여 사용 된다.
- ▶ Secret 오브젝트는 ConfigMap 오브젝트와 동일한 방법으로 사용되며, **Secret 오브젝트의 크기는 1MB 미만**이어야 한다.
- ▶ Pod 생성 시 해당 Pod에서 특정 Secret 오브젝트 사용을 명시한 경우 반드시 Secret 오브젝트가 존재해야 Pod 생성이 가능하다.

Kubernetes ConfigMap & Secret (EX.1 : Basic [CM])

```
$ kubectl create configmap my-cm --from-literal=NAME="Kube" \  
> --from-literal=MESSAGE="Hello Kubernetes" -n delivery
```

configmap/my-cm created

```
$ kubectl get cm -n delivery
```

NAME	DATA	AGE
my-cm	2	42s

```
$ kubectl get cm -o yaml -n delivery
```

```
apiVersion: v1
```

```
items:
```

```
- apiVersion: v1
```

```
  data:
```

```
    MESSAGE: Hello Kubernetes
```

```
    NAME: Kube
```

```
  kind: ConfigMap
```

- ▣ kubectl create configmap 명령어를 이용하여 delivery Namespace에 ConfigMap 오브젝트를 생성한다.
- ▣ ConfigMap 데이터는 "--from-literal=KEY=VALUE" 형식으로 정의한다. (NAME="Kube" / MESSAGE="Hello Kubernetes")
- ▣ 생성 된 ConfigMap 오브젝트를 YAML 형식으로 출력 후 정의 된 데이터 목록 및 오브젝트의 속성정보등을 확인한다.

Kubernetes ConfigMap & Secret (EX.1 : Basic [CM])

< 작업대상 : cm-pod-env-valuefrom.yml >

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-valuefrom # Pod 오브젝트 이름
  labels:
    app: cm-test      # Pod 오브젝트 라벨 정의
  namespace: delivery # 오브젝트 생성 Namespace 정의
```

```
apiVersion: v1
items:
- apiVersion: v1
  data:
    MESSAGE: Hello Kubernetes
    NAME: Your Name
  kind: ConfigMap
```

ConfigMap Object
[my-cm]



Pod 환경변수를 이용한 ConfigMap 참조
(env.valueFrom.configMapKeyRef)

```
spec:
  containers:
    - name: my-pod
      image: chlzzz/kube-image:debug
      env:
        - name: NAME
          valueFrom:
            configMapKeyRef:
              name: my-cm
              key: NAME
        - name: MESSAGE
          valueFrom:
            configMapKeyRef:
              name: my-cm
              key: MESSAGE
      command: ['sh', '-c', 'tail -f /dev/null' ]
```

Kubernetes ConfigMap & Secret (EX.1 : Basic [CM])

```
$ kubectl apply -f ./cm-pod-env-valuefrom.yml
```

```
pod/pod-valuefrom created
```

```
$ kubectl get pod -n delivery
```

NAME	READY	STATUS	RESTARTS	AGE
pod-valuefrom	1/1	Running	0	9s

```
$ kubectl exec pod-valuefrom -n delivery -- env
```

```
NAME=Your Name
```

```
MESSAGE=Hello Kubernetes
```

```
$ kubectl delete pod pod-valuefrom -n delivery
```

```
pod "pod-valuefrom" deleted
```

- ▣ Pod의 오브젝트의 환경변수 목록을 출력 후 ConfigMap에서 정의한 Value 값을 갖는 NAME, MESSAGE 환경변수를 확인 한다.
- ▣ 다음 TEST를 위해 생성 한 Pod 오브젝트 삭제 작업을 진행한다.

Kubernetes ConfigMap & Secret (EX.1 : Basic [CM])

< 작업대상 : cm-pod-env-envfrom.yml >

apiVersion: v1

kind: Pod

metadata:

name: pod-envfrom # Pod 오브젝트 이름

labels:

app: cm-test # Pod 오브젝트 라벨 정의

namespace: delivery # 오브젝트 생성 Namespace 정의

apiVersion: v1

items:

- apiVersion: v1

data:

MESSAGE: Hello Kubernetes

NAME: Your Name

kind: ConfigMap

ConfigMap Object

[my-cm]



Pod 환경변수를 이용한 ConfigMap 참조
(envFrom.configMapRef)

spec:

containers:

- name: my-pod

image: chlzzz/kube-image:debug

envFrom:

- configMapRef:

name: my-cm

command: ['sh','-c','tail -f /dev/null']

ConfigMap 오브젝트 데이터를 참조하여 환경변수를 정의

Kubernetes ConfigMap & Secret (EX.1 : Basic [CM])

```
$ kubectl apply -f configmap_secret/cm-pod-env-envfrom.yml
```

```
pod/pod-envfrom created
```

```
$ kubectl get pod -n delivery
```

NAME	READY	STATUS	RESTARTS	AGE
pod-envfrom	1/1	Running	0	38s

```
$ kubectl exec pod-envfrom -n delivery -- env
```

```
NAME= Your Name
```

```
MESSAGE=Hello Kubernetes
```

```
$ kubectl delete pod pod-envfrom -n delivery
```

```
pod "pod-envfrom" deleted
```

```
$ kubectl delete cm my-cm -n delivery
```

```
configmap "my-cm" deleted
```

- ▣ Pod의 오브젝트의 환경변수 목록을 출력 후 ConfigMap에서 정의한 Value 값을 갖는 NAME, MESSAGE 환경변수를 확인 한다.
- ▣ 다음 TEST를 위해 생성 한 Pod 및 ConfigMap 오브젝트 삭제 작업을 진행한다.

Kubernetes ConfigMap & Secret (EX.1 : Basic [CM])

```
$ mkdir configmap_secret/config
```

```
$ echo "Your Name" > configmap_secret/config/NAME
```

```
$ echo "Hello Kubernetes" > configmap_secret/config/MESSAGE
```

```
$ tree configmap_secret/config
```

```
configmap_secret/config
```

```
├── MESSAGE
```

```
└── NAME
```

```
$ kubectl create configmap my-cm --from-file=configmap_secret/config -n delivery
```

```
$ kubectl get cm -n delivery
```

NAME	DATA	AGE
kube-root-ca.crt	1	6h33m
my-cm	2	25s

- ▣ 파일형태의 ConfigMap을 구성하기위하여 config 디렉터리 하위에 파일2개를 생성한다.
- ▣ 파일형태의 ConfigMap을 구성 할 경우 파일이름이 KEY값이 되고 파일내용이 VALUE 값으로 정의된다.
- ▣ ConfigMap 오브젝트를 생성 (데이터는 "--from-file=<file or dir>" 형식을 이용하여 config directory 하위 File 전체를 지정)

Kubernetes ConfigMap & Secret (EX.1 : Basic [CM])

```
$ kubectl get cm my-cm -o yaml -n delivery
```

```
apiVersion: v1
```

```
data:
```

```
  MESSAGE: |  
    Hello Kubernetes
```

```
  NAME: |  
    Your Name
```

```
$ kubectl apply -f configmap_secret/cm-pod-env-envfrom.yaml
```

```
$ kubectl exec pod-envfrom -n delivery -- env
```

```
NAME=Your Name
```

```
MESSAGE=Hello Kubernetes
```

- 이전 테스트에서 사용 한 envfrom Pod를 이용하여 ConfigMap 테스트를 수행한다.
- 생성 된 Pod의 환경변수 목록을 출력하여 ConfigMap에서 정의한 데이터를 갖는 NAME, MESSAGE 환경변수를 확인 한다.

```
$ kubectl delete pod pod-envfrom -n delivery
```

```
$ kubectl delete cm my-cm -n delivery
```

- 다음 TEST를 위해 생성 한 Pod 및 ConfigMap 오브젝트 삭제 작업을 진행한다.

Kubernetes ConfigMap & Secret (EX.1 : Basic [Secret])

```
$ kubectl run mysql-pod --image=mysql:5.7 -n delivery
```

```
$ kubectl get pod mysql-pod -n delivery
```

NAME	READY	STATUS	RESTARTS	AGE
mysql-pod	0/1	CrashLoopBackOff	1	11s

```
$ kubectl logs mysql-pod -n delivery
```

```
2022-07-25 07:24:53+00:00 [Note] [Entrypoint]: Entrypoint script for MySQL Server 5.7.38-1.el7 started.
```

```
2022-07-25 07:24:53+00:00 [Note] [Entrypoint]: Switching to dedicated user 'mysql'
```

```
2022-07-25 07:24:53+00:00 [Note] [Entrypoint]: Entrypoint script for MySQL Server 5.7.38-1.el7 started.
```

```
2022-07-25 07:24:53+00:00 [ERROR] [Entrypoint]: Database is uninitialized and password option is not specified
```

```
You need to specify one of the following:
```

- MYSQL_ROOT_PASSWORD
- MYSQL_ALLOW_EMPTY_PASSWORD
- MYSQL_RANDOM_ROOT_PASSWORD

```
$ kubectl delete pod mysql-pod -n delivery
```

■ MySQL 컨테이너의 경우 실행 시 관리자의 PW가 반드시 지정되어야 프로세스 실행이 가능하다. (Pod 오브젝트 삭제 진행)

■ 현재 Pod 구성 시 관리자 PW를 지정하는 환경변수를 정의하지 않았으므로 컨테이너 동작이 불가능한 상태가 된다.

Kubernetes ConfigMap & Secret (EX.1 : Basic [Secret])

```
$ kubectl run mysql-pod --env=MYSQL_ROOT_PASSWORD=root --image=mysql:5.7 -n delivery
```

```
$ kubectl get pod mysql-pod -n delivery
```

NAME	READY	STATUS	RESTARTS	AGE
mysql-pod	1/1	Running	0	16s

```
$ kubectl describe pod mysql-pod -n delivery
```

Containers:

mysql-pod:

Container ID: docker://d97dfa281280ccc33ae36bfd0745d47929e75995769bc1aa7e86227ab4a5f9da

Image: mysql:5.7

Image ID: docker-pullable://mysql@sha256:4279d155f8cab19149c6078b20d53976f1498e31d6f848ac83e11323909b41f1

State: Running

Environment:

MYSQL_ROOT_PASSWORD: root

```
$ kubectl delete pod mysql-pod -n delivery
```

- 관리자의 PW를 정의하는 환경변수 정의하여 Pod 오브젝트 생성 후 Pod 오브젝트의 상태정보 및 상세정보를 확인한다. (확인 후 Pod 오브젝트 삭제)
- Pod 생성 시 정의한 관리자의 PW 정보의 값이 그대로 노출되어있는 것을 확인 (DB의 관리자 PW 정보는 절대로 외부에 노출되어서는 안된다)

Kubernetes ConfigMap & Secret (EX.1 : Basic [Secret])

```
$ kubectl create secret generic mysql-pass --from-literal MYSQL_ROOT_PASSWORD=root -n delivery
secret/mysql-pass created
```

```
$ kubectl get secret -n delivery
```

NAME	TYPE	DATA	AGE
mysql-pass	Opaque	1	21s

- ▣ Secret 오브젝트를 생성하여 MYSQL_ROOT_PASSWORD 환경변수의 값을 지정하여 저장한다. (ConfigMap 오브젝트와 동일한 방법으로 정의)
- ▣ 일반적인 Secret 오브젝트의 타입은 Generic 형식을 지정하며, Data의 종류에 따라 타입이 변경 될 수 있다. (EX: SSL/TLS Type: "tls")

```
$ kubectl get secret mysql-pass -n delivery -o yaml
```

```
apiVersion: v1
```

```
data:
```

```
  MYSQL_ROOT_PASSWORD: cm9vdA==
```

```
kind: Secret
```

- ▣ 생성 된 Secret 오브젝트를 YAML 형식으로 출력 후 정의 된 데이터 목록 및 오브젝트의 속성정보등을 확인한다.
- ▣ Secret 오브젝트 데이터 확인 (Base64 형식의 인코딩 후 데이터가 저장되어있는 것을 확인)

Kubernetes ConfigMap & Secret (EX.1 : Basic [Secret])

< 작업대상: secret-pod-env-envfrom.yml >

```
apiVersion: v1
```

```
kind: Pod
```

```
metadata:
```

```
  name: mysql-pod      # Pod 오브젝트 이름
```

```
  labels:
```

```
    app: secret-test   # Pod 오브젝트 라벨 정의
```

```
  namespace: delivery # 오브젝트 생성 Namespace 정의
```

```
apiVersion: v1
```

```
data:
```

```
  MYSQL_ROOT_PASSWORD: cm9vdA==
```

```
kind: Secret
```

Secret Object

[**mysql-pass**]



Pod 환경변수를 이용한 Secret 참조
(**envFrom.secretRef**)

```
spec:
```

```
  containers:
```

```
    - name: my-pod
```

```
      image: mysql:5.7
```

```
      envFrom:
```

```
        - secretRef:
```

```
          name: mysql-pass
```

Secret 오브젝트를 참조 속성 : envFrom.secretRef
Pod 생성 전 반드시 Secret 오브젝트가 존재해야한다.

Kubernetes ConfigMap & Secret (EX.1 : Basic [Secret])

```
$ kubectl apply -f configmap_secret/secret-pod-env-envfrom.yml
```

```
pod/pod-envfrom created
```

```
$ kubectl get pod -n delivery
```

NAME	READY	STATUS	RESTARTS	AGE
mysql-pod	1/1	Running	0	7s

```
$ kubectl describe pod mysql-pod -n delivery
```

Containers:

my-pod:

State: **Running**

Started: Mon, 25 Jul 2022 16:58:08 +0900

Ready: True

Restart Count: 0

Environment Variables from:

mysql-pass Secret Optional: false

Environment: <none>

- ▣ Pod 오브젝트에 정의 된 환경변수가 직접적으로 표시되지 않는것을 확인 할 수 있다.
- ▣ Secret 오브젝트를 이용하여 보안이 필요한 값에 대한 직접적인 노출을 막을 수 있다.

Kubernetes ConfigMap & Secret (EX.1 : Basic [Secret])

```
$ kubectl exec -it mysql-pod -n delivery -- sh
```

```
sh-4.2# env | grep MYSQL_ROOT_PASSWORD
```

```
MYSQL_ROOT_PASSWORD=root
```

▣ Container 내부에 접속 후 환경변수를 확인 (환경변수의 값이 실제 참조되어 사용 될 때에는 Base64 디코딩 작업을 진행)

```
sh-4.2# mysql -u root -p
```

```
Enter password: "root"
```

```
Your MySQL connection id is 59
```

```
Server version: 5.7.38 MySQL Community Server (GPL)
```

```
Copyright (c) 2000, 2022, Oracle and/or its affiliates.
```

```
Oracle is a registered trademark of Oracle Corporation and/or its
```

```
affiliates. Other names may be trademarks of their respective
```

```
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.
```

```
mysql> exit
```

```
$ kubectl delete pod mysql-pod -n delivery
```

```
$ kubectl delete secret mysql-pass -n delivery
```

▣ 다음 TEST를 위해 생성 한 Pod 및 Secret 오브젝트 삭제 작업을 진행한다.

Kubernetes ConfigMap & Secret (EX.2 : SSL / TLS)

```
$ vi ~/configmap_secret/config/tls.conf
```

```
ssl_protocols TLSv1 TLSv1.1 TLSv1.2;
server {
    listen 443 ssl;
    ssl_certificate /etc/cert/tls.crt;
    ssl_certificate_key /etc/cert/tls.key;

    location / {
        root /usr/share/nginx/html;
        index index.html;
    }
}
```

```
$ rpm -qa | grep openssl
```

```
openssl-1.0.2k-24.amzn2.0.7.x86_64
```

```
openssl-libs-1.0.2k-24.amzn2.0.7.x86_64
```

▣ Nginx WEB 서버의 SSL/TLS 구성을 위한 설정파일 생성 후 SSL/TLS 설정값을 정의한다. (Nginx DOCS 참조)

▣ SSL/TLS 암호화 통신을 위해서는 사이트 인증서 및 개인 Key가 필요하다 (사이트 인증서, 개인 Key를 생성을 위해 openssl 사용)

Kubernetes ConfigMap & Secret (EX.2 : SSL / TLS)

```
$ openssl req -new -newkey rsa:4096 -days 365 -nodes -x509 -keyout cert.key -out cert.crt
```

Generating a 4096 bit RSA private key

writing new private key to 'cert.key'

Country Name (2 letter code) [XX]:KR

State or Province Name (full name) []:Seoul

Locality Name (eg, city) [Default City]:Gangnam

Organization Name (eg, company) [Default Company Ltd]:Cloud

Organizational Unit Name (eg, section) []:CloudTeam

Common Name (eg, your name or your server's hostname) []: Enter(생략)

Email Address []: Enter(생략)

```
$ ls cert*
```

```
cert.crt  cert.key
```

▣ openssl을 이용하여 SSL/TLS 암호화 통신에 사용 할 사이트 인증서 및 개인 Key를 생성한다.

▣ 정상적으로 사이트 인증서와 개인 Key 파일이 생성되었는지 반드시 확인

Kubernetes ConfigMap & Secret (EX.2 : SSL / TLS)

```
$ kubectl create configmap tls-config --from-file=configmap_secret/config/tls.conf -n delivery
```

```
$ kubectl describe cm tls-config -n delivery
```

Data

====

tls.conf:

```
ssl_protocols TLSv1 TLSv1.1 TLSv1.2;
```

```
server {
```

```
    listen 443 ssl;
```

```
    ssl_certificate /etc/cert/tls.crt;
```

```
    ssl_certificate_key /etc/cert/tls.key;
```

```
    location / {
```

```
        root /usr/share/nginx/html;
```

```
        index index.html;
```

```
    }
```

```
}
```

▣ Nginx SSL/TLS 설정 파일을 이용하여 ConfigMap 오브젝트를 생성 후 ConfigMap 오브젝트 데이터를 확인

Kubernetes ConfigMap & Secret (EX.2 : SSL / TLS)

```
$ kubectl create secret tls tls-secret --cert=cert.crt --key=cert.key -n delivery
```

```
$ kubectl describe secret tls-secret -n delivery
```

Type: **kubernetes.io/tls**

Data

====

tls.crt: 1960 bytes

tls.key: 3272 bytes

▣ Secret TLS 타입의 오브젝트를 생성 후 Secret 오브젝트의 데이터를 확인한다.

```
$ kubectl get cm,secret -n delivery
```

NAME	DATA	AGE
configmap/tls-config	1	85s

NAME	TYPE	DATA	AGE
secret/tls-secret	kubernetes.io/tls	2	14s

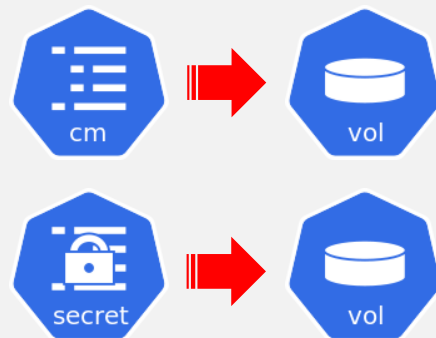
▣ Pod 오브젝트 생성 전 반드시 ConfigMap, Secret 오브젝트가 생성되었는지 확인 후 작업을 진행한다.

Kubernetes ConfigMap & Secret (EX.2 : SSL / TLS)

<작업대상: cm-secret-deploy-volume.yml >

※ [Deployment 오브젝트 전체 내용은 TXT 교안을 참조]

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-svc
  labels:
    app: my-nginx
  namespace: delivery
spec:
  type: LoadBalancer
  selector:
    app: my-nginx
  ports:
    - port: 443
      targetPort: 443
```



ConfigMap, Secret Volume 생성
Volume을 Container Directory에 Mount
Service 및 Container Port는 443번을 정의

```
spec:
  volumes:
    - name: tls-config-volume
      configMap:
        name: tls-config
    - name: tls-secret-volume
      secret:
        secretName: tls-secret
  containers:
    - name: my-nginx
      image: nginx:latest
      ports:
        - containerPort: 443
      volumeMounts:
        - name: tls-config-volume
          mountPath: /etc/nginx/conf.d/
        - name: tls-secret-volume
          mountPath: /etc/cert/
```

Kubernetes ConfigMap & Secret (EX.2 : SSL / TLS)

```
$ kubectl apply -f configmap_secret/cm-secret-deploy-volume.yml
```

```
service/nginx-svc created
```

```
deployment.apps/my-deployment created
```

```
$ kubectl get pod,svc,deploy -n delivery
```

NAME	READY	STATUS	RESTARTS	AGE
pod/my-deployment-588d6fb885-lbtxc	1/1	Running	0	20s
pod/my-deployment-588d6fb885-ldwsc	1/1	Running	0	20s

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
service/nginx-svc	LoadBalancer	10.100.171.59	amazonaws.com	443:30339/TCP	43s

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/my-deployment	2/2	2	2	4m51s

- Deployment, Service 오브젝트 생성 후 생성 된 오브젝트 목록을 확인한다.
- 로컬 PC에서 ELB의 Endpoint 주소로 (HTTPS : SSL/TLS) 연결 테스트를 진행한다.
- 현재 사이트 인증서는 정식기관으로부터 발급 된 인증서가 아니므로 안전하지 않은 사이트 메시지가 표시된다.
- 연결 테스트 작업시에는 반드시 웹브라우저 캐시삭제 작업을 수행 후 테스트를 진행해야 정확한 테스트가 가능하다.

Kubernetes ConfigMap & Secret (EX.2 : SSL / TLS)

```
$ kubectl exec my-deployment-588d6fb885-lbtxc -n delivery -- cat /etc/nginx/conf.d/tls.conf
```

```
ssl_protocols TLSv1 TLSv1.1 TLSv1.2;
server {
    listen 443 ssl;
    ssl_certificate /etc/cert/tls.crt;
    ssl_certificate_key /etc/cert/tls.key;

    location / {
        root /usr/share/nginx/html;
        index index.html index.htm;
    }
}
```

- 현재 Pod 내부 컨테이너의 ConfigMap 오브젝트를 참조하는 tls.conf 파일의 내용을 확인한다.
- ConfigMap 변경사항 테스트를 위해 해당 설정파일의 정보를 담고있는 ConfigMap 오브젝트를 수정한다.

Kubernetes ConfigMap & Secret (EX.2 : SSL / TLS)

```
$ kubectl edit cm tls-config -n delivery
```

```
apiVersion: v1
```

```
data:
```

```
  tls.conf: |-
```

```
    server {
```

```
      listen 443 ssl;
```

```
      ssl_certificate /etc/cert/tls.crt;
```

```
      ssl_certificate_key /etc/cert/tls.key;
```

```
      location / {
```

```
        root /usr/share/nginx/html;
```

```
        index index.html;
```

```
      }
```

```
    }
```

```
kind: ConfigMap
```

```
configmap/tls-config edited
```

▣ `ssl_protocols TLSv1 TLSv1.1 TLSv1.2;` 부분을 찾아 삭제 작업을 수행한다.

▣ 삭제 작업 완료 후 저장 후 종료 (잘못 삭제 할 경우 ConfigMap 오브젝트를 다시 생성 후 테스트)

Kubernetes ConfigMap & Secret (EX.2 : SSL / TLS)

```
$ kubectl exec my-deployment-588d6fb885-lbtxc -n delivery -- cat /etc/nginx/conf.d/tls.conf
```

```
ssl_protocols TLSv1 TLSv1.1 TLSv1.2;
```

```
server {  
    listen 443 ssl;  
    ssl_certificate /etc/cert/tls.crt;  
    ssl_certificate_key /etc/cert/tls.key;  
  
    location / {  
        root /usr/share/nginx/html;  
        index index.html index.htm;  
    }  
}
```

- ConfigMap 오브젝트 변경 시 컨테이너 설정 파일이 함께 변경 된 것을 확인한다. (딜레이 시간이 어느정도 필요하며 여유있게 테스트)
- ConfigMap 설정이 컨테이너에 자동 반영되어 설정파일이 변경되었다고해서 실제 변경 사항을 반영하여 컨테이너가 동작하는것은 아니다.
- 이미 기존의 설정파일의 내용을 참조하여 실행중인 프로세스는 변경 된 설정파일 내용을 프로세스에 자동반영하지 못하기 때문이다.
- 만약 수정 된 설정파일의 변경사항을 반영하여 컨테이너를 동작시키고 싶은 경우 컨테이너 재 시작이 반드시 필요하다. (`systemctl restart`)
- 재시작 EX.1: 사이드카 컨테이너 구성 후 변경사항 체크 스크립트를 작성하여 변경사항 발생 시 SIGHUP 시그널을 서비스 컨테이너로 전송하는 방법
- 재시작 EX.2: Application 자체적으로 설정파일 변경사항을 체크하여 변경사항 발생 시 변경사항을 자체적으로 리로드 하도록 구성하는 방법

Kubernetes ConfigMap & Secret (EX.2 : SSL / TLS)

```
$ kubectl edit cm tls-config -n delivery
```

```
apiVersion: v1
```

```
data:
```

```
  tls.conf: "KEY\n"
```

```
kind: ConfigMap
```

```
configmap/tls-config edited
```

```
$ kubectl exec my-deployment-588d6fb885-lbtxc -n delivery -- cat /etc/nginx/conf.d/tls.conf
```

```
Key
```

- ▣ tls.conf 설정파일의 내용변경 (SSL/TLS 설정 영역을 전부 삭제 후 "KEY\n" 값을 적용)
- ▣ 컨테이너의 변경 된 설정값이 적용되었는지 확인 후 로컬 PC에서 SSL/TLS 접속을 테스트한다.
- ▣ 실제 설정파일의 내용이 변경되었지만 이전 정보를 바탕으로 SSL/TLS가 계속 동작하는 것을 확인 할 수 있다.

```
$ kubectl delete -f configmap_secret/cm-secret-deploy-volume.yml
```

```
$ kubectl delete cm tls-config -n delivery
```

```
$ kubectl delete secret tls-secret -n delivery
```

- ▣ 다음 TEST를 위해 생성 한 Deployment 및 ConfigMap, Secret 오브젝트 삭제 작업을 진행한다.



Kubernetes Pod Health-Check

Kubernetes Pod Health-Check (Liveness & Readiness Probe)

◎ Kubernetes Pod Health-Check

- ▶ Kubernetes Cluster에 생성 되는 Pod 내부 컨테이너 Application이 정상적으로 동작중인지 확인하는 기능을 의미한다.
- ▶ Pod Health-Check 기능은 Application의 이상이 감지 될 경우 해당 컨테이너를 강제 종료 후 재시작 하도록 구성 할 수 있다.
- ▶ Pod Health-Check 기능은 Kubelet이 담당하며 "Liveness Probe"와 "Readiness Probe"를 사용하여 해당 기능을 수행한다.
- ▶ Pod Health-Check를 수행하기위한 Handler의 종류는 "EXEC", "HTTP GET", "TCP Socket"등을 사용한다.
- ▶ Pod Health-Check를 이용하여 Self-Healing을 구현하고 컨테이너 장애가 클라이언트에게 미치는 영향을 최소화 할 수 있다.

◎ Kubernetes Pod Health-Check Handler

- ▶ EXEC 핸들러: 컨테이너 내부에서 특정 커맨드를 수행 (커맨드 수행 결과에 대한 종료 코드값: "0" [성공], 그 외 [실패])
- ▶ HTTP GET: 지정 된 WEB URI 경로로 요청을 정기적으로 실행 (HTTP 상태코드 값: 200이상 400미만 [성공], 그 외 [실패])
- ▶ TCP Socket: 지정 한 TCP 포트번호로 연결이 가능 할 경우 성공 (지정 Port : Open[성공], Close[실패])

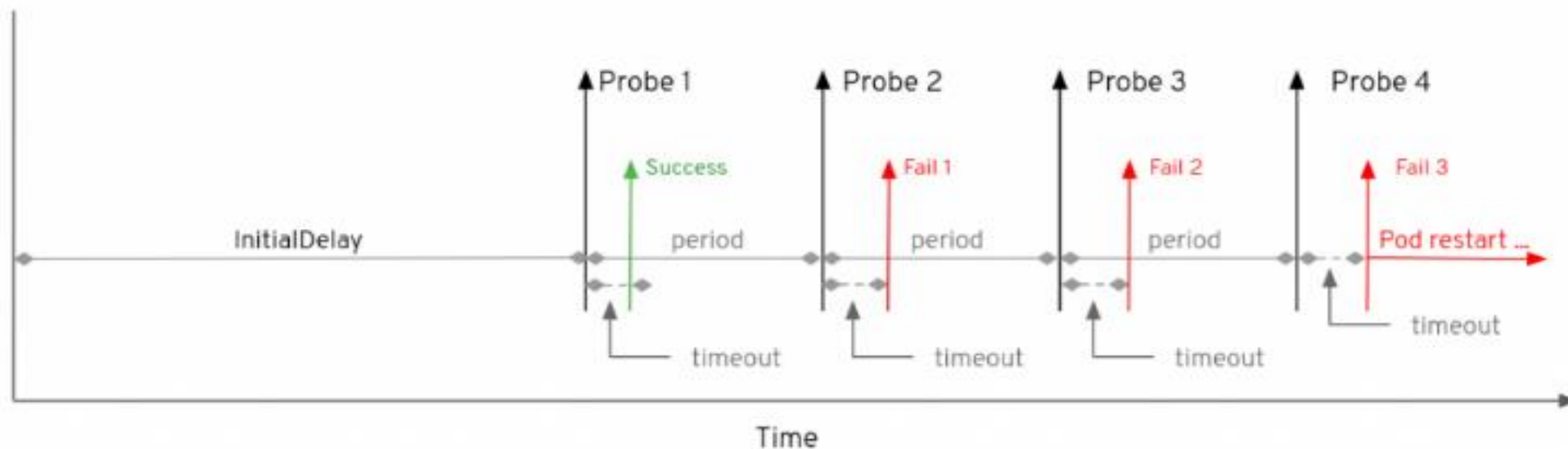
◎ Liveness Probe & Readiness Probe

- ▶ Liveness: Pod 내부 컨테이너 Application이 정상적으로 실행 중인 상태인지 검사한다. (Self-Healing 구현체)
- ▶ Readiness: Pod 내부 컨테이너 Application이 클라이언트 요청을 받을 준비가 되었는지 검사한다. (get 명령어: READY 정보)
- ▶ Readiness Probe는 신규 Pod 배포 후 내부 컨테이너 Application 준비상태를 확인하는 용도로 주로 사용된다.

Kubernetes Pod Health-Check (Liveness & Readiness Probe)

◎ Liveness Probe & Readiness Probe Attribute

- ▶ `initialDelaySeconds`: Pod 구성 후 지정 된 `initialDelaySeconds` 만큼 대기 후 Pod Health-Check를 진행한다.
- ▶ `periodSeconds`: Liveness Probe & Readiness Probe를 전송하는 주기를 설정 한다.
- ▶ `successThreshold`: 상태검사 실패 횟수 초기화를 위한 검사 성공 횟수 지정한다.
- ▶ `failureThreshold`: 연속적으로 몇 번 상태검사에 실패했을경우 컨테이너를 초기화 할 것인지 정의한다.
- ▶ `timeoutSeconds`: 상태 검사를 위한 응답 데이터를 기다리는 대기시간을 설정한다.



Kubernetes Pod Health-Check (EX.1: Liveness Probe)

< 작업대상: liveness-probe.yml >

spec:

containers:

※ [Pod 오브젝트 전체 내용은 TXT 교안을 참조]

- name: my-app

image: chlzzz/kube-image:list

ports:

- containerPort: 8000

livenessProbe:

httpGet:

상태검사 수행을 위한 핸들러 정의 ("HTTP GET")

path: /liveness

List App에서는 현재 "/liveness" URI를 지원하지 않는다. (상태검사 실패)

port: 8000

initialDelaySeconds: 10

Pod 배포 후 10초 뒤 Liveness Probe 상태검사를 시작

periodSeconds: 3

Liveness Probe 상태검사를 3초 주기로 수행

successThreshold: 1

상태검사 성공 시 실패 횟수 초기화

failureThreshold: 3

3번 연속 상태검사에 실패 할 경우 컨테이너 재시작을 수행

timeoutSeconds: 5

상태검사 응답대기시간을 5초로 지정하여 5초이내의 응답이 오지 않은경우 상태검사 실패

Kubernetes Pod Health-Check (EX.1: Liveness Probe)

```
$ kubectl apply -f health_check/liveness-probe.yml
```

```
$ kubectl get pod -n delivery -w
```

NAME	READY	STATUS	RESTARTS	AGE
liveness-probe	1/1	Running	0	15s
liveness-probe	1/1	Running	1	18s
liveness-probe	1/1	Running	2	36s
liveness-probe	1/1	Running	3	55s

▣ Watch(-w) 모드를 이용하여 Pod의 상태 변화를 실시간으로 확인한다. (RESTARTS : 0 -> 1 -> 2 -> 3)

▣ 일정 시간이 경과 후 Pod의 상태는 CrashLoopBackOff 상태로 변경 된다. (CrashLoopBackOff : 컨테이너를 사용할 수 없는 상태)

```
$ kubectl logs liveness-probe -n delivery
```

```
[25/Jul/2022 19:19:26] "GET /liveness HTTP/1.1" 404 2171
```

```
Not Found: /liveness
```

```
[25/Jul/2022 19:19:29] "GET /liveness HTTP/1.1" 404 2171
```

```
Not Found: /liveness
```

▣ 지정 된 주기에 따라 Liveness Probe를 실행하는 것을 확인 할 수 있다.

▣ 현재 "/liveness" 경로에 해당하는 Page는 존재하지 않으므로 "404 Error Code"가 반환 된다.

▣ HTTP 상태 코드값이 200이상 400미만이 리턴 될 경 성공 / 그 외의 값은 실패로 처리된다.

Kubernetes Pod Health-Check (EX.1: Liveness Probe)

```
$ kubectl describe pod liveness-probe -n delivery
```

Containers:

Restart Count: 6

Liveness: http-get http://:8000/liveness delay=10s timeout=5s period=3s #success=1 #failure=3

Events:

Type	Reason	Age	From	Message
----	-----	----	----	-----
Type	Reason	Age	From	Message
----	-----	----	----	-----
Normal	Scheduled	71s	default-scheduler	Successfully assigned delivery/liveness-probe to node
Normal	Pulled	17s (x4 over 70s)	kubelet	Container image "chlzzz/kube-image:list" already
Normal	Created	17s (x4 over 70s)	kubelet	Created container my-app
Normal	Started	17s (x4 over 70s)	kubelet	Started container my-app
Normal	Killing	17s (x3 over 53s)	kubelet	Container my-app failed liveness probe, will be restarted
Warning	Unhealthy	5s (x10 over 59s)	kubelet	Liveness probe failed: HTTP probe failed statuscode: 404

▣ Pod의 상세정보를 확인 (Restart Count, Liveness 조건, Event List)

Kubernetes Pod Health-Check (EX.1: Liveness Probe)

```
$ kubectl delete pod liveness-probe -n delivery
```

< 작업대상: liveness-probe.yml >

spec:

containers:

※ [Pod 오브젝트 전체 내용은 TXT 교안을 참조]

- name: my-app

image: chlzzz/kube-image:list

ports:

- containerPort: 8000

livenessProbe:

httpGet:

상태검사 수행을 위한 핸들러 정의 ("HTTP GET")

path: /list

List App에서는 현재 "/list" URI를 지원한다. (상태검사 성공)

port: 8000

initialDelaySeconds: 10

Pod 배포 후 10초 뒤 Liveness Probe 상태검사를 시작

periodSeconds: 3

Liveness Probe 상태검사를 3초 주기로 수행

successThreshold: 1

상태검사 성공 시 실패 횟수 초기화

failureThreshold: 3

3번 연속 상태검사에 실패 할 경우 컨테이너 재시작을 수행

timeoutSeconds: 5

상태검사 응답대기시간을 5초로 지정하여 5초이내의 응답이 오지 않은경우 상태검사 실패

Kubernetes Pod Health-Check (EX.1: Liveness Probe)

```
$ kubectl apply -f health_check/liveness-probe.yml
```

```
$ kubectl get pod -n delivery -w
```

NAME	READY	STATUS	RESTARTS	AGE
liveness-probe	1/1	Running	0	15s

■ Watch(-w) 모드를 이용하여 Pod의 상태 변화를 실시간으로 확인한다. (컨테이너 상태가 정상이므로 재시작하지 않는다)

```
$ kubectl logs liveness-probe -n delivery
```

```
[25/Jul/2022 19:38:41] "GET /list HTTP/1.1" 301 0
```

```
[25/Jul/2022 19:38:41] "GET /list/ HTTP/1.1" 200 1923
```

```
[25/Jul/2022 19:38:44] "GET /list HTTP/1.1" 301 0
```

```
[25/Jul/2022 19:38:44] "GET /list/ HTTP/1.1" 200 1923
```

```
$ kubectl describe pod liveness-probe -n delivery
```

Conditions:

Ready True

ContainersReady True

```
$ kubectl delete pod liveness-probe -n delivery
```

■ Pod의 상세정보를 확인 (Conditions, Event List) / 다음 TEST를 위해 Pod 오브젝트 삭제

Kubernetes Pod Health-Check (EX.2: Readiness Probe)

< 작업대상: readiness-probe.yml >

spec:

containers:

※ [Pod 오브젝트 전체 내용은 TXT 교안을 참조]

- name: my-app

image: chlzzz/kube-image:list

ports:

- containerPort: 8000

readinessProbe:

exec:

상태검사 수행을 위한 핸들러 정의 ("EXEC")

command:

컨테이너 내부에서 실행 될 명령어를 정의

- ls

컨테이너 내부에 /usr/src/app/ready 파일이 존재하는지 확인

- /usr/src/app/ready

initialDelaySeconds: 10

Pod 배포 후 10초 뒤 Liveness Probe 상태검사를 시작

periodSeconds: 3

Liveness Probe 상태검사를 3초 주기로 수행

successThreshold: 1

상태검사 성공 시 실패 횟수 초기화

failureThreshold: 3

3번 연속 상태검사에 실패 할 경우 컨테이너 재시작을 수행

timeoutSeconds: 5

상태검사 응답대기시간을 5초로 지정하여 5초이내의 응답이 오지 않은경우 상태검사 실패

Kubernetes Pod Health-Check (EX.2: Readiness Probe)

```
$ kubectl apply -f health_check/readiness-probe.yml
```

```
$ kubectl get pod -n delivery -w
```

NAME	READY	STATUS	RESTARTS	AGE
readiness-probe	0/1	Running	0	15s

■ Pod 오브젝트는 정상적으로 생성되었지만 시간이 흘러도 READY의 값이 계속해서 "0/1"인 것을 확인한다.

■ 현재 컨테이너의 상태는 Liveness Probe 상태검사는 통과하였지만 Application이 사용자의 요청을 받아들일 준비가 되지 않은 상태가 된다.

```
$ kubectl describe pod readiness-probe -n delivery
```

Containers:

Readiness: `exec [ls /usr/src/app/ready] delay=10s timeout=5s period=3s #success=1 #failure=3`

Conditions:

Ready `False`

Events:

Type	Reason	Age	From	Message
----	-----	----	----	-----

Warning	Unhealthy	8s	kubelet	Readiness probe failed: ls: cannot access 'ready': No such file or directory
---------	-----------	----	---------	--

■ Pod의 상세정보를 확인 (Readiness Probe 조건, Conditions, Event List)

Kubernetes Pod Health-Check (EX.2: Readiness Probe)

```
$ kubectl exec -it readiness-probe -n delivery -- touch /usr/src/app/ready
```

```
$ kubectl exec -it readiness-probe -n delivery -- ls /usr/src/app/ready
```

```
/usr/src/app/ready
```

▣ Readiness Probe 조건에 맞게 해당 파드내부의 컨테이너에 "ready" 파일을 생성 후 확인

```
$ kubectl get pod readiness-probe -n delivery
```

NAME	READY	STATUS	RESTARTS	AGE
readiness-probe	1/1	Running	0	5m32s

▣ Readiness Probe 조건을 만족하므로 현재 컨테이너의 상태가 사용자의 요청을 받아들일 수 있는 상태로 변경 된 것을 확인한다.

```
$ kubectl delete pod readiness-probe -n delivery
```

```
pod "readiness-probe" deleted
```

```
$ kubectl get all -n delivery
```

```
No resources found in delivery namespace.
```

▣ 다음 TEST를 위해 Pod 오브젝트 삭제



Kubernetes Resource Management

Kubernetes Resource Management

◎ Kubernetes Resource Management

- ▶ Kubernetes는 Cluster를 구성하는 여러대의 서버의 컴퓨팅 자원(CPU,RAM)을 하나로 묶어 Resource Pool로 사용 할 수 있다.
- ▶ Kubernetes에서는 Cluster 내부에 생성되는 컨테이너의 컴퓨팅 자원 사용량 제한을 통해 효과적인 컴퓨팅 자원 사용이 가능하다.
- ▶ Kubernetes에서는 자원 사용량 제한을 Pod 단위로 제어하며 Request, Limit을 이용하여 컴퓨팅 자원 사용량을 제한한다.
- ▶ Request는 컨테이너가 최소한으로 보장받아야하는 자원 사용량을 정의하며, Pod가 생성 될 Node를 선택하는 기준이 된다.
- ▶ Request에서 정의 된 최소한의 자원 사용량을 충족 시킬 수 없는 Node에서는 Pod가 생성되지 않는다.
- ▶ OverCommit은 컨테이너가 Request에 정의 된 자원 사용량을 넘어서는 자원을 사용하고 상태를 의미한다.
- ▶ Limit은 컨테이너가 최대로 사용 할 수 있는 자원 사용량을 정의하며, OverCommit의 실질적인 제한값으로 사용된다.

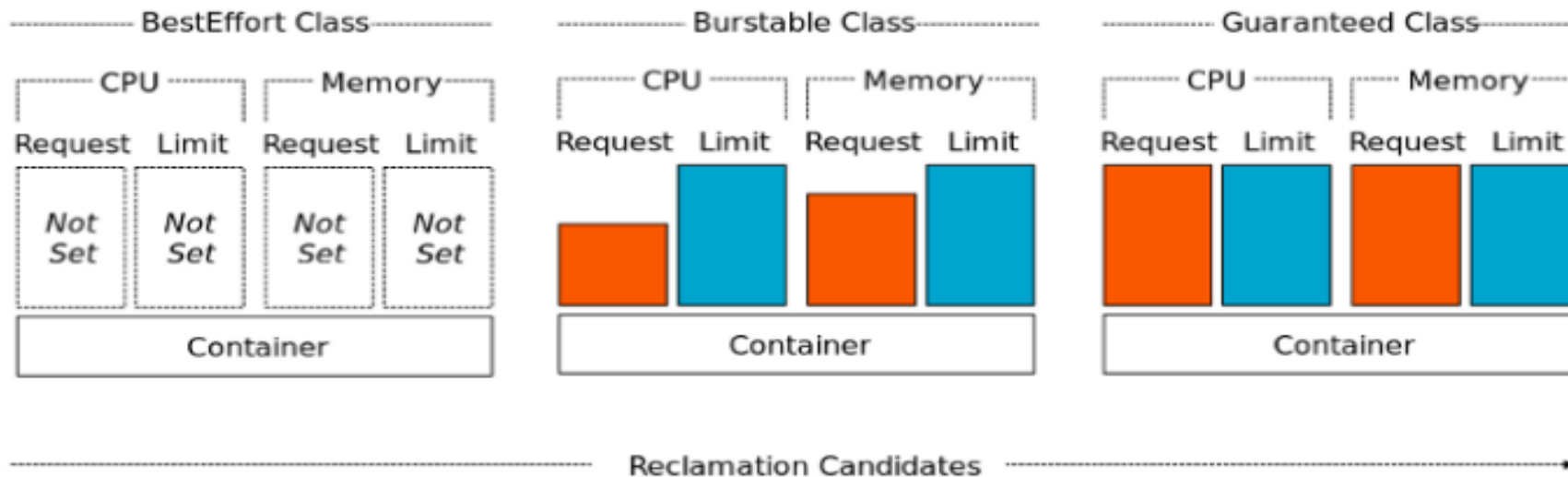
◎ Kubernetes Resource OverCommit EX

- ▶ Node-1에서 현재 사용가능 한 물리 메모리 용량 1000M
 - ▶ A 컨테이너 : Request (500M) / Limit (700M) / B 컨테이너 : Request (300M) / Limit (500M)
 - ▶ A 컨테이너, B 컨테이너를 시작 후 Node-1에 남은 메모리용량 : 200M
 - ▶ A 컨테이너에서 추가 메모리 할당이 필요한 경우 남은 메모리용량 내에서 최대 Limit까지 추가 메모리 할당 (OverCommit)
 - ▶ B 컨테이너에서도 추가 메모리 할당이 필요한 경우 메모리 점유 경합이 발생한다. (Out Of Memory 처리 진행)
- ※ OverCommit을 사용하지 않을경우 OOM 처리 문제가 발생하지 않아 OverCommit을 사용하는 환경보다 안정적인 운영이 가능하다.
- ※ CPU 사용률 경합의 경우 컨테이너 자체적으로 CPU 사용률을 낮추는 처리를 진행하여 우선순위 비교작업을 수행하지 않는다.

Kubernetes Resource Management

◎ Kubernetes OOM (Out Of Memory)

- ▶ Kubernetes OOM 처리는 Pod의 우선순위를 비교하여 우선 순위가 낮은 Pod를 종료 후 다른 Node로 이동시킨다.
- ▶ Pod의 우선순위를 비교하기 위해 Kubernetes는 3가지 QoS 클래스를 사용 한다. (Guaranteed / BestEffort / Burstable)
- ▶ Guaranteed : Limit, Request 값이 동일하게 설정 된 경우 적용 (OverCommit 사용 안 함)
- ▶ BestEffort : Limit, Request 값이 설정되지 않은 경우 적용 (Container Resource 정의가 없는 경우)
- ▶ Burstable : Limit, Request 값이 다르게 설정 된 경우 적용 (OverCommit 사용)
- ▶ 우선순위 : 1순위(Guaranteed), 2순위(Burstable or BestEffort), 3순위(Burstable or BestEffort)
- ▶ Burstable과 BestEffort의 경합의 경우 메모리 사용량이 더 많은 파드가 우선순위가 낮아진다.



Kubernetes Resource Management (EX.1: Limit & Request)

```
$ kubectl label node ip-192-168-10-147.ap-northeast-2.compute.internal resource=node-1
```

```
$ kubectl get node -L resource
```

NAME	STATUS	ROLES	AGE	VERSION	RESOURCE
ip-192-168-10-147.ap-northeast-2.compute.internal	Ready	<none>	2d4h	v1.24.16-eks-8ccc7ba	node-1
ip-192-168-20-156.ap-northeast-2.compute.internal	Ready	<none>	2d4h	v1.24.16-eks-8ccc7ba	

```
$ kubectl describe nodes ip-192-168-10-147.ap-northeast-2.compute.internal
```

Conditions:

Type	Status	LastHeartbeatTime	LastTransitionTime	Reason
MemoryPressure	False	Fri,	Wed,	KubeletHasSufficientMemory

Allocated resources:

Resource	Requests	Limits
cpu	125m (6%)	0 (0%)
memory	0 (0%)	0 (0%)

■ Node-1의 상세정보를 확인한다. (Conditions 영역, Allocated resources 영역)

■ MemoryPressure의 상태가 True로 변경 될 경우 OOM 처리를 진행한다. (Node의 가용 메모리 공간이 100Mi 이하일 때 True로 변경)

Kubernetes Resource Management (EX.1: Limit & Request)

< 작업대상: resource-limit-request.yml >

spec:

nodeSelector: ※ [Pod 오브젝트 전체 내용은 TXT 교안을 참조]

resource: node-1

containers:

- name: res-con

image: chlzzz/kube-image:debug

command: ['sh', '-c', 'echo The app is running! && sleep 3600']

resources: # Pod의 Resource 제한 정의 영역

limits: # Limit 속성 정의 (Memory, CPU 자원 최대 사용량)

memory: "500Mi" # Memory 최대 사용 : 500M

cpu: "500m" # CPU 최대 사용 : 0.5 코어

requests: # Request 속성 정의 (Memory, CPU 자원 최소 사용량) / 생략 할 경우 Limit 값이 자동 설정

memory: "500Mi" # Memory 최소 사용 : 500M

cpu: "500m" # CPU 최소 사용 : 0.5 코어

- Pod는 nodeSelector에의해 node-1에서 생성 되며 컨테이너는 최소 500Mi 메모리와 0.5코어의 사용을 보장받는다. (Request)
- 현재 Request와 Limit의 값이 동일하므로 Guaranteed QOS 클래스가 지정된다. (Request와 Limit이 동일 할 경우 Request는 생략가능)
- CPU 제한시 제어 단위는 밀리코어(M) 단위를 사용한다. (1코어 = 1000M)

Kubernetes Resource Management (EX.1: Limit & Request)

```
$ kubectl apply -f resources/resource-limit-request.yml
```

```
pod/res-pod created
```

```
$ kubectl get pod -n delivery -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS GATES
res-pod	1/1	Running	0	2m8s	192.168.10.156	node-1	<none>	<none>

```
$ kubectl describe pod res-pod -n delivery
```

Containers:

Limits:

cpu: 500m

memory: 500Mi

Requests:

cpu: 500m

memory: 500Mi

QoS Class: Guaranteed

▣ Pod의 상세정보를 확인한다. (Limits 영역, Requests 영역)

Kubernetes Resource Management (EX.1: Limit & Request)

```
$ kubectl describe nodes ip-192-168-10-147.ap-northeast-2.compute.internal
```

Non-terminated Pods:

Namespace	Name	CPU Requests	CPU Limits	Memory Requests	Memory Limits	AGE
-----	----	-----	-----	-----	-----	---
delivery	res-pod	500m (50%)	500m (50%)	500Mi (7%)	500Mi (7%)	4m26s

Allocated resources:

(Total limits may be over 100 percent, i.e., overcommitted.)

Resource	Requests	Limits
-----	-----	-----
cpu	625m (60%)	500m (60%)
memory	550Mi (7%)	500Mi (7%)
ephemeral-storage	0 (0%)	0 (0%)
hugepages-1Gi	0 (0%)	0 (0%)
hugepages-2Mi	0 (0%)	0 (0%)
attachable-volumes-aws-ebs	0	0

- Node-1의 상세정보를 확인한다. (Non-terminated Pods영역, Allocated resources 영역)
- 새롭게 생성 된 Pod의 자원 사용 현황이 Request에 정의 된 값 만큼 사용되고있는 것을 확인한다.

Kubernetes Resource Management (EX.1: Limit & Request)

< 작업대상: resource-overpod.yml >

spec:

nodeSelector: ※ [Pod 오브젝트 전체 내용은 TXT 교안을 참조]

resource: node-1

containers:

- name: res-con

image: chlzzz/kube-image:debug

command: ['sh', '-c', 'echo The app is running! && sleep 3600']

resources: # Pod의 Resource 제한 정의 영역

limits: # Limit 속성 정의 (Memory, CPU 자원 최대 사용량)

memory: "500Mi" # Memory 최대 사용 : 500M

cpu: "2000m" # CPU 최대 사용 : 2 코어

requests: # Request 속성 정의 (Memory, CPU 자원 최소 사용량) / 생략 할 경우 Limit 값이 자동 설정

memory: "500Mi" # Memory 최소 사용 : 500M

cpu: "2000m" # CPU 최소 사용 : 2 코어

■ node-1 에서 보장할 수 없는 Resource 요청을 갖는 Pod 생성을 정의한다. (TEST Pod)

■ Pod는 nodeSelector에의해 node-1에서 생성 되며 컨테이너는 최소 500Mi 메모리와 2코어의 사용을 보장받는다. (Request)

Kubernetes Resource Management (EX.1: Limit & Request)

```
$ kubectl apply -f resources/resource-overpod.yml
```

```
pod/over-pod created
```

```
$ kubectl get pod -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS GATES
over-pod	0/1	Pending	0	2m24s	<none>	<none>	<none>	<none>

```
$ kubectl describe pod over-pod
```

Events:

Type	Reason	Age	From	Message
----	-----	----	----	-----
Warning	FailedScheduling	90s	default-scheduler	0/2 nodes are available: 1 Insufficient cpu, 1 node(s) didn't match Pod's node affinity/selector. precondition: 0/2 nodes are available: 1 No precondition victims found for incoming pod, 1 Preemption is not helpful for scheduling.

■ 최소 500Mi 메모리와 0.5코어의 사용을 보장하는 새로운 Pod를 node-1을 선택하여 추가로 생성한다. (default NameSpace)

■ 현재 node-1의 경우 최소 500Mi 메모리와 0.5코어의 사용을 보장 할 수 없으므로 해당 파드는 Pending 상태가 지속된다.

Kubernetes Resource Management (EX.1: Limit & Request)

```
$ kubectl delete pod over-pod
```

```
pod "over-pod" deleted
```

```
$ kubectl delete pod res-pod -n delivery
```

```
pod "res-pod" deleted
```

```
$ kubectl describe nodes node-1
```

Allocated resources:

(Total limits may be over 100 percent, i.e., overcommitted.)

Resource	Requests	Limits
-----	-----	-----
cpu	100m (10%)	100m (10%)
memory	50Mi (5%)	50Mi (5%)
ephemeral-storage	0 (0%)	0 (0%)
hugepages-1Gi	0 (0%)	0 (0%)
hugepages-2Mi	0 (0%)	0 (0%)
attachable-volumes-aws-ebs	0	0

▣ 다음 테스트를 위하여 Pod 오브젝트를 삭제 후 Node-1의 자원 사용 현황을 확인한다. (Allocated Resources 영역)

Kubernetes Resource Management (EX.2: Quota & Range)

■ [Resource Quota & Limit Range Object]

- . Resource Quota: Cluster의 Namespace 별 Pod 오브젝트의 자원 사용량 제한설정 및 특정 오브젝트의 최대, 최소 범위지정 오브젝트
- . Limit Range: Cluster의 Namespace 별 Pod 오브젝트의 자원 할당 기본값 지정 오브젝트
- . 운영환경, 테스트환경, 개발환경 등의 Namespace를 분리하여 각 Namespace의 자원 사용량 제한을 통해 효과적인 자원 운용을 지원

< 작업대상: resource-quota.yml >

apiVersion: v1

kind: ResourceQuota # ResourceQuota 오브젝트

metadata:

name: resource-quota

namespace: delivery # ResourceQuota 오브젝트 적용 Namespace 정의

spec:

hard:

requests.cpu: "500m" # Request 속성에서 사용 할 Quota 값 정의

requests.memory: "500Mi"

limits.cpu: "700m" # Limit 속성에서 사용 할 Quota 값 정의

limits.memory: "700Mi"

■ [Kubernetes Resource 수 제한 설정]

count/pods: 10

count/deployments: 3

count/services.nodeports: 10

count/services.loadbalancers: 3

count/secrets: 5

count/configmaps: 5

Kubernetes Resource Management (EX.2: Quota & Range)

```
$ kubectl apply -f resources/resource-quota.yml
```

```
resourcequota/resource-quota created
```

```
$ kubectl get resourcequota -n delivery
```

NAME	AGE	REQUEST	LIMIT
resource-quota	19s	requests.cpu: 0/500m, requests.memory: 0/500Mi	limits.cpu: 0/700m, limits.memory: 0/700Mi

```
< 작업대상: resource-overpod.yml >
```

```
metadata:
```

```
  namespace: delivery
```

```
  resources: # Pod의 Resource 제한 정의 영역
```

```
    limits: # Limit 속성 정의 ( Memory, CPU 자원 최대 사용량 )
```

```
      memory: "1000Mi" # Memory 최대 사용 : 500M
```

```
      cpu: "500m" # CPU 최대 사용 : 2 코어
```

```
    requests: # Request 속성 정의 ( Memory, CPU 자원 최소 사용량 ) / 생략 할 경우 Limit 값이 자동 설정
```

```
      memory: "1000Mi" # Memory 최소 사용 : 500M
```

```
      cpu: "500m" # CPU 최소 사용 : 2 코어
```

▣ Over-Pod 정의 Manifest File 수정 (배포 Namespace 지정, 최소, 최대 메모리 용량부분을 수정)

Kubernetes Resource Management (EX.2: Quota & Range)

```
$ kubectl apply -f resources/resource-overpod.yml
```

```
Error from server (Forbidden): error when creating "resources/resource-overpod.yml": pods "over-pod" is forbidden:
exceeded quota: resource-quota, requested: limits.memory=1000Mi,requests.memory=1000Mi, used:
limits.memory=0,requests.memory=0, limited: limits.memory=700Mi,requests.memory=500Mi
```

- ▣ Resource Quota에서 정의한 최대 사용 메모리의 용량을 초과하는 Pod 오브젝트 생성 실패를 확인한다.
- ▣ NameSpace에 정의 된 Resource Quota Limit의 값을 초과하는 Pod 오브젝트는 사용이 불가능하다.

[Over-Pod의 Resources 영역을 수정하며 TEST]

1. resources 영역에서 limits, request 주식 후 TEST (Forbidden: resources -> Required)
 2. resources 영역에서 limits 영역 주식, request 영역 CPU 500m, Memory 500Mi 수정 후 TEST (Forbidden: limits 정의 X)
 3. resources 영역에서 limits 영역 Memory 500Mi, request 영역 Memory 500Mi 수정 후 TEST (Forbidden: CPU 정의 X)
 4. resources 영역에서 limits 영역 CPU 500m, Memory 500Mi, request영역 CPU 500m, Memory 500Mi 수정 후 TEST (생성 성공)
- ▣ Resource Quota가 정의 된 NameSpace에서 Pod 오브젝트를 생성 할 때에는 반드시 Resource 제한 설정을 필수로 함께 정의해야한다.
 - ▣ Resource Quota에서 정의되는 모든 항목(Request, Limit)을 반드시 함께 설정해야 한다. (BestEffort 클래스를 갖는 Pod는 생성 불가)
 - ▣ Resource Quota 오브젝트 생성 시 Requests 혹은 Limits만 사용가능하며, CPU & Memory 또한 선택적 사용이 가능하다.

```
$ kubectl delete pod over-pod -n delivery
```

```
pod "over-pod" deleted
```


Kubernetes Resource Management (EX.2: Quota & Range)

< 작업대상: resource-limitrage.yml >

```
apiVersion: v1
```

```
kind: LimitRange                                # LimitRange 오브젝트
```

```
metadata:
```

```
  name: resource-limitrage
```

```
  namespace: delivery                          # LimitRange 오브젝트 적용 NameSpace 정의
```

```
spec:
```

```
  limits:
```

```
    - default:                                # default 영역에서 Limit의 기본값을 정의한다.
```

```
      memory: 700Mi
```

```
      cpu: 700m
```

```
    defaultRequest:                            # defaultRequest 영역에서 Request의 기본값을 정의한다.
```

```
      memory: 500Mi
```

```
      cpu: 500m
```

```
    type: Container                            # LimitRange 적용 단위를 컨테이너로 정의한다.
```

```
$ kubectl apply -f resources/resource-limitrage.yml
```

```
limitrange/resource-limitrage created
```

Kubernetes Resource Management (EX.2: Quota & Range)

```
$ kubectl get limitrange -n delivery
```

NAME	CREATED AT
resource-limitrange	2023-09-15T06:52:41Z

```
$ kubectl run over-pod --image=nginx -n delivery
```

```
$ kubectl describe pod over-pod -n delivery
```

Limits:

cpu: 700m, memory: 700Mi

Requests:

cpu: 500m, memory: 500Mi

▣ Resource 제한 설정없이 Pod 오브젝트를 생성 후 생성 된 Pod의 Resource 제한 설정값을 확인한다.

▣ 별도의 지정 값을 이용하여 Pod 오브젝트를 생성하는 것도 가능하며, Resource 제한 설정값을 지정하지 않았을 경우에만 기본 설정값을 적용한다.

```
$ kubectl delete pod over-pod -n delivery
```

```
$ kubectl delete resourcequota resource-quota -n delivery
```

```
$ kubectl delete limitrange resource-limitrange -n delivery
```

▣ 다음 TEST를 위해 생성 한 Pod 및 Resource Quota, Limit Range 오브젝트 삭제 작업을 진행한다.

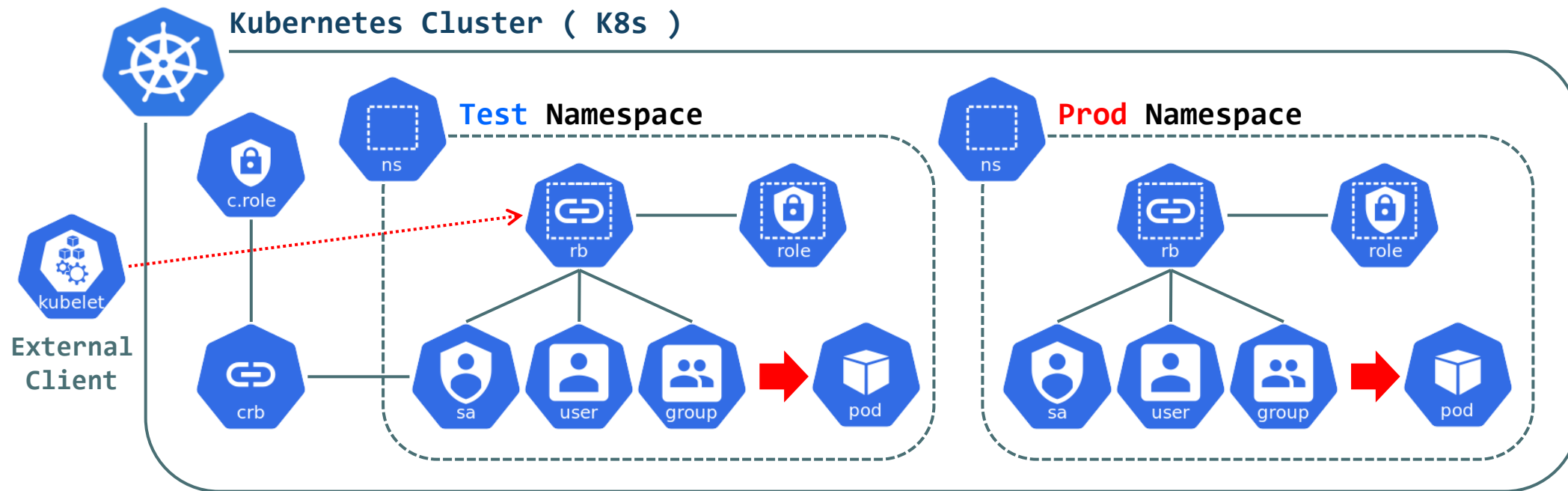


Kubernetes SA & RBAC

Kubernetes Service Account & Role-Based Access Control

◎ Service Account & Role-Based Access Control

- ▶ Service Account는 Kubernetes Cluster 내에서 체계적인 권한관리를 위한 오브젝트로 Namespace별로 구분된다.
- ▶ Kubernetes Cluster 내부 모든 Namespace에는 "default Service Account"를 자동으로 생성 및 포함한다.
- ▶ Service Account에 특정 권한을 부여하기 위해서는 Role 혹은 Cluster Role을 정의 후 Role Binding 작업을 수행해야한다.
- ▶ Role & Cluster Role은 Service Account에 **부여할 권한이 무엇인지 정의**하는 오브젝트 (**Role Binding : SA <-> Role**)
- ▶ Role: **Namespace 한정 Object (Pod, Deploy)** 권한 정의 / Cluster Role: **Cluster 전체 (Node, Volume)** 권한 정의
- ▶ Cluster Role에서는 Namespace 한정 Object 권한 정의도 가능하며, Cluster 내부의 모든 Object에 대한 권한 정의가 가능하다.



Kubernetes Role-Based Access Control (EX.1 : Role)

<작업대상: rbac-role.yml >

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: my-sa
  namespace: delivery
---
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-viewer-role
  namespace: delivery
rules:
- apiGroups: [""]           # Object API Group
  resources: ["pods"]       # Object Name
  verbs: ["get","watch","list"] # Allow Permission
```

```
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: pod-viewer-role-rb
  namespace: delivery
subjects:
- kind: ServiceAccount
  name: my-sa
  namespace: delivery
roleRef:
  kind: Role
  apiGroup: rbac.authorization.k8s.io # Role Object API Group
  name: pod-viewer-role
```

Kubernetes Role-Based Access Control (EX.1 : Role)

```
$ kubectl apply -f rbac/rbac-role.yml
```

```
$ kubectl get sa,role,rolebindings -n delivery
```

NAME	SECRETS	AGE
serviceaccount/my-sa	0	18s

NAME	CREATED AT
role.rbac.authorization.k8s.io/pod-viewer-role	2023-09-21T08:44:57Z

NAME	ROLE	AGE
rolebinding.rbac.authorization.k8s.io/pod-viewer-role-rb	Role/pod-viewer-role	18s

```
$ kubectl describe rolebindings pod-viewer-role-rb -n delivery
```

Name: pod-viewer-role-rb

Role:

Kind: Role

Name: pod-viewer-role

Subjects:

Kind	Name	Namespace
-----	-----	-----
ServiceAccount	my-sa	delivery

Kubernetes Role-Based Access Control (EX.1 : Role)

```
$ kubectl apply -f resources/resource-limit-request.yml
```

pod/res-pod created

▣ Resource Management에서 구성한 Manifest File을 이용하여 TEST Pod를 생성

```
$ kubectl auth can-i get pod -n delivery --as system:serviceaccount:delivery:my-sa
```

```
$ kubectl get pod -n delivery --as system:serviceaccount:delivery:my-sa
```

NAME	READY	STATUS	RESTARTS	AGE
res-pod	1/1	Running	0	6m45s

```
$ kubectl delete pod res-pod -n delivery --as system:serviceaccount:delivery:my-sa
```

Error from server (**Forbidden**): pods "res-pod" is forbidden: User "system:serviceaccount:delivery:my-sa" cannot delete resource "pods" in API group "" in the namespace "delivery"

```
$ kubectl delete pod res-pod -n delivery
```

```
$ kubectl delete -f rbac/rbac-role.yml
```

▣ "--as system:serviceaccount:delivery:mysa" 지정 된 Service Account를 이용하여 Kubectl 명령을 수행

▣ Role에서 정의 된 Pod 오브젝트에 대한 작업만 가능한 것을 확인 / 허용되지 않은 "delete" 작업은 Forbidden Error가 발생하는 것을 확인

▣ 다음 TEST를 위해 생성한 오브젝트를 삭제

Kubernetes Role-Based Access Control (EX.2 : Cluster Role)

<작업대상: rbac-cluster-role.yml >

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: my-sa
  namespace: delivery
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: node-viewer-role
rules:
- apiGroups: [""]           # Object API Group
  resources: ["nodes"]      # Object Name
  verbs: ["get","watch","list"] # Allow Permission
```

```
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: node-viewer-role-rb
subjects:
- kind: ServiceAccount
  name: my-sa
  namespace: delivery
roleRef:
  kind: ClusterRole
  apiGroup: rbac.authorization.k8s.io # Role Object API Group
  name: node-viewer-role
```


Kubernetes Role-Based Access Control (EX.2 : Cluster Role)

```
$ kubectl apply -f rbac/rbac-cluster-role.yml
```

```
$ kubectl get clusterrole node-viewer-role
```

NAME	CREATED AT
node-viewer-role	2023-09-21T09:39:05Z

```
$ kubectl get clusterrolebinding node-viewer-role-rb
```

NAME	ROLE	AGE
node-viewer-role-rb	ClusterRole/node-viewer-role	99s

```
$ kubectl describe clusterrole node-viewer-role
```

Name: node-viewer-role

Labels: <none>

Annotations: <none>

PolicyRule:

Resources	Non-Resource URLs	Resource Names	Verbs
-----	-----	-----	-----
nodes	[]	[]	[get watch list]

Kubernetes Role-Based Access Control (EX.2 : Cluster Role)

```
$ kubectl auth can-i get nodes --as system:serviceaccount:delivery:my-sa
```

```
$ kubectl get nodes --as system:serviceaccount:delivery:my-sa
```

NAME	STATUS	ROLES	AGE	VERSION
ip-192-168-10-147.ap-northeast-2.compute.internal	Ready	<none>	8d	v1.24.16-eks-8ccc7ba
ip-192-168-20-156.ap-northeast-2.compute.internal	Ready	<none>	8d	v1.24.16-eks-8ccc7ba

```
$ kubectl cordon ip-192-168-20-156.ap-northeast-2.compute.internal \
--as system:serviceaccount:delivery:my-sa
```

```
error: unable to cordon node "ip-192-168-20-156.ap-northeast-2.compute.internal":
nodes "ip-192-168-20-156.ap-northeast-2.compute.internal" is forbidden: User
"system:serviceaccount:delivery:my-sa" cannot patch resource "nodes" in API group "" at the cluster scope
```

```
$ kubectl delete -f rbac/rbac-cluster-role.yml
```

- "--as system:serviceaccount:delivery:mysa" 지정 된 Service Account를 이용하여 Kubectl 명령을 수행
- ClusterRole에서 정의 된 Node에 대한 작업만 가능한 것을 확인 / 허용되지 않은 "cordon" 작업은 Forbidden Error가 발생하는 것을 확인
- 다음 TEST를 위해 생성한 오브젝트를 삭제

Kubernetes Role-Based Access Control (EX.3 : User & Group)

■ Kubernetes User & Group

- >. User : Service Account를 지칭하는 고유한 이름 ["**system:serviceaccount**:<Namespace>:<ServiceAccountName>"]
- >. Group : Kubernetes User를 모아놓은 집합 ["**system:serviceaccounts**(Group)"]
- >. **serviceaccounts** Group은 Kubernetes Cluster 내부의 ServiceAccount를 모아놓은 Group이다.
- >. ServiceAccount Group 처럼 Kubernetes에 의해 미리 정의되어 사용되는 Group의 접두어에는 "**system:**"을 사용한다.
- >. Kubernetes User & Group은 RoleBinding Object의 **kind** 속성의 값으로 사용이 가능하다.

[User & Group RoleBinding Subjects Example]

```
subjects:  
- kind: User  
  name: devops
```

```
subjects:  
- kind: User  
  name: system:serviceaccount:delivery:my-sa
```

```
subjects:  
- kind: Group  
  name: system:serviceaccounts:default
```

```
subjects:  
- kind: Group  
  name: dev-group
```

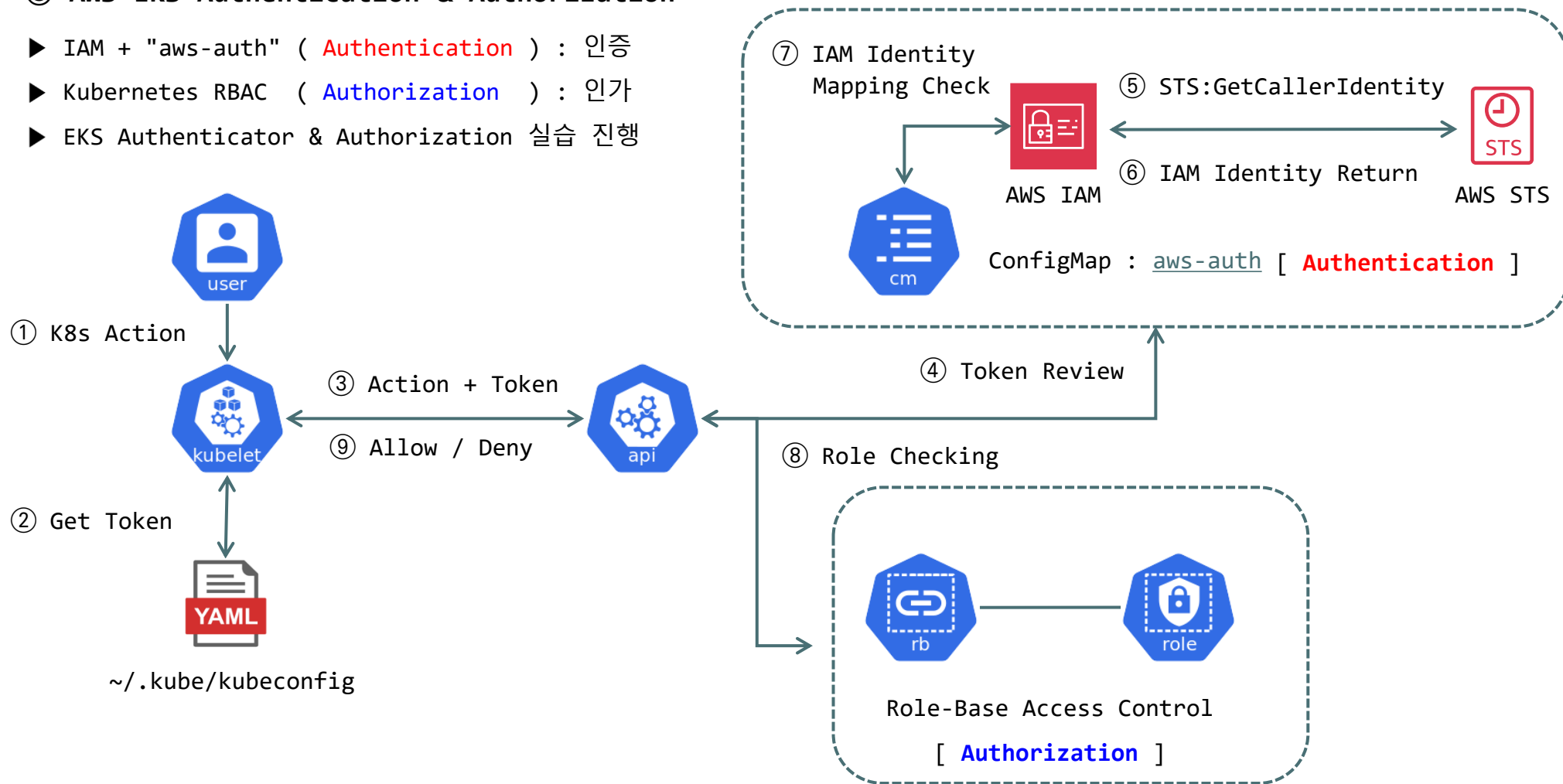
```
subjects:  
- kind: Group  
  name: system:serviceaccounts
```

```
subjects:  
- kind: Group  
  name: system:authenticated
```

Kubernetes RBAC (EKS Authentication & Authorization)

◎ AWS EKS Authentication & Authorization

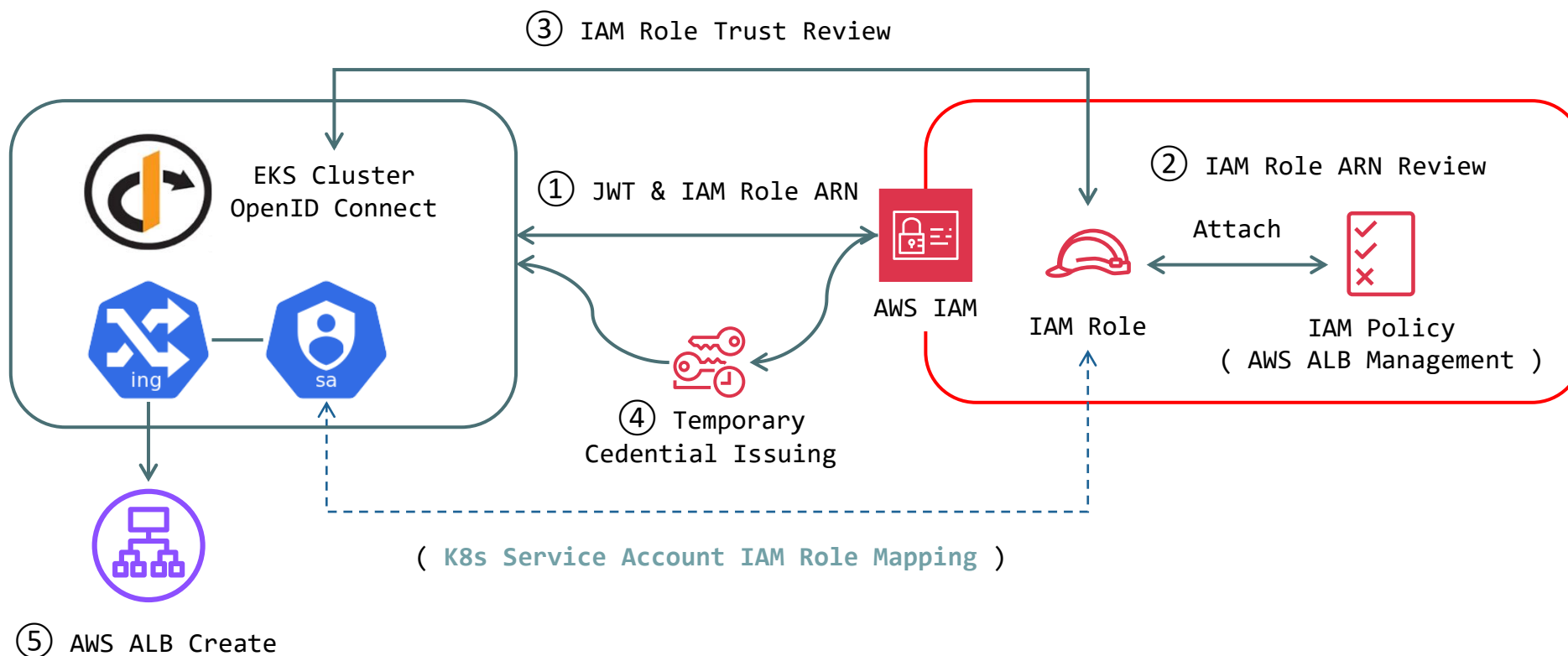
- ▶ IAM + "aws-auth" (**Authentication**) : 인증
- ▶ Kubernetes RBAC (**Authorization**) : 인가
- ▶ EKS Authenticator & Authorization 실습 진행



Kubernetes RBAC (EKS Iam Role for Service Account)

◎ AWS EKS IRSA (Iam Role for Service Account)

- ▶ K8s 내부 Service Account와 AWS IAM Role을 Mapping하여 K8s 내부에서 AWS Resource를 관리 할 수 있는 권한을 부여한다.
- ▶ IRSA를 구현하기 위해서는 K8s를 식별 OIDC(OpenID Connect)가 필요하며, EKS Cluster 생성 시 함께 생성 (콘솔에서 확인)
- ▶ EKS와 다른 AWS Resource를 통합 운영하기 위해서는 IRSA 구성이 필요하며, 상황에 따라 EKS 추가기능까지 구성해야 한다.





Kubernetes Ingress Network

Kubernetes Ingress Network Management

◎ Kubernetes Ingress Network Management

- ▶ 외부에서 내부로 유입되는 트래픽을 처리하는 네트워크를 의미하며 Ingress Network를 구현을 위해 Ingress 오브젝트를 사용한다.
- ▶ Ingress 오브젝트란 외부에서 k8s 내부로 들어오는 요청에 대한 처리를 어떻게 수행 할 것인지를 결정하는 오브젝트가 된다.
- ▶ Ingress 오브젝트는 요청에 대한 처리작업을 Layer 7 기반 트래픽 처리 작업을 수행한다.
- ▶ Ingress 오브젝트가 수행하는 외부요청 트래픽 처리 : 외부 요청 라우팅 처리, 가상 호스트 기반 요청 처리, SSL/TLS 요청 처리
 - >. 외부 요청 라우팅 : `/list`, `/order` 등의 특정 경로에 대한 요청을 어떠한 서비스로 연결 할 것인지를 결정 한다.
 - >. 가상 호스트 기반 요청 : `"www"`, `list` 등 서로 다른 호스트 네임에 따른 요청을 어떠한 서비스로 연결 할 것인지를 결정한다.
 - >. SSL/TLS 요청 : 보안 연결을 위한 인증서 적용 (하나의 Ingress SSL/TLS 구성 vs 다수의 서비스별 SSL/TLS 구성)

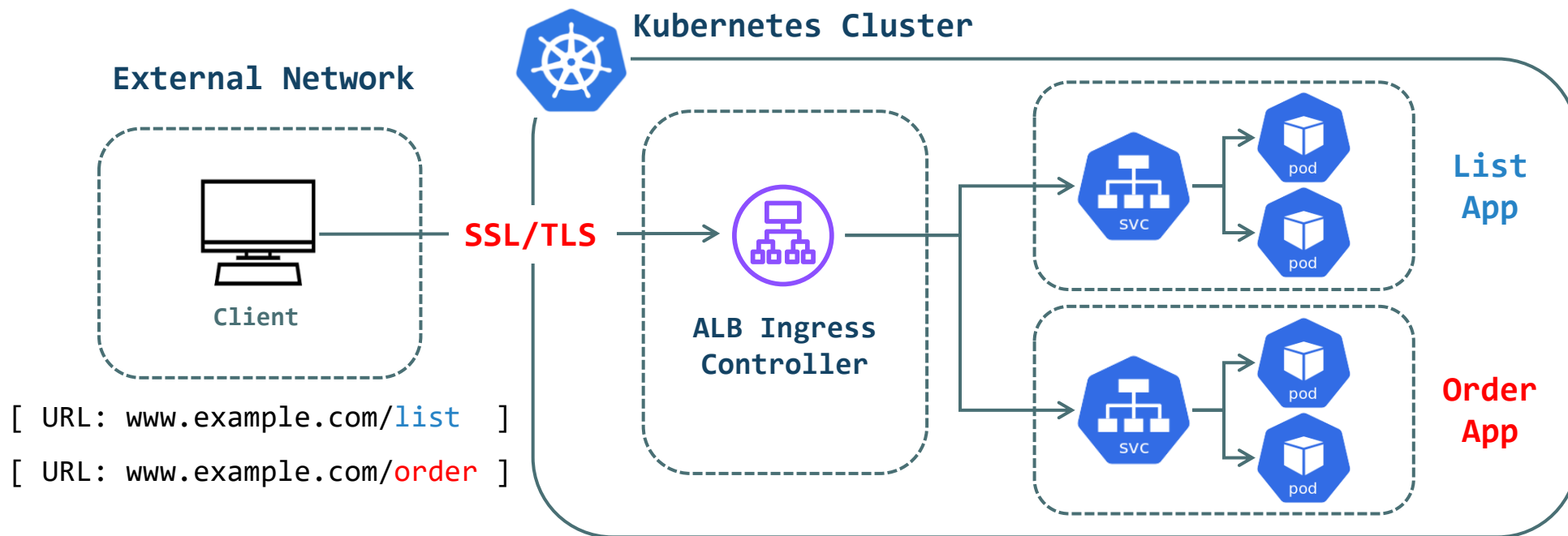
◎ Kubernetes Ingress Controller

- ▶ Ingress Controller는 외부요청을 받아 Ingress 오브젝트에서 정의 된 규칙에 맞게 요청처리를 수행하는 서버를 의미한다.
- ▶ Ingress Network를 구현하기 위해서는 Ingress Controller 반드시 필요하며 다양한 Ingress Controller가 존재한다.
- ▶ Kong API GW, Nginx Ingress Controller, GKE Ingress Controller, AWS Load Balancer 등 (종류에 따라 기능의 차이가 존재)
- ▶ TEST EKS 환경에서는 Helm을 이용하여 AWS Load Balancer Controller를 구성 후 Ingress Network Test를 진행한다.

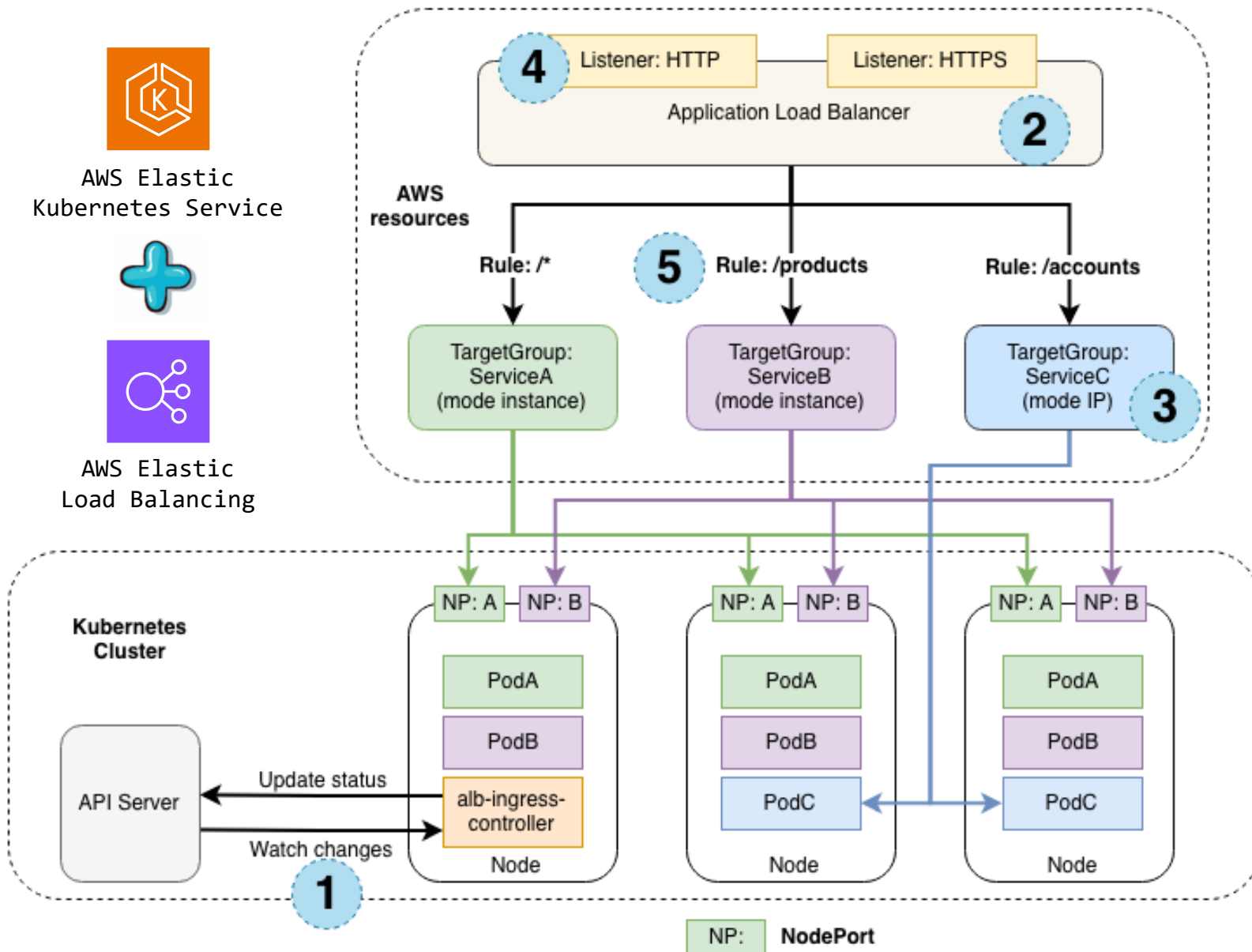
Kubernetes Ingress Network Management

◎ Kubernetes Ingress Network를 사용하는 이유

- ▶ 서로 다른 서비스를 구성하는 Deployment 오브젝트가 다수 배포되어있을 경우 해당 Deployment 별 서비스 오브젝트를 연결해야 한다.
- ▶ 클라이언트는 서비스 오브젝트별 Port 번호 혹은 External IP를 전부 기억하고 있어야 외부에서 접근이 가능하다.
- ▶ 보안 연결 작업 시 각 서비스 오브젝트마다 SSL/TLS 구성을 전부 따로 해주어야하며, 인증서 관리작업에 어려움이 많아지게 된다.
- ▶ Ingress 오브젝트를 이용하여 클라이언트에게 단일 엔드포인트를 제공하고 해당 단일 엔드포인트로만 접근하도록 구성한다.
- ▶ 단일 엔드포인트로 들어오는 외부요청에 처리규칙을 정의하여 적절한 서비스와 연결해주는 Service Discovery를 구현 할 수 있다.
- ▶ 단일 엔드포인트가 구성 될 경우 해당 엔드포인트에 대한 SSL/TLS 보안연결 작업만 수행하면된다.



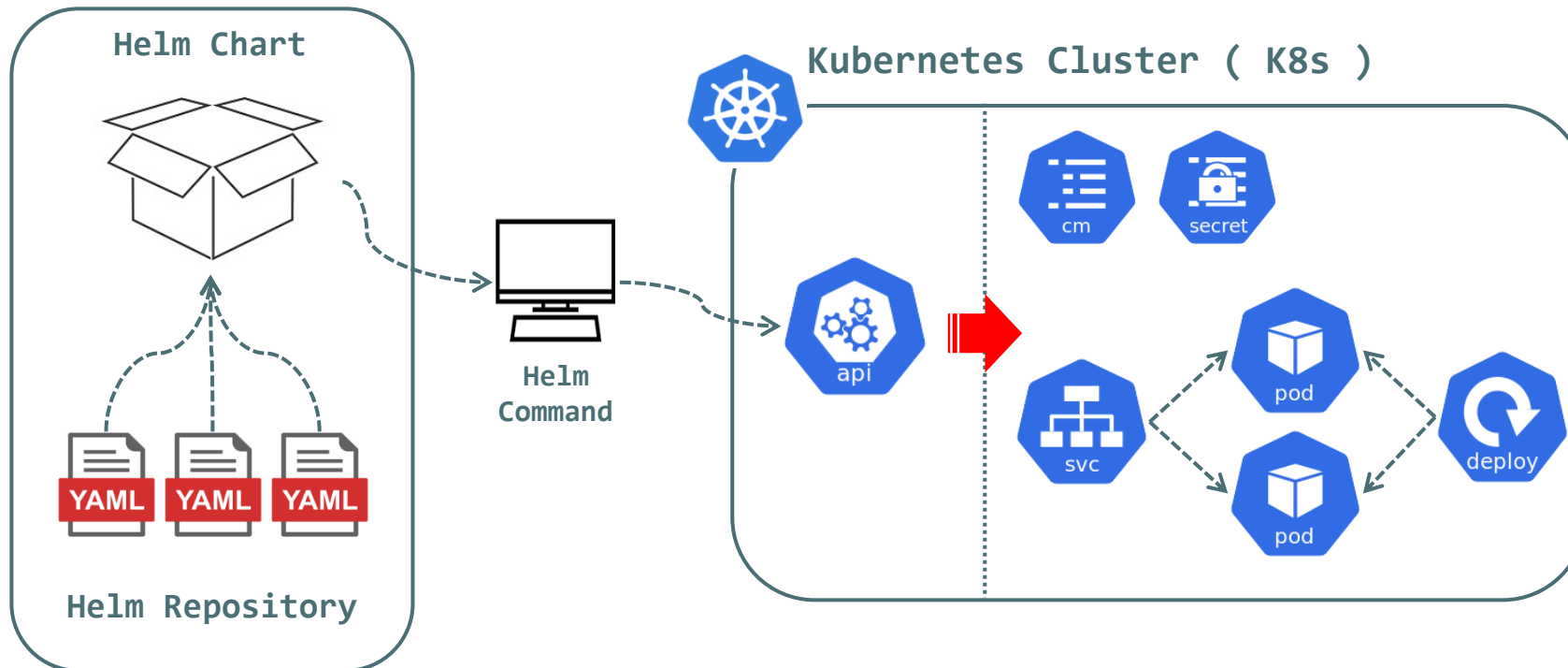
Kubernetes Ingress Network Management



Kubernetes Ingress Network Management

© Kubernetes Package Manager HELM

- ▶ Helm : Kubernetes Package Manager (Linux OS: Package Manager와 동일한 역할)
- ▶ Helm은 Kubernetes Cluster에서 배포되는 Application을 **Chart**(패키지) 형태로 제공한다.
- ▶ Chart 내부에는 Application이 가동되기 위한 모든 Resource를 포함하고 있다. (Deploy / ConfigMAP / Secret / Service)
- ▶ Helm은 Revision 관리를 진행하며 동일한 Chart를 여러번 설치가 가능하며, 각 설치 작업을 Revision 정보로 관리한다.



Kubernetes Ingress Network Management

■ [Helm Install]

```
$ curl -fsSL -o ~/get_helm.sh https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3
```

```
$ chmod 700 ~/get_helm.sh
```

```
$ ~/get_helm.sh
```

Downloading <https://get.helm.sh/helm-v3.9.1-linux-amd64.tar.gz>

Verifying checksum... Done.

Preparing to install helm into **/usr/local/bin**

helm installed into **/usr/local/bin/helm**

■ Helm을 설치를 위해 Curl 명령어를 이용해 Helm 공식 Git-Hub 저장소에 저장되어있는 Helm Install Script를 내려받는다.

■ 내려받은 Helm Install Script에 실행 권한을 부여하고 "get_heml.sh" Shell Script 파일을 실행하여 설치 작업을 완료한다.

■ Helm Install 완료 시 Helm Command가 BastionHost System에 구성되며 "heml" 명령어를 사용 할 수 있다.

■ Helm Install 시 작업 Directory는 어떠한 곳이든 상관없다. ("~/kube" Directory에서 Helm Install 작업을 수행)

```
$ chmod go-r ~/.kube/config
```

```
$ helm version
```

```
version.BuildInfo{Version:"v3.12.3", GitCommit:"3a31588ad33fae6eaa5e", GitTreeState:"clean", GoVersion:"go1.20.7"}
```

■ Helm Install 완료 후 helm 명령어를 이용하여 설치 된 Helm의 버전정보를 확인한다.

Kubernetes Ingress Network Management

■ [AWS Load Balancer Controller 구성 (Helm Install)]

```
$ helm repo add eks https://aws.github.io/eks-charts
```

```
$ helm repo update eks
```

```
$ helm install aws-load-balancer-controller eks/aws-load-balancer-controller \
-n kube-system \
```

```
--set clusterName=my-eks \
```

```
--set serviceAccount.create=false \
```

```
--set serviceAccount.name=aws-load-balancer-controller
```

```
$ kubectl get deployment -n kube-system aws-load-balancer-controller
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
aws-load-balancer-controller	2/2	2	2	24s

■ AWS Load Balancer Controller Install을 위한 EKS Repo 등록 및 업데이트

■ "-n kube-system" : AWS Load Balancer Controller Pod 생성 Namespace

■ "--set clusterName=my-eks" : EKS Cluster Name

■ "--set serviceAccount.create=false" : Service Account는 위에서 생성했으므로 False 정의

■ "--set serviceAccount.name=aws-load-balancer-controller" : 위에서 생성한 EKS Service Account 이름 입력

Kubernetes Ingress Network Management (EX.1 : Basic)

<작업대상: ingress-singlehost.yml >

```
apiVersion: networking.k8s.io/v1
kind: Ingress          # Ingress API Version 정의
metadata:              # Resource 종류 : Ingress 오브젝트
  name: my-ingress
  namespace: delivery
  annotations:
    kubernetes.io/ingress.class: alb
    alb.ingress.kubernetes.io/load-balancer-name: my-eks-alb
    alb.ingress.kubernetes.io/scheme: internet-facing
    alb.ingress.kubernetes.io/target-type: instance
spec:
  defaultBackend:      # Default 값으로 연결 할 서비스 오브젝트 정의
    service:
      name: list-svc
      port:
        number: 80
```

```
rules: # Ingress Routing Rule 정의
- http:
  paths:
    - pathType: Prefix
      path: /list
      backend:
        service:
          name: list-svc
          port:
            number: 80
    - pathType: Prefix
      path: /order
      backend:
        service:
          name: order-svc
          port:
            number: 80
```

Client URI "/list" 요청
list-svc 서비스와 연결

Client URI "/order" 요청
order-svc 서비스와 연결

Kubernetes Ingress Network Management (EX.1 : Basic)

```
$ kubectl apply -f ingress/ingress-singlehost.yml
```

```
ingress.networking.k8s.io/my-ingress created
```

```
$ kubectl get ingress -n delivery
```

NAME	CLASS	HOSTS	ADDRESS	PORTS	AGE
my-ingress	<none>	*	k8s-delivery-myingres	80	43s

```
$ cp ~/kube/service/svc-nodeport-list.yml ~/kube/ingress
```

```
$ cp ~/kube/service/svc-nodeport-order.yml ~/kube/ingress
```

```
$ kubectl apply -f ingress/svc-nodeport-list.yml
```

```
service/list-svc created
```

```
deployment.apps/list-deploy created
```

```
$ kubectl apply -f ingress/svc-nodeport-order.yml
```

```
service/order-svc created
```

```
deployment.apps/order-deploy created
```

■ Ingress Controller와 연결 할 Service 오브젝트 및 Deployment 오브젝트를 생성한다.

■ Service 오브젝트와 Deployment 오브젝트는 Service 오브젝트 TEST에서 사용 한 Manifest File을 그대로 사용한다.

Kubernetes Ingress Network Management (EX.1 : Basic)

```
$ kubectl get pod,svc -o wide -n delivery
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED	READINESS
pod/list-deploy-549c9bbb7-85s5x	1/1	Running	0	14s	192.168.10.14	node-1	<none>	<none>
pod/list-deploy-549c9bbb7-jh4dc	1/1	Running	0	14s	192.168.20.117	node-2	<none>	<none>
pod/order-deploy-7d6794d495-8d9md	1/1	Running	0	10s	192.168.10.187	node-2	<none>	<none>
pod/order-deploy-7d6794d495-zq8nb	1/1	Running	0	10s	192.168.20.130	node-1	<none>	<none>

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE	SELECTOR
service/list-svc	ClusterIP	10.100.180.147	<none>	80/TCP	14s	app=list-app
service/order-svc	ClusterIP	10.100.119.139	<none>	80/TCP	10s	app=order-app

```
$ kubectl get endpoints -n delivery
```

NAME	ENDPOINTS	AGE
list-svc	192.168.10.14:8000,192.168.20.117:8000	28s
order-svc	192.168.10.187:8000,192.168.20.130:8000	24s

- ▣ Ingress 오브젝트에서 list-svc, order-svc 연결 규칙을 미리 정의 한 상태이며 각 Service 오브젝트에 연결되는 Endpoints 정보를 확인
- ▣ 로컬 PC에서 접속 테스트 [http://alb-dns-name, http://alb-dns-name/list, / http://alb-dns-name/order]

```
$ kubectl delete -f ./ingress-singlehost.yml
```

```
ingress.networking.k8s.io "my-ingress" deleted
```

Kubernetes Ingress Network Management (EX.1 : Basic)

<작업대상: ingress-virtualhost.yml >

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  # Ingress API Version 정의
  name: my-ingress
  # Resource 종류 : Ingress 오브젝트
  namespace: delivery
  annotations:
    kubernetes.io/ingress.class: alb
    alb.ingress.kubernetes.io/load-balancer-name: my-eks-alb
    alb.ingress.kubernetes.io/scheme: internet-facing
    alb.ingress.kubernetes.io/target-type: instance
    external-dns.alpha.kubernetes.io/hostname: mydevsecops.link
spec:
  defaultBackend:
    service:
      # Default 값으로 연결 할 서비스 오브젝트 정의
      name: list-svc
      port:
        number: 80
```

```
rules:
  # Ingress Routing Rule 정의 (가상 호스트 기반)
  - host: list.mydevsecops.link
    http:
      paths:
        # "list.mydevsecops.link"
        # list-svc 서비스와 연결
        - pathType: Prefix
          path: /
          backend:
            service:
              name: list-svc
              port:
                number: 80
  - host: order.mydevsecops.link
    http:
      paths:
        # "order.mydevsecops.link"
        # order-svc 서비스와 연결
        - pathType: Prefix
          path: /
          backend:
            service:
              name: order-svc
              port:
                number: 80
```


Kubernetes Ingress Network Management (EX.1 : Basic)

```
$ kubectl apply -f ingress/ingress-virtualhost.yml
```

```
$ kubectl get ingress -n delivery
```

NAME	CLASS	HOSTS	ADDRESS	PORTS	AGE
my-ingress	<none>	list.mydevsecops.link , order.mydevsecops.link	elb.amazonaws.com	80	14s

■ [로컬 PC Web Browser (Chrome) 접속 테스트]

- . <http://mydevsecops.link> [list Main Page]
- . <http://list.mydevsecops.link> [list Main Page]
- . <http://list.mydevsecops.link/list> [list Menu Page]
- . <http://order.mydevsecops.link> [order Main Page]
- . <http://order.mydevsecops.link/order> [order Menu Page]

■ 접속 정보가 출력되는 Page에서 새로고침 작업을 통해 접속되는 Pod 오브젝트가 달라지는 것을 확인한다. (Session 유지 TEST)

■ Ingress 오브젝트를 이용 한 Session 유지 작업을 정의하기 위해서는 Ingress Controller Session Affinity 설정을 진행해야한다.

■ Ingress Controller는 Service 오브젝트가 연결하고있는 Pod 오브젝트의 Endpoints 정보만 참조하여 직접 Pod 오브젝트에 접근한다.

```
$ kubectl delete -f ingress/svc-nodeport-list.yml
```

```
$ kubectl delete -f ingress/svc-nodeport-order.yml
```

■ 다음 테스트를 위해 테스트에 사용한 오브젝트를 삭제한다.

Kubernetes Ingress Network Management (EX.1 : Basic)

■ [AWS Load Balancer Controller Sticky Session TEST]

```
$ kubectl edit ingress my-ingress -n delivery
```

annotations:

```
kubernetes.io/ingress.class: alb
```

```
alb.ingress.kubernetes.io/load-balancer-name: my-eks-alb
```

```
alb.ingress.kubernetes.io/scheme: internet-facing
```

```
alb.ingress.kubernetes.io/target-type: ip
```

```
alb.ingress.kubernetes.io/target-group-attributes: stickiness.enabled=true,stickiness.lb_cookie.duration_seconds=300
```

```
external-dns.alpha.kubernetes.io/hostname: mydevsecops.link
```

```
$ kubectl apply -f ingress/svc-clusterip-list.yml
```

```
$ kubectl apply -f ingress/svc-clusterip-order.yml
```

- AWS Load Balancer Controller에서 Sticky Session 기능을 활용하여 Client의 Session을 유지하도록 Annotations를 수정
- Sticky Session 기능을 사용하기위해 Target Type을 IP로 변경하고, Pod와 연결되는 Service의 Type 또한 Cluster-IP 형식으로 변경한다.
- 로컬 PC Web Browser에서 (list or order) 접속 테스트를 진행하여 Session 유지기능이 정상적으로 동작하는지 확인한다.

```
$ kubectl delete -f ingress/ingress-virtualhost.yml
```

- 다음 TEST를 위해 Ingress 오브젝트를 삭제한다.

Kubernetes Ingress Network Management (EX.1 : Basic)

■ [AWS Load Balancer Controller SSL/TLS(ACM) TEST]

<작업대상: ingress-tls.yml >

metadata:

```
name: my-ingress      # Ingress Object의 전체 Manifest File 내용은
namespace: delivery   TXT 교안을 참고하여 작성
annotations:
  kubernetes.io/ingress.class: alb
  alb.ingress.kubernetes.io/load-balancer-name: my-eks-alb
  alb.ingress.kubernetes.io/scheme: internet-facing
  alb.ingress.kubernetes.io/target-type: ip
  external-dns.alpha.kubernetes.io/hostname: www.mydevsecops.link
  alb.ingress.kubernetes.io/listen-ports: '[{"HTTP": 80},
{"HTTPS":443}]'
  alb.ingress.kubernetes.io/certificate-arn: "ACM ARN"
  alb.ingress.kubernetes.io/actions.ssl-redirect:
'{"Type": "redirect", "RedirectConfig": { "Protocol": "HTTPS",
  "Port": "443", "StatusCode": "HTTP_301"}}'
```

rules: # Ingress Routing Rule 정의

- http:

paths:

- pathType: Prefix

path: /list

backend:

service:

name: list-svc

port:

number: 80

- pathType: Prefix

path: /order

backend:

service:

name: order-svc

port:

number: 80

Client URI "/list" 요청
list-svc 서비스와 연결

Client URI "/order" 요청
order-svc 서비스와 연결

Kubernetes Ingress Network Management (EX.1 : Basic)

```
$ kubectl apply -f ingress/ingress-tls.yml
```

```
ingress.networking.k8s.io/my-ingress created
```

```
$ kubectl get ingress -n delivery
```

NAME	CLASS	HOSTS	ADDRESS	PORTS	AGE
my-ingress	<none>	*	elb.amazonaws.com	80	8s

▣ [로컬 PC Web Browser 접속 테스트]

- . http://www.mydevsecops.link [list Main Page]

- . http://www.mydevsecops.link/list [list Menu Page]

- . http://www.mydevsecops.link/order [order Menu Page]

▣ http 프로토콜을 이용한 접속 시 자동으로 https Redirect가 진행되는 것을 확인한다.

```
$ kubectl delete -f ingress/ingress-tls.yml
```

```
$ kubectl delete -f ingress/svc-clusterip-list.yml
```

```
$ kubectl delete -f ingress/svc-clusterip-order.yml
```

▣ 다음 TEST를 위해 Ingress TEST에서 사용 한 Ingress, Service, Deployment 오브젝트를 삭제한다.



Kubernetes Volume Management

Kubernetes Volume Management (Local Volume)

◎ Kubernetes Volume Management

- ▶ Kubernetes Volume : "Local Volume", "Persistent Volume" 2가지로 구분된다.
- ▶ Local Volume : k8s 내부에 Data 저장 경로를 정의하고 Data를 보관하는 방식 (노드 장애 발생 시 Data 사용 불가)
- ▶ Persistent Volume : Network 연결이 가능 한 Storage Server의 Volume을 Mount하여 사용하는 방식
- ▶ Network 연결이 가능 한 Persistent Volume의 대표적인 종류 : AWS EBS, GCE Persistent Volume, NFS, GlusterFS 등

◎ Kubernetes Local Volume (hostPath & emptyDir)

■ hostPath

- . Pod가 생성되는 Node의 FileSystem Directory와 Pod Container의 Directory를 Mount하여 Data를 저장하는 방식
- . hostPath에 저장되는 Data는 k8s 내부의 또 다른 Node에서 동작중인 Pod Container에서는 해당 Data를 사용 할 수 없다.
- . hostPath를 사용하는 Node에 장애 발생 시 내부 Pod가 다른 Node로 이동 될 경우 hostPath에 저장 된 Data는 사용이 불가능하다.
- . 보안성 및 Data 활용 측면에 부족한 부분이 많아 자주 사용되지 않는다.

■ emptyDir

- . Pod의 Data를 영속적으로 보존하는것이 아닌 Pod 동작 중 필요한 휘발성 Data를 저장하는 임시저장 공간
- . emptyDir은 비어있는 Directory의 형태로 생성되며 Pod 오브젝트 삭제 시 함께 삭제되는 특징을 갖는다.
- . emptyDir은 동일 Pod 오브젝트 내부의 Container간 Data 공유에 사용 될 수 있다.
- . EX: Git Hub에 저장 된 Source Cod를 내려받아 운영 Container로 공유하는 Side-Car Container 구성

Kubernetes Volume Management (EX.1 : hostPath)

```
$ kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
ip-192-168-10-147.ap-northeast-2.compute.internal	Ready	<none>	6d1h	v1.24.16-eks-8ccc7ba
ip-192-168-20-156.ap-northeast-2.compute.internal	Ready	<none>	6d1h	v1.24.16-eks-8ccc7ba

```
$ kubectl label node ip-192-168-10-147.ap-northeast-2.compute.internal node=node-1
```

```
$ kubectl label node ip-192-168-20-156.ap-northeast-2.compute.internal node=node-2
```

```
$ kubectl get nodes --show-labels
```

NAME	STATUS	ROLES	AGE	VERSION	LABELS
ip-192-168-10-147.ap-northeast-2.compute.internal	Ready	<none>	6d1h	v1.24.16-eks-8ccc7ba	
kubernetes.io/os=linux,node.kubernetes.io/instance-type=t3.large,node=node-1,topology.kubernetes.io/region=ap-northeast-2,topology.kubernetes.io/zone=ap-northeast-2a					
ip-192-168-20-156.ap-northeast-2.compute.internal	Ready	<none>	6d1h	v1.24.16-eks-	
kubernetes.io/os=linux,node.kubernetes.io/instance-type=t3.large,node=node-2,topology.kubernetes.io/region=ap-northeast-2,topology.kubernetes.io/zone=ap-northeast-2c					

▣ HostPath Volume TEST를 위해 Worker Node에 추가 Label을 부여

Kubernetes Volume Management (EX.1 : hostPath)

< 작업대상 : local-hostpath-1.yml >

apiVersion: apps/v1

kind: Deployment

metadata:

name: hostpath-deploy-1

namespace: delivery

spec:

selector:

matchLabels:

app: hostpath-app

Pod Template 영역에서 hostPath Type Volume 생성 (hostpath-volume)

Container 속성 영역에서 생성 된 Volume을 Mount 하도록 정의

hostpath-volume은 Container의 "/mnt" Directory와 연결된다.

template:

metadata:

labels:

app: hostpath-app

spec:

nodeSelector:

node: node-1

containers:

- name: hostpath-pod

image: chlzzz/kube-image:debug

volumeMounts:

- name: hostpath-volume

mountPath: /mnt

command: ['sh', '-c', 'tail -f /dev/null']

volumes:

- name: hostpath-volume

hostPath:

path: /tmp

Kubernetes Volume Management (EX.1 : hostPath)

< 작업대상 : local-hostpath-2.yml >

apiVersion: apps/v1

kind: Deployment

metadata:

name: hostpath-deploy-2

namespace: delivery

spec:

selector:

matchLabels:

app: hostpath-app

Pod Template 영역에서 hostPath Type Volume 생성 (hostpath-volume)

Container 속성 영역에서 생성 된 Volume을 Mount 하도록 정의

hostpath-volume은 Container의 "/mnt" Directory와 연결된다.

template:

metadata:

labels:

app: hostpath-app

spec:

nodeSelector:

node: node-2

containers:

- name: hostpath-pod

image: chlzzz/kube-image:debug

volumeMounts:

- name: hostpath-volume

mountPath: /mnt

command: ['sh', '-c', 'tail -f /dev/null']

volumes:

- name: hostpath-volume

hostPath:

path: /tmp

Kubernetes Volume Management (EX.1 : hostPath)

```
$ kubectl apply -f volume/
```

```
$ kubectl get pod -n delivery -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
hostpath-deploy-1-5585f44566-g8wm9	1/1	Running	0	119s	192.168.10.184	node-1
hostpath-deploy-2-6cc4cb89b7-dhp51	1/1	Running	0	119s	192.168.20.179	node-2

```
$ kubectl exec -it hostpath-deploy-1-5585f44566-g8wm9 -n delivery - sh
```

```
/ # touch /mnt/test.data
```

```
/ # ls -l /mnt/test.data
```

```
-rw-r--r--    1 root    root              0 Jul 27 04:44 /mnt/test.data
```

■ 생성 된 Pod 오브젝트의 Node 정보를 확인, 현재 node-1에서 Pod 오브젝트가 생성 된 것을 확인 할 수 있다.

■ Pod 오브젝트의 컨테이너로 접속 후 hostPath Local Volume과 연결 된 디렉터리 "/mnt" 디렉터리 내부에 테스트용 Data를 생성한다.

```
$ kubectl exec -it hostpath-deploy-2-6cc4cb89b7-dhp51 -n delivery - sh
```

```
/ # ls -l /mnt/test.data
```

```
ls: /mnt/test.data: No such file or directory
```

■ hostPath Volume은 Node의 실제 FileSystem과 연결되므로 Pod 오브젝트가 생성되는 Node가 변경 될 경우 기존 Data를 사용 할 수 없다.

Kubernetes Volume Management (EX.1 : hostPath)

```
$ kubectl exec -it hostpath-deploy-1-5585f44566-g8wm9 -n delivery - sh
```

```
/ # rm -rf /mnt/test.data
```

```
/ # ls -l /mnt/test.data
```

```
ls: /mnt/test.data: No such file or directory
```

▣ TEST에 사용한 데이터를 삭제

```
$ kubectl delete -f volume/
```

```
deployment.apps "hostpath-deploy-1" deleted
```

```
deployment.apps "hostpath-deploy-2" deleted
```

```
$ kubectl label node ip-192-168-10-147.ap-northeast-2.compute.internal node-
```

```
$ kubectl label node ip-192-168-20-156.ap-northeast-2.compute.internal node-
```

▣ Deployment 삭제 및 Worker Node의 추가한 Label 삭제

```
$ kubectl get nodes --show-labels
```

▣ Label 삭제 확인

Kubernetes Volume Management (EX.1 : emptyDir)

< 작업대상 : local-emptydir.yml >

apiVersion: apps/v1

kind: Deployment

metadata:

name: emptydir-deploy

namespace: delivery

spec:

selector:

matchLabels:

app: emptydir-app

template:

metadata:

labels:

app: emptydir-app

Pod Template 영역에서 emptyDir Type Volume 생성 (emptydir-volume)

emptydir-pod-write 컨테이너 : Data 생성 (Side-Car 컨테이너)

emptydir-pod-read 컨테이너 : Data 사용 (운영 컨테이너)

spec:

containers:

- name: emptydir-pod-write

image: chlzzz/kube-image:debug

volumeMounts:

- name: emptydir-volume

mountPath: /mnt

command: ['sh', '-c', 'tail -f /dev/null']

- name: emptydir-pod-read

image: nginx:latest

volumeMounts:

- name: emptydir-volume

mountPath: /usr/share/nginx/html

ports:

- containerPort: 80

volumes:

- name: emptydir-volume

emptyDir: {}

Kubernetes Volume Management (EX.1 : emptyDir)

```
$ kubectl apply -f volume/local-emptydir.yml
```

```
$ kubectl get pod -n delivery
```

NAME	READY	STATUS	RESTARTS	AGE
emptydir-deploy-55964b46c4-v8kkz	2/2	Running	0	7s

```
$ kubectl describe pod emptydir-deploy-55964b46c4-v8kkz -n delivery
```

Status: Running

IP: 192.168.10.21

Containers:

emptydir-pod-write:

Container ID: containerd://a185dd9278e85d12bc4f3a639ec1a08ce51d1912931d4ca9330d0acc1440006e

Image: chlzzz/kube-image:debug

emptydir-pod-read:

Container ID: containerd://eff2da2c7ac253613eb8e8f7b9c1cf9c92d1c1853092e3e972d1a357afb3a923

Image: nginx

▣ Pod 오브젝트 내부 컨테이너 2개가 정상적으로 동작중인지 확인한다.

▣ 하나의 Pod 오브젝트에서 여러 개의 컨테이너를 운영 할 경우 컨테이너 이름을 이용하여 구분한다.

Kubernetes Volume Management (EX.1 : emptyDir)

```
$ kubectl exec -it emptydir-deploy-55964b46c4-v8kkz -c emptydir-pod-write -n delivery - sh
/ # echo "Empty Dir Test" > /mnt/index.html
/ # cat /mnt/index.html
Empty Dir Test
```

▣ 컨테이너 구분을 위한 "-c" 옵션을 함께 사용하여 emptydir-pod-write 컨테이너에 접근한다.

▣ nginx가 구동중인 emptydir-pod-read 컨테이너에서 사용 할 index.html 데이터를 생성 후 확인

```
$ kubectl exec -it emptydir-deploy-55964b46c4-v8kkz -c emptydir-pod-read -n delivery - sh
# ls -l /usr/share/nginx/html
-rw-r--r-- 1 root root 15 Sep 19 03:01 index.html
```

```
$ kubectl run -i --rm --tty debug --image=chlzzz/kube-image:debug -- sh
/ # curl 192.168.10.21
Empty Dir Test
```

▣ curl 명령어를 이용하여 emptydir-pod-read 컨테이너에서 동작중인 nginx WEB Server에게 페이지 요청을 진행한다.

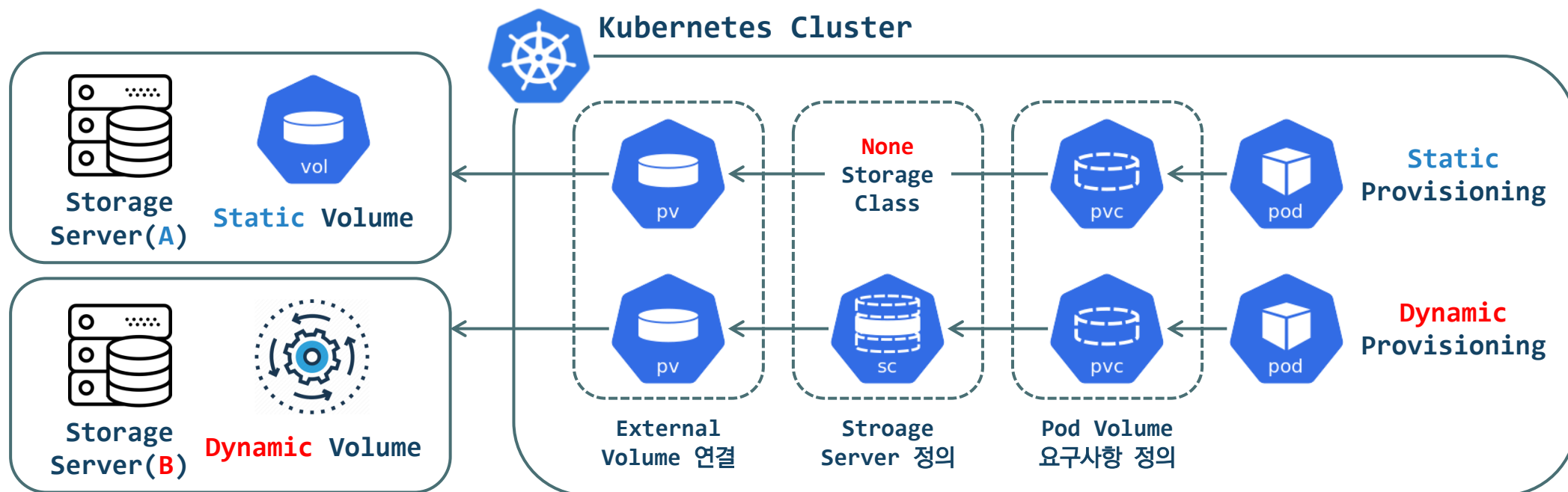
```
$ kubectl delete -f volume/local-emptydir.yml
```

▣ emptyDir Type의 Volume의 경우 Deployment 오브젝트에서 관리하는 Pod 오브젝트 삭제 시 함께 삭제된다.

Kubernetes Volume Management (Persistent Volume)

© Kubernetes Persistent Volume

- ▶ Kubernetes Persistent Volume 사용방법은 2가지로 구분 될 수 있다. (**Static Provisioning**, **Dynamic Provisioning**)
- ▶ Static Provisioning: 외부 Storage Server에서 Kubernetes에게 공유 할 목적의 Volume을 미리 구성
- ▶ Dynamic Provisioning: 외부 Storage Server에서 Kubernetes PVC 오브젝트의 요구사항에 맞는 새로운 Volume을 생성
- ▶ Public Cloud 환경 : Volume Provisioning (AWS EBS, GCE Persistent Volume)
- ▶ On-Premises 환경 : Static Provisioning (NFS) / Dynamic Provisioning (Gluster FS)



Kubernetes Volume Management (EX.2 : Persistent Volume)

< 작업대상 : efs-pv.yml >

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: efs-pv
  labels:
    name: efs-pv
spec:
  capacity:
    storage: 5Gi # Static Provisioning의 경우 구문만 작성
  volumeMode: Filesystem
  accessModes:
    - ReadWriteMany
  storageClassName: ""
  persistentVolumeReclaimPolicy: Retain
  csi:
    driver: efs.csi.aws.com
    volumeHandle: fs-09c234fc747386d4d # EFS-ID
```

< 작업대상 : efs-pvc.yml >

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: efs-claim
  namespace: delivery
spec:
  storageClassName: ""
  resources:
    requests:
      storage: 5Gi # Static Provisioning의 경우 구문만 작성
  selector:
    matchLabels:
      name: efs-pv
  accessModes:
    - ReadWriteMany
```


Kubernetes Volume Management (EX.2 : Persistent Volume)

```
$ kubectl apply -f volume/efs-pv.yml
```

```
$ kubectl get pv
```

NAME	CAPACITY	ACCESS MODES	RECLAIM POLICY	STATUS	CLAIM	STORAGECLASS	REASON	AGE
efs-pv	5Gi	RWX	Retain	Available				13m

▣ PV 오브젝트를 생성 후 확인, Kubernetes 전체에서 사용되므로 NameSpace 지정이 필요 없다.

```
$ kubectl apply -f volume/efs-pvc.yml
```

```
$ kubectl get pvc -n delivery
```

NAME	STATUS	VOLUME	CAPACITY	ACCESS MODES	STORAGECLASS	AGE
efs-claim	Bound	efs-pv	5Gi	RWX		13m

▣ PVC 오브젝트를 생성 후 확인, PVC 오브젝트는 특정 NameSpace에서만 사용 가능하므로 NameSpace를 지정한다.

```
$ kubectl get pv
```

NAME	CAPACITY	ACCESS MODES	RECLAIM POLICY	STATUS	CLAIM	STORAGECLASS	REASON	AGE
efs-pv	5Gi	RWX	Retain	Bound	delivery/efs-claim			13m

▣ 현재 PV 및 PVC 오브젝트의 Status 정보가 Bound 상태인 것을 반드시 확인한다. (NFS 연결이 정상적으로 진행 된 상태)

Kubernetes Volume Management (EX.2 : Persistent Volume)

< 작업대상 : efs-deploy.yml >

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: efs-deploy
  namespace: delivery
spec:
  replicas: 2
  selector:
    matchLabels:
      app: efs-app
  template:
    metadata:
      labels:
        app: efs-app
```

```
spec:
  containers:
    - name: efs-pod
      image: chlzzz/kube-image:debug
      volumeMounts:
        - name: efs
          mountPath: /mnt
      command: ['sh','-c','tail -f /dev/null' ]
      resources:
        limits:
          memory: "100Mi"
          cpu: "500m"
  volumes:
    - name: efs ←
      persistentVolumeClaim:
        claimName: efs-claim
```

Kubernetes Volume Management (EX.2 : Persistent Volume)

```
$ kubectl apply -f volume/efs-deploy.yml
```

```
$ kubectl get pod -n delivery -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED	NODE	READINESS
GATES									
efs-deploy-96b48c99b-g66zb	1/1	Running	0	17m	192.168.20.16	node-2	<none>		<none>
efs-deploy-96b48c99b-qp99v	1/1	Running	0	17m	192.168.10.182	node-1	<none>		<none>

```
$ kubectl exec -it efs-deploy-96b48c99b-g66zb -n delivery -- sh
```

```
/ # touch /mnt/efs.data
```

```
/ # ls -l /mnt
```

```
-rw-r--r--    1 root    root              0 Sep 20 04:30 efs.data
```

```
$ kubectl exec -it efs-deploy-96b48c99b-qp99v -n delivery -- sh
```

```
/ # ls -l /mnt
```

```
-rw-r--r--    1 root    root              0 Sep 20 04:30 efs.data
```

▣ EFS Persistent Volume을 사용하는 Pod내부에서 Test용 Data를 생성한다.

▣ 서로 다른 Worker Node에서 생성 된 Pod간 공유 스토리지를 사용하는 것을 확인한다.

Kubernetes Volume Management (EX.2 : Persistent Volume)

```
$ kubectl delete deploy efs-deploy -n delivery
```

```
deployment.apps "efs-deploy" deleted
```

```
$ kubectl delete pvc efs-claim -n delivery
```

```
persistentvolumeclaim "efs-claim" deleted
```

```
$ kubectl delete pv efs-pv
```

```
persistentvolume "efs-pv" deleted
```

```
$ kubectl apply -f volume/efs-pv.yml
```

```
persistentvolume/efs-pv created
```

```
$ kubectl apply -f volume/efs-pvc.yml
```

```
persistentvolumeclaim/efs-claim created
```

```
$ kubectl apply -f volume/efs-deploy.yml
```

```
deployment.apps/efs-deploy created
```

▣ Retain TEST를위해 Deployment, PVC, PV 오브젝트 삭제 후 재 생성을 진행한다.

Kubernetes Volume Management (EX.2 : Persistent Volume)

```
$ kubectl get pod -n delivery
```

NAME	READY	STATUS	RESTARTS	AGE
efs-deploy-96b48c99b-k5xrf	1/1	Running	0	25s
efs-deploy-96b48c99b-t2p1l	1/1	Running	0	25s

```
$ kubectl exec -it efs-deploy-96b48c99b-k5xrf -n delivery -- sh
```

```
/ # ls -l /mnt
```

```
-rw-r--r--    1 root    root              0 Sep 20 04:30 efs.data
```

■ PVC 오브젝트의 "ReclaimPolicy" = Retain 으로 정의 한 상태

■ PV, PVC, POD 오브젝트가 재 생성되더라도 기존 Data는 유지된다.

```
$ kubectl delete deploy efs-deploy -n delivery
```

```
deployment.apps "efs-deploy" deleted
```

```
$ kubectl delete pvc efs-claim -n delivery
```

```
persistentvolumeclaim "efs-claim" deleted
```

```
$ kubectl delete pv efs-pv
```

```
persistentvolume "efs-pv" deleted
```

■ 다음 TEST를 위하여 생성 된 Pod, Pvc, Pv 오브젝트 삭제를 진행한다.

Kubernetes Volume Management (EX.3 : Persistent Volume)

■ [EBS Dynamic Provisioning]

< 작업대상 : ebs-sc.yml >

apiVersion: storage.k8s.io/v1

kind: StorageClass

metadata:

name: ebs-sc

provisioner: ebs.csi.aws.com

volumeBindingMode: WaitForFirstConsumer

reclaimPolicy: Delete

parameters:

type: gp2

fsType: ext4

ReclaimPolicy : 해당 볼륨의 재사용 상태를 지정

-. Retain : PVC 오브젝트 삭제 시 PV에 저장 된 내용을 유지

-. Delete : PVC 오브젝트 삭제 시 PV 오브젝트 함께 삭제 (Dynamic 전용)

volumeBindingMode : Dynamic Provisioning 수행 시기

-. Immediate : Volume 즉시 생성

-. WaitForFirstConsumer : 연결 Pod 생성 완료 후 Volume 생성

\$ kubectl apply -f volume/ebs-sc.yml

\$ kubectl get sc

NAME	PROVISIONER	RECLAIMPOLICY	VOLUMEBINDINGMODE	ALLOWVOLUMEEXPANSION	AGE
ebs-sc	ebs.csi.aws.com	Delete	WaitForFirstConsumer	false	22s
gp2 (default)	kubernetes.io/aws-ebs	Delete	WaitForFirstConsumer	false	6d5h

Kubernetes Volume Management (EX.3 : Persistent Volume)

< 작업대상 : ebs-pvc.yml >

```
apiVersion: v1
```

```
kind: PersistentVolumeClaim
```

```
metadata:
```

```
  name: ebs-claim
```

```
  namespace: delivery
```

```
spec:
```

```
  accessModes:
```

```
    - ReadWriteOnce
```

```
  storageClassName: ebs-sc
```

```
  resources:
```

```
    requests:
```

```
      storage: 1Gi
```

accessModes : 생성되는 Persistent Volume에 대한 접근 정책을 정의

-. accessModes는 provisioner에 따라 지원하는 Mode가 다르다.

-. ReadOnlyMany : 다수의 Node에서 Mount 가능 한 Volume (읽기 전용)

-. ReadWriteMany : 다수의 Node에서 Mount 가능 한 Volume (AWS EFS)

-. ReadWriteOnce : 단일 Node에서만 Mount 가능 한 Volume (AWS EBS)

```
$ kubectl apply -f volume/ebs-pvc.yml
```

```
$ kubectl get pvc -n delivery
```

NAME	STATUS	VOLUME	CAPACITY	ACCESS MODES	STORAGECLASS	AGE
ebs-claim	Pending				ebs-sc	61s

Kubernetes Volume Management (EX.3 : Persistent Volume)

< 작업대상 : ebs-deploy.yml >

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: ebs-deploy
  namespace: delivery
spec:
  replicas: 1
  selector:
    matchLabels:
      app: ebs-app
  template:
    metadata:
      labels:
        app: ebs-app
```

```
spec:
  containers:
    - name: ebs-pod
      image: chlzzz/kube-image:debug
      volumeMounts:
        - name: ebs
          mountPath: /mnt
      command: ['sh', '-c', 'tail -f /dev/null' ]
      resources:
        limits:
          memory: "100Mi"
          cpu: "500m"
  volumes:
    - name: ebs
      persistentVolumeClaim:
        claimName: ebs-claim
```


Kubernetes Volume Management (EX.3 : Persistent Volume)

```
$ kubectl apply -f volume/ebs-deploy.yml
```

```
$ kubectl get pod -n delivery
```

NAME	READY	STATUS	RESTARTS	AGE
ebs-deploy-ff99546b5-wcjs2	1/1	Running	0	21s

```
$ kubectl get pvc -n delivery
```

NAME	STATUS	VOLUME	CAPACITY	ACCESS MODES	STORAGECLASS	AGE
ebs-claim	Bound	pvc-29787f0a-728f-4d45-b02d-f3ac2658ea00	1Gi	RWO	ebs-sc	26s

▣ Pod의 모든 구성이 완료 된 후 PVC 정보를 확인 STATUS "Bound" 확인

```
$ kubectl exec -it ebs-deploy-ff99546b5-wcjs2 -n delivery -- sh
```

```
/ # df -h | grep /mnt
```

```
/dev/nvme1n1          973.4M    24.0K    957.4M    0% /mnt
```

```
/ # touch /mnt/test.data
```

```
/ # ls -l /mnt
```

```
-rw-r--r--    1 root    root          0 Sep 19 08:13 test.data
```

▣ Pod 내부로 접속하여 TEST Data를 생성

Kubernetes Volume Management (EX.3 : Persistent Volume)

```
$ kubectl delete pod ebs-deploy-ff99546b5-wcjs2 -n delivery
```

```
$ kubectl get pod -n delivery
```

NAME	READY	STATUS	RESTARTS	AGE
ebs-deploy-ff99546b5-cmcct	1/1	Running	0	93s

```
$ kubectl exec -it ebs-deploy-ff99546b5-cmcct -n delivery -- sh
```

```
/ # ls -l /mnt
```

```
-rw-r--r--    1 root    root          0 Sep 19 08:13 test.data
```

- Persistent Volume의 데이터 영속성을 테스트하기 위해 현재 생성 된 Pod 오브젝트를 삭제 후 Pod 재배포 작업을 수행한다.
- 재배포 된 Pod 오브젝트로 접속하여 Persistent Volume 마운트 경로인 "/mnt" 디렉터리 하위에 테스트용 파일이 존재하는지 확인한다.
- Pod 오브젝트의 컨테이너는 1회성 사용(Stateless)이지만 Persistent Volume은 데이터의 영속성 보장(Stateful)한다.

```
$ kubectl delete -f volume/ebs-deploy.yml
```

```
$ kubectl delete -f volume/ebs-pvc.yml
```

- PVC 오브젝트를 삭제 할 경우 ReclaimPolicy가 "Delete"이므로 Persistent Volume이 함께 삭제 된다.
- StorageClass 오브젝트의 경우 Statefulset Controller에서 그대로 사용 할 예정이므로 삭제하지 않는다.

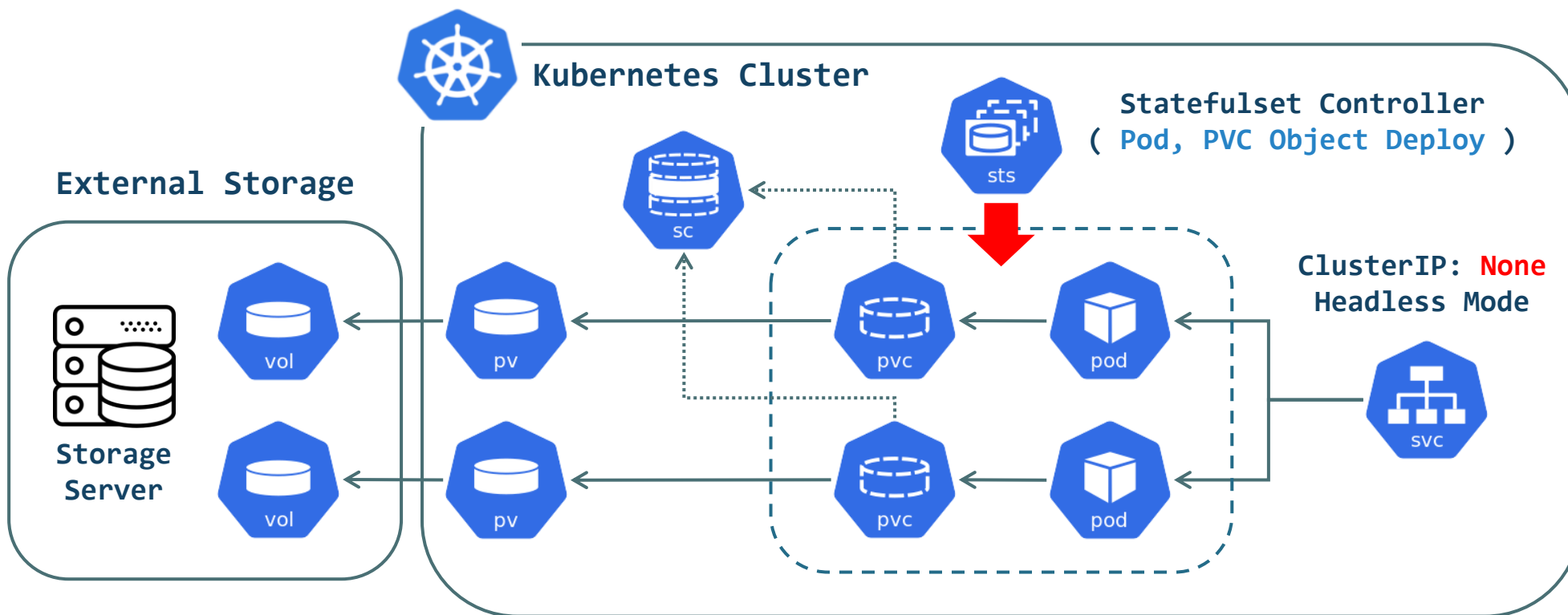


Kubernetes Statefulset

Kubernetes Statefulset Controller (Pod + Persistent Volume)

© Kubernetes Statefulset Controller

- ▶ Statefulset은 Data 보관이 필요한 Pod와 Persistent Volume을 조합하여 배포하기 위한 Controller (Deployment : Stateless)
- ▶ Statefulset은 Replica에서 지정한 수 만큼 Pod 오브젝트가 생성되며 각 Pod에 대응하는 Persistent Volume이 함께 생성된다.
- ▶ Statefulset에서는 Pod와 그에 대응하는 Persistent Volume을 하나의 관리 단위로 취급한다.
- ▶ Statefulset 제거 시 Pod는 삭제되지만 그에 대응하는 Persistent Volume은 삭제되지 않는다.



Kubernetes Statefulset Controller (EX.1 : MySQL DBMS 배포)

```
$ kubectl create namespace mysql
```

```
$ kubectl get ns mysql
```

NAME	STATUS	AGE
mysql	Active	10s

```
$ kubectl create secret generic mysql-secret --from-literal MYSQL_ROOT_PASSWORD=root -n mysql
```

```
$ kubectl get secret -n mysql
```

NAME	TYPE	DATA	AGE
mysql-secret	Opaque	1	8s

■ 새로운 NameSpace mysql을 생성 후 MySQL DBMS Statefulset 배포 작업을 진행한다.

■ MySQL 관리자 패스워드 환경변수는 Secret 오브젝트를 정의하여 작업을 진행한다. (MySQL 관리자 패스워드 : root 지정)

```
$ kubectl get sc
```

NAME	PROVISIONER	RECLAIMPOLICY	VOLUMEBINDINGMODE	ALLOWVOLUMEEXPANSION	AGE
ebs-sc	ebs.csi.aws.com	Delete	WaitForFirstConsumer	false	3s
gp2 (default)	kubernetes.io/aws-ebs	Delete	WaitForFirstConsumer	false	7d5h

■ Pod 오브젝트가 사용 할 Volume은 EBS Dynamic Provisioning을 사용 할 예정

■ EBS StorageClass 오브젝트가 존재하는지 확인한다. (SC 오브젝트가 존재하지 않을 경우 반드시 새로 생성)

Kubernetes Statefulset Controller (EX.1 : MySQL DBMS 배포)

< 작업대상 : sts-mysql.yml >

```
apiVersion: v1
kind: Service
metadata:
  name: mysql-service    # Service 이름정의 ( DNS 등록이름 )
  labels:
    app: mysql-sts
  namespace: mysql
spec:
  ports:
    - port: 3306
      name: mysql
  clusterIP: None      # ClusterIP Headless Mode
  selector:
    app: mysql-sts      # Statefulset Pod Template 연결
```

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: mysql-app
  namespace: mysql
spec:
  selector:
    matchLabels:
      app: mysql-sts    # Pod Template 이름 정의
  serviceName: mysql-service # Pod와 연결 할 Service 이름 정의
  replicas: 2
  volumeClaimTemplates: # PVC Template 정의
    - metadata:
        name: pvc    # PVC 오브젝트 이름 정의
      spec:
        storageClassName: ebs-sc    # StorageClass 이름 정의
        accessModes: [ "ReadWriteOnce" ] # 단일 Node에서만 접근 허용
        resources:
          requests:
            storage: 5Gi # Volume Size 정의
```

Kubernetes Statefulset Controller (EX.1 : MySQL DBMS 배포)

[※ Spec 영역에 이어서 작성]

```
template:                                # Template 영역 ( ★ )
  metadata:
    labels:
      app: mysql-sts                    # Teamplate 이름 정의
  spec:
    containers:
      - name: mysql                    # Container 이름 정의
        image: mysql:5.7              # Container 이미지 정의
        args:
          - "--ignore-db-dir=lost+found"
        ports:
          - containerPort: 3306        # Container Port번호 지정
            name: mysql
        envFrom:                       # Container 환경변수 정의
          - secretRef:
              name: mysql-secret        # Secret 오브젝트의 이름지정
```

```
volumeMounts:                          # Container Mount 영역
  - name: pvc                          # PVC 오브젝트 이름 정의
    mountPath: /var/lib/mysql          # Container Mount Point

livenessProbe:
  exec:
    command:
      - ls
      - /var/lib/mysql
    initialDelaySeconds: 10
    periodSeconds: 10
    successThreshold: 1
    failureThreshold: 3
    timeoutSeconds: 5
```

```
# EXEC 핸들러 정의 ( "/var/lib/mysql" 디렉터리가 존재하는지 확인 )
# Pod 배포 후 10초 뒤 실행 ( 실행 주기 : 10초 )
# 5번 연속 상태검사에 실패했을 경우 Container 재시작
# 상태 검사 성공 시 실패횟수는 초기화
```

Kubernetes Statefulset Controller (EX.1 : MySQL DBMS 배포)

```
$ kubectl apply -f statefulset/sts-mysql.yml
```

```
service/mysql-service created
```

```
statefulset.apps/mysql-app created
```

```
$ kubectl get all -n mysql
```

NAME	READY	STATUS	RESTARTS	AGE
pod/mysql-app-0	1/1	Running	1	41s
pod/mysql-app-1	1/1	Running	0	31s

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
service/mysql-service	ClusterIP	None	<none>	3306/TCP	41s

NAME	READY	AGE
statefulset.apps/mysql-app	2/2	41s

■ CLUSTER-IP가 할당되지 않은 Service 오브젝트 확인 (mysql-service)

■ Pod의 이름은 statfulset의 이름 + 순서번호로 정의된다.

■ Statefulset Controller에 의해 관리되는 Pod는 순차적으로 생성되므로 약간의 대기 시간이 필요하다.

Kubernetes Statefulset Controller (EX.1 : MySQL DBMS 배포)

```
$ kubectl get pv,pvc -n mysql
```

NAME	CAPACITY	ACCESS MODES	RECLAIM POLICY	STATUS	CLAIM	STORAGECLASS
persistentvolume/pvc-19922090	5Gi	RWO	Delete	Bound	mysql/pvc-mysql-app-1	ebs-sc
persistentvolume/pvc-eeaa80d5	5Gi	RWO	Delete	Bound	mysql/pvc-mysql-app-0	ebs-sc

NAME	STATUS	VOLUME	CAPACITY	ACCESS MODES	STORAGECLASS
persistentvolumeclaim/pvc-mysql-app-0	Bound	pvc-eeaa80d5-ed52	5Gi	RWO	ebs-sc
persistentvolumeclaim/pvc-mysql-app-1	Bound	pvc-19922090-6f32	5Gi	RWO	ebs-sc

▣ Statefulset Controller에 의해 생성된 PV, PVC 오브젝트를 확인

```
$ kubectl exec -it mysql-app-0 -n mysql -- bash
```

```
bash-4.2# mysql -u root -p mysql / Enter password: root
```

```
mysql> create database StatefulSet_DB;
```

```
mysql> show databases;
```

```
+-----+
| StatefulSet_DB |
+-----+
```

▣ Statefulset에 의해 관리되는 Pod의 컨테이너로 접속하여 새로운 데이터베이스를 생성 후 확인

Kubernetes Statefulset Controller (EX.1 : MySQL DBMS 배포)

```
$ kubectl delete -f statefulset/sts-mysql.yml
```

```
service "mysql-service" deleted
```

```
statefulset.apps "mysql-app" deleted
```

```
$ kubectl get pv,pvc -n mysql
```

NAME	CAPACITY	ACCESS MODES	RECLAIM POLICY	STATUS	CLAIM	STORAGECLASS
persistentvolume/pvc-19922090	5Gi	RWO	Delete	Bound	mysql/pvc-mysql-app-1	ebs-sc
persistentvolume/pvc-eeaa80d5	5Gi	RWO	Delete	Bound	mysql/pvc-mysql-app-0	ebs-sc

NAME	STATUS	VOLUME	CAPACITY	ACCESS MODES	STORAGECLASS
persistentvolumeclaim/pvc-mysql-app-0	Bound	pvc-eeaa80d5-ed52	5Gi	RWO	ebs-sc
persistentvolumeclaim/pvc-mysql-app-1	Bound	pvc-19922090-6f32	5Gi	RWO	ebs-sc

```
$ kubectl apply -f statefulset/sts-mysql.yml
```

```
service/mysql-service created
```

```
statefulset.apps/mysql-app created
```

- ▣ Statefulset 오브젝트의 Persistent Volume 영속성 유지 테스트를 위해 Statefulset 오브젝트를 삭제
- ▣ Statefulset 오브젝트를 삭제하여도 Persistent Volume은 삭제되지 않는 것을 확인한다.

Kubernetes Statefulset Controller (EX.1 : MySQL DBMS 배포)

```
$ kubectl exec -it mysql-app-0 -n mysql -- bash
```

```
bash-4.2# mysql -u root -p mysql
```

```
Enter password: root
```

```
mysql> show databases;
```

```
+-----+
```

```
| Database          |
```

```
+-----+
```

```
| information_schema |
```

```
| StatefulSet_DB     |
```

```
| mysql              |
```

```
| performance_schema |
```

```
| sys                 |
```

```
+-----+
```

```
5 rows in set (0.01 sec)
```

- Statefulset 오브젝트를 재 배포 후 Pod의 컨테이너로 접속 후 DB 목록을 확인한다. (Persistent Volume 데이터 유지)
- mysql-app-1 Pod가 연결하는 Persistent Volume에는 물리적으로 서로 다른 Volume이므로 해당 DB정보가 존재하지 않는다.
- 동기화 작업을 구현하기 위해서는 MQ 서비스를 사용하거나 DBMS Replication 등을 추가로 구현해야한다.

Kubernetes Statefulset Controller (EX.1 : MySQL DBMS 배포)

```
/ # ping mysql-app-0.mysql-service.mysql -c 4
```

```
PING mysql-service.mysql (10.244.1.191): 56 data bytes  
64 bytes from 192.168.10.231: seq=0 ttl=62 time=0.529 ms  
64 bytes from 192.168.10.231: seq=1 ttl=62 time=0.375 ms  
64 bytes from 192.168.10.231: seq=2 ttl=62 time=0.347 ms  
64 bytes from 192.168.10.231: seq=3 ttl=62 time=0.398 ms
```

```
/ # ping mysql-app-1.mysql-service.mysql -c 4
```

```
PING mysql-service.mysql (10.244.2.202): 56 data bytes  
64 bytes from 192.168.20.51: seq=0 ttl=64 time=0.371 ms  
64 bytes from 192.168.20.51: seq=1 ttl=64 time=0.089 ms  
64 bytes from 192.168.20.51: seq=2 ttl=64 time=0.165 ms  
64 bytes from 192.168.20.51: seq=3 ttl=64 time=0.080 ms
```

- Service 오브젝트와 연결된 Pod의 이름을 직접 지정할 경우 지정된 Pod와 고정적인 통신이 가능하다.
- Statefulset에 의해 관리되는 Pod의 경우 Pod 재시작이 진행되어도 항상 동일한 이름 부여 규칙에 따라 고정된 Pod 이름을 부여받는다.
- 고정된 Pod의 이름을 사용할 경우 Pod로 요청처리를 진행하는 클라이언트는 항상 동일한 Service 오브젝트 이름으로 요청처리를 진행할 수 있다.
- EX: DBMS를 운영하는 Pod의 접근 이름이 달라질 경우 해당 DBMS Pod를 사용하는 Application의 소스코드를 수정해야 하는 상황이 발생한다.

Kubernetes Statefulset Controller (EX.1 : MySQL DBMS 배포)

```
$ kubectl delete -f statefulset/sts-mysql.yml
```

```
service "mysql-service" deleted
```

```
statefulset.apps "mysql-app" deleted
```

```
$ kubectl get all -n mysql
```

```
No resources found in mysql namespace.
```

```
$ kubectl delete secret mysql-secret -n mysql
```

```
secret "mysql-secret" deleted
```

```
$ kubectl delete pvc pvc-mysql-app-0 -n mysql
```

```
persistentvolumeclaim "pvc-mysql-app-0" deleted
```

```
$ kubectl delete pvc pvc-mysql-app-1 -n mysql
```

```
persistentvolumeclaim "pvc-mysql-app-1" deleted
```

```
$ kubectl get pv,pvc -n mysql
```

```
No resources found
```

```
$ kubectl delete namespace mysql
```

▣ TEST에 사용한 Statefulset, Secret, PV, PVC, namespace 오브젝트 삭제를 진행한다.

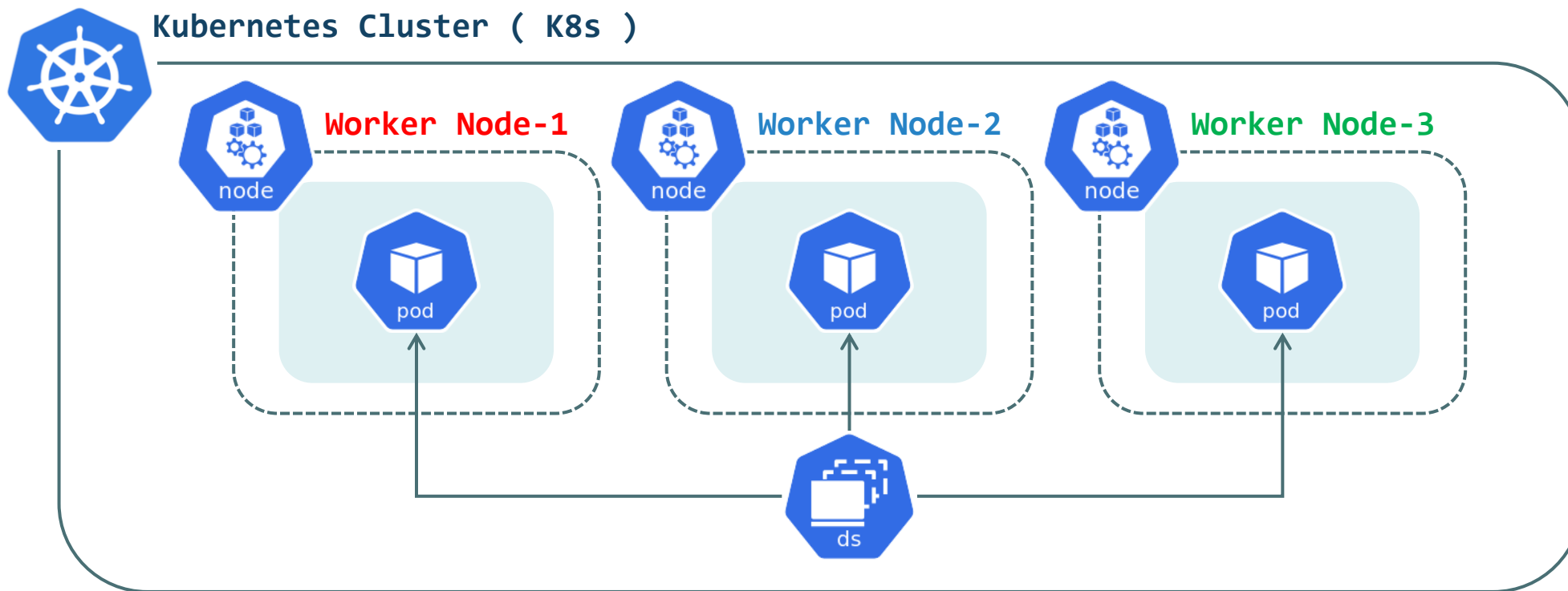


Kubernetes DaemonSet

Kubernetes DaemonSet Controller

◎ Kubernetes DaemonSet

- ▶ Kubernetes DaemonSet Controller는 K8s 내부 모든 Worker Node에 동일한 Pod를 생성하는 오브젝트
- ▶ DaemonSet Controller는 주로 Logging, Monitoring, Networking 등을 위한 Agent를 각 Worker Node에 구성 할 때 사용된다.
- ▶ K8s를 구성 시 Worker Node에서 생성되는 "Kube-Proxy"가 가장 대표적인 DaemonSet Controller에 의해 관리되는 Pod이다.
- ▶ DaemonSet에 의해 관리되는 Pod는 Node 장애시 다른 Node로 옮겨지지 않아야 한다, 이를 위해 자동으로 Toleration 설정이 포함된다.



Kubernetes DaemonSet Controller (EX.1 : Fluentd Pod 배포)

< 작업대상 : daemonset-test.yml >

apiVersion: apps/v1

kind: DaemonSet

metadata:

name: fluentd-elasticsearch

namespace: logging

labels:

k8s-app: fluentd-logging

spec:

selector:

matchLabels:

name: fluentd-elasticsearch

template:

metadata:

labels:

name: fluentd-elasticsearch

Deployment Controller 배포작업과 유사한 구조를 갖는다.

NodeAffinity / Tolerations 설정도 가능하다.

spec:

containers:

- name: fluentd-elasticsearch

image: quay.io/fluentd_elasticsearch/fluentd:v2.5.2

resources:

limits:

cpu: 100m

memory: 200Mi

requests:

cpu: 100m

memory: 200Mi

volumeMounts:

- name: varlog

mountPath: /var/log

volumes:

- name: varlog

hostPath:

path: /var/log

Kubernetes DaemonSet Controller (EX.1 : Fluentd Pod 배포)

```
$ kubectl create namespace logging
```

```
$ kubectl apply -f daemonset/daemonset-test.yml
```

```
daemonset.apps/fluentd-elasticsearch created
```

■ DaemonSet Controller를 활용하여 전체 Worker Node에 Fluentd Agent Pod를 배포한다.

■ Fluentd : OpenSource Data(Log) Collector Program (K8s에서 Worker Node의 로그를 수집하는데 자주사용 된다.)

```
$ kubectl get ds -n logging
```

NAME	DESIRED	CURRENT	READY	UP-TO-DATE	AVAILABLE	NODE SELECTOR	AGE
fluentd-elasticsearch	2	2	2	2	2	<none>	70s

■ DaemonSet Controller의 DESIRED / CURRENT / READY 확인 (현재 Worker Node는 2개가 운영중인 상태)

```
$ kubectl get pod -n logging -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE
READINESS GATES							
fluentd-elasticsearch-h2xkn	1/1	Running	0	39s	192.168.10.156	ip-192-168-10-147	<none> <none>
fluentd-elasticsearch-pzqmq	1/1	Running	0	39s	192.168.20.51	ip-192-168-20-156	<none> <none>

■ Fluentd Agent Pod가 운영중인 각 Worker Node에 배포된것을 확인한다.

Kubernetes DaemonSet Controller (EX.1 : Fluentd Pod 배포)

```
$ kubectl exec -it fluentd-elasticsearch-h2xkn -n logging -- sh
```

```
# tail /var/log/messages
```

```
Sep 25 11:27:42 ip-192-168-10-147 kubelet: I0925 11:27:42.7726732955 kubelet.go:2132]
Sep 25 11:28:30 ip-192-168-10-147 dhclient[2165]: XMT: Solicit on eth0, interval 108760ms.
Sep 25 11:30:18 ip-192-168-10-147 dhclient[2165]: XMT: Solicit on eth0, interval 115230ms.
Sep 25 11:31:54 ip-192-168-10-147 dhclient[2130]: DHCPREQUEST on eth0 to 192.168.10.1 port 67 (xid=0xb741a3d)
Sep 25 11:31:54 ip-192-168-10-147 dhclient[2130]: DHCPACK from 192.168.10.1 (xid=0xb741a3d)
Sep 25 11:31:54 ip-192-168-10-147 NET: dhclient: Locked /run/dhclient/resolv.lock
Sep 25 11:31:54 ip-192-168-10-147 dhclient[2130]: bound to 192.168.10.147 -- renewal in 1692 seconds.
Sep 25 11:32:14 ip-192-168-10-147 dhclient[2165]: XMT: Solicit on eth0, interval 123400ms.
Sep 25 11:34:17 ip-192-168-10-147 dhclient[2165]: XMT: Solicit on eth0, interval 130780ms.
Sep 25 11:36:28 ip-192-168-10-147 dhclient[2165]: XMT: Solicit on eth0, interval 113760ms.
```

▣ Fluentd Agent Pod에 접속 후 수집 된 Messages Log 기록을 확인한다.

▣ Fluentd Agent에서 수집 된 Log는 CloudWatch, Elasticsearch 등으로 전송하여 통합 로그 모니터링 환경을 구축 할 수도 있다.

```
$ kubectl delete -f daemonset/daemonset-test.yml
```

```
daemonset.apps "fluentd-elasticsearch" deleted
```

▣ TEST에 사용한 Daemonset 오브젝트 삭제를 진행한다.



Kubernetes AutoScaling

Kubernetes AutoScaling Summary

◎ Kubernetes Pod AutoScaling

- ▶ Pod AutoScaling : Pod의 자원 사용률에 따라 Pod나 Worker Node의 수를 자동으로 늘리고 줄이는 기능을 의미한다.
- ▶ Pod AutoScaling을 구현하기 위해서는 Metrics-Server 기능이 반드시 필요하다. (Resource의 자원 사용량을 수집)
- ▶ Pod AutoScaling 동작 지표 : HPA 기준 [CPU / Memory / Packets-Per-Seconds(Pod) / Request-Per-Seconds(Ingress)]
- ▶ Public Cloud의 경우 서비스 이용시간 및 사용률에 따른 다른 비용이 달라지기 때문에 비용 최적화에 많은 도움이 된다.

◎ Kubernetes AutoScaling HPA & VPA & CA

- ▶ Kubernetes AutoScaling 종류 : 수평 파드 오토스케일러(HPA), 수직 파드 오토스케일러(VPA), 클러스터 오토 스케일러(CA)
 - ▶ HPA : Pod의 자원 사용률을 감시하여 Pod의 Replicas 수를 증가 혹은 감소 시키는 것을 말한다. (**Scale-IN** / **Scale-OUT**)
 - ▶ VPA : Pod의 자원 사용률을 감시하여 Pod의 Resource 할당을 증가 혹은 감소 시키는 것을 말한다. (**Scale-UP** / **Scale-DOWN**)
 - ▶ CA : Node의 추가증설이 필요한 경우 Public Cloud API와 연동하여 K8s에 새로운 Worker Node를 추가하는 것을 말한다.
- ※ 참고 : AutoScaling 우선순위 [VPA > HPA > CA] / VPA의 경우 자원 사용률을 미리 예측해야하는 어려움이 있다.

◎ HPA (Horizontal Pod Autoscaler) 동작과정

- ▶ HPA는 Metrics-Server를 이용하여 현재 구성 된 Pod의 CPU 사용률을 수집한다. (Metrics-Server Addon 설치)
- ▶ 수집 된 Pod의 CPU 사용률 정보를 바탕으로 HPA에서 지정 된 Pod의 목표 CPU 평균 사용률을 만족하기 위해 Replicas 수를 조정한다.
- ▶ HPA는 Replicas 수정 작업을 30초 주기로 진행하며, Replicas의 변경사항이 필요한 경우에만 수정작업을 진행한다.
- ▶ Replicas 수정 작업 주기는 kubecontroller-manager의 속성으로 정의되어있다. (horizontal-pod-autoscaler-sync-period)
- ▶ HPA는 급격한 Pod 오브젝트의 개수 변화를 방지하기 위해 Pod 확장 / Pod 축소에 대한 약간의 대기 시간을 갖는다.

Kubernetes AutoScaling (Horizontal Pod AutoScaler)

■ [Metrics-Server Install]

```
$ kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml
```

- Metrics-Server는 Kubernetes Cluster 전체의 Resource 사용 Data를 수집한다. (Node의 kubelet을 이용하여 Resource 사용량을 수집)
- Kubernetes Cluster 구성 시 자동으로 구성되지 않으며 별도의 설치작업이 필요하다.
- Metrics-Server를 구성하는 방법은 Kubernetes 공식 Git-Hub에서 Metrics-Server Components.yaml을 이용하여 간단하게 설치가 가능하다.

```
$ kubectl get pod -n kube-system
```

NAME	READY	STATUS	RESTARTS	AGE
aws-load-balancer-controller-8684f5455c-2gkzd	1/1	Running	0	5d21h
aws-load-balancer-controller-8684f5455c-mrzsw	1/1	Running	0	10d
metrics-server-8cc45cd8d-5xvt7	1/1	Running	0	2m58s

```
$ kubectl top node
```

NAME	CPU(cores)	CPU%	MEMORY(bytes)	MEMORY%
ip-192-168-10-147.ap-northeast-2.compute.internal	64m	3%	1124Mi	15%
ip-192-168-20-156.ap-northeast-2.compute.internal	39m	2%	897Mi	12%

- 모든 설정을 완료 후 정상적으로 Metrics-Server의 역할을 수행하는 Pod 오브젝트가 Running 상태인지 확인한다.
- kubectl top node 명령어가 정상적으로 실행 될 경우 Metrics-Server 구성이 완료 된 상태.

Kubernetes AutoScaling (Horizontal Pod AutoScaler)

< 작업대상 : hpa-autoscaling.yml >

apiVersion: v1

kind: Service

metadata:

name: scale-svc

namespace: delivery

spec:

type: ClusterIP

selector:

app: scale-app

ports:

- port: 80

targetPort: 80

Service의 Type은 내부 연결을 통해 TEST할 예정이므로 ClusterIP로 정의

Resources 속성의 Request와 Limit의 값은 동일하게 설정

Guaranteed Class : Memory:100Mi, CPU:200M

spec:

replicas: 1

selector:

matchLabels:

app: scale-app

template:

metadata:

labels:

app: scale-app

spec:

containers:

- name: scale-app

image: chlzzz/kube-image:hpa

resources:

limits:

memory: 100Mi

cpu: 200m

requests:

memory: 100Mi

cpu: 200m

Kubernetes AutoScaling (Horizontal Pod AutoScaler)

< 작업대상 : hpa-test.yml >

apiVersion: autoscaling/v2

kind: HorizontalPodAutoscaler

metadata:

name: my-hpa

namespace: delivery

spec:

scaleTargetRef:

apiVersion: apps/v1

kind: Deployment

name: scale-deploy

minReplicas: 1

maxReplicas: 5

metrics:

- type: Resource

resource:

name: cpu

target:

type: Utilization

averageUtilization: 10

```
$ kubectl apply -f autoscaling/
```

```
service/scale-svc created
```

```
deployment.apps/scale-deploy created
```

```
horizontalpodautoscaler.autoscaling/my-hpa created
```

```
$ kubectl get hpa -n delivery
```

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
my-hpa	Deployment/scale-deploy	0%/10%	1	5	1	21s

■ HPA를 정의 후 hpa 오브젝트 정보를 확인 TARGETS의 데이터는 몇분이 흐른 뒤 다시확인한다.

■ 현재 HPA를 구성한 Deployment의 파드 전체의 CPU 사용량을 수집하는데 시간이 필요하다.

Kubernetes AutoScaling (Horizontal Pod AutoScaler)

[1번 Terminal] : Pod 오브젝트 부하도 증가 작업

```
$ kubectl run -i --tty --rm debug --image=chlzzz/kube-image:debug --restart=Never -n delivery -- sh
```

▣ Debug 전용 Pod를 하나 생성 후 생성 된 Pod 내부 Container로 접속

```
/ # while true; do wget -q -O - $SCALE_SVC_SERVICE_HOST:$SCALE_SVC_SERVICE_PORT > /dev/null; done
```

▣ 무한으로 wget 명령어를 이용하여 Pod 오브젝트의 부하도를 증가시킨다.

▣ 목표 평균 CPU 사용률을 만족 한 경우 작업을 중지한다.

[2번 Terminal] : HPA 모니터링 작업 (30초 주기로 HPA 오브젝트의 정보를 확인)

```
$ while true; do kubectl get hpa -n delivery; sleep 30; done
```

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
my-hpa	Deployment/scale-deploy	0%/10%	1	5	1	10m
NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
my-hpa	Deployment/scale-deploy	0%/10%	1	5	1	10m
NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
my-hpa	Deployment/scale-deploy	100%/10%	1	5	3	11m

▣ 부하도가 증가함에 따라서 Pod 오브젝트의 수가 함께 증가되는 것을 확인 ([다음 페이지에서 계속](#))

Kubernetes AutoScaling (Horizontal Pod AutoScaler)

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
my-hpa	Deployment/scale-deploy	57%/10%	1	5	5	11m

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
my-hpa	Deployment/scale-deploy	58%/10%	1	5	5	12m

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
my-hpa	Deployment/scale-deploy	58%/10%	1	5	5	12m

===== [목표 평균 CPU 사용률 만족 (Pod 부하도 증가 작업 중지)] =====

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
my-hpa	Deployment/scale-deploy	0%/10%	1	5	1	14m

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
my-hpa	Deployment/scale-deploy	0%/10%	1	5	1	15m

▣ REPLICAS의 수가 "5"까지 증가 (목표 평균 CPU 사용률 만족) / 부하도 작업 중지 후 REPLICAS의 수가 감소하는 것을 확인

▣ Pod 오브젝트 목록도 함께 확인 (`kubectl get pod -n delivery` / Pod 확장 (3분) / Pod 축소 (5분))

\$ kubectl delete -f autoscaling/

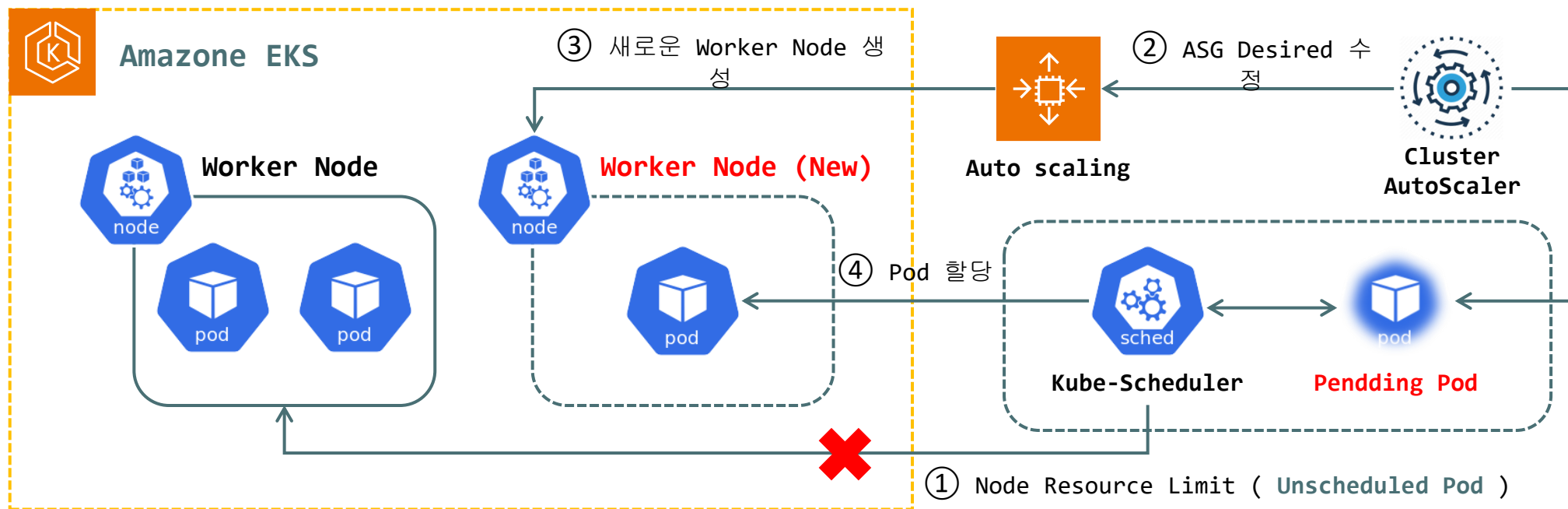
▣ 다음 TEST를 위해 HPA TEST에서 사용 한 HPA, Deployment 오브젝트를 삭제한다.

Kubernetes AutoScaling (Cluster AutoScaler)

◎ EKS Cluster AutoScaler 동작과정

1. HPA에 의한 수평확장이 최대치에 도달하면 Pod는 배포 Node를 배정 받지 못한 **Unscheduled Pod (Pending)** 상태가 된다.
2. CA는 이러한 Pod들의 상태를 지속 감시하면서 Pending 상태가 유지 될 경우 Node Group의 ASG Desired Capacity 값을 수정한다.
3. Node Group의 ASG Desired Capacity 값이 증가 될 경우 새로운 Worker Node가 생성되며 Pod를 배포 할 준비를 한다.
4. 새롭게 추가 된 Worker Node의 모든 배포 준비가 완료 될 경우 "kube-scheduler"가 새롭게 추가 된 Node에 Pod를 할당한다.

※ Cluster AutoScaler 단점으로는 AWS ASG 서비스의 의존도가 높아 높은 대기 시간을 갖게된다. / (Karpenter 도입 고려)



Kubernetes AutoScaling (Cluster AutoScaler)

< 작업대상 : ca-autoscaling.yml >

[※ ASG IRSA 설정 진행 후 TEST]

apiVersion: apps/v1

kind: Deployment

metadata:

name: scale-deploy

namespace: delivery

spec:

replicas: 3

selector:

matchLabels:

app: scale-app

T3 Family는 Xlarge, 2Xlarge를 제외하고 전부 물리 1 코어를 지원

Pod에 할당 할 코어를 1 코어로 지정하여 총 3개 Pod를 배포

Node 2EA: Running Pod : 2 / Pending Pod : 1

template:

metadata:

labels:

app: scale-app

spec:

containers:

- name: scale-pod

image: chlzzz/kube-image:hpa

resources:

limits:

memory: 100Mi

cpu: 1000m

requests:

memory: 100Mi

cpu: 1000m

Kubernetes AutoScaling (Cluster AutoScaler)

```
$ kubectl apply -f autoscaling/ca-autoscaling.yml
```

```
deployment.apps/scale-deploy created
```

```
$ kubectl get pod -n delivery
```

NAME	READY	STATUS	RESTARTS	AGE
scale-deploy-845778775d-ff6x2	1/1	Running	0	6s
scale-deploy-845778775d-k4wf2	1/1	Running	0	7s
scale-deploy-845778775d-zq2k9	0/1	Pending	0	6s

```
$ kubectl logs cluster-autoscaler-6cb97f77b4-74crx -n kube-system | grep Final
```

```
I0927 06:30:13.902533      1 scale_up.go:405] Final scale-up plan: [{eks-initial-8f5a-bfe1a6bc79d9 2->3 (max: 3)}]
```

```
$ kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
ip-192-168-10-147.ap-northeast-2.compute.internal	Ready	<none>	14d	v1.24.16-eks-8ccc7ba
ip-192-168-20-156.ap-northeast-2.compute.internal	Ready	<none>	14d	v1.24.16-eks-8ccc7ba
ip-192-168-20-158.ap-northeast-2.compute.internal	Ready	<none>	3m53s	v1.24.16-eks-8ccc7ba

■ 최초 Running Pod : 2 / Pending Pod : 1 / 새로운 Worker Node 생성 로그 확인 및 생성 된 Worker Node "Ready" 상태 확인

■ AWS 콘솔에서 ASG Group의 Desired Capacity의 값 "2"에서 "3"으로 변경 확인

Kubernetes AutoScaling (Cluster AutoScaler)

```
$ kubectl get pod -n delivery
```

NAME	READY	STATUS	RESTARTS	AGE
scale-deploy-845778775d-ff6x2	1/1	Running	0	86s
scale-deploy-845778775d-k4wf2	1/1	Running	0	87s
scale-deploy-845778775d-zq2k9	1/1	Running	0	86s

```
$ kubectl delete -f autoscaling/ca-autoscaling.yml
```

```
deployment.apps "scale-deploy" deleted
```

```
$ kubectl logs cluster-autoscaler-6cb97f77b4-74crx -n kube-system | grep Scale-down
```

```
I0927 06:49:59.766657      1 actuator.go:161] Scale-down: removing empty node "ip-192-168-20-158"
```

```
$ kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
ip-192-168-10-147.ap-northeast-2.compute.internal	Ready	<none>	14d	v1.24.16-eks-8ccc7ba
ip-192-168-20-156.ap-northeast-2.compute.internal	Ready	<none>	14d	v1.24.16-eks-8ccc7ba

■ 배포한 Pod를 삭제 후 약 10분정도의 대기시간을 갖고 Worker Node의 목록을 확인한다.

■ 새롭게 추가 된 Worker Node가 제거가 된 것을 확인 할 수 있다.



Kubernetes Related Tools

Kubernetes Related Tools (K9s / Lens / kubectl & kubens)

◎ Kubernetes Related Tools

▣ K9s

- ▶ K9s : Kubernetes Object관리를 위한 TUI 기반의 유틸리티 (Linux TOP 명령어와 유사한 형태로 Kubernetes Object를 관리)
- ▶ K9s는 오픈소스이며 구성이 간편하고 Kubernetes Cluster 관리작업을 직관적으로 수행 할 수 있는 장점이 있다.
- ▶ K9s URL: <https://k9scli.io/>

▣ Lens

- ▶ Lens : Kubernetes Object관리를 위한 Desktop App (Desktop APP 형식으로 Kubernetes Object를 관리)
- ▶ Lens의 경우 2022년 유료화가 되었으며 OpenLens 형식의 무료버전을 사용 할 수 있다.
- ▶ Lens는 Kubernetes Cluster 관리작업을 직관적으로 수행 할 수 있는 장점이 있다.
- ▶ Lens URL: <https://k8slens.dev/>

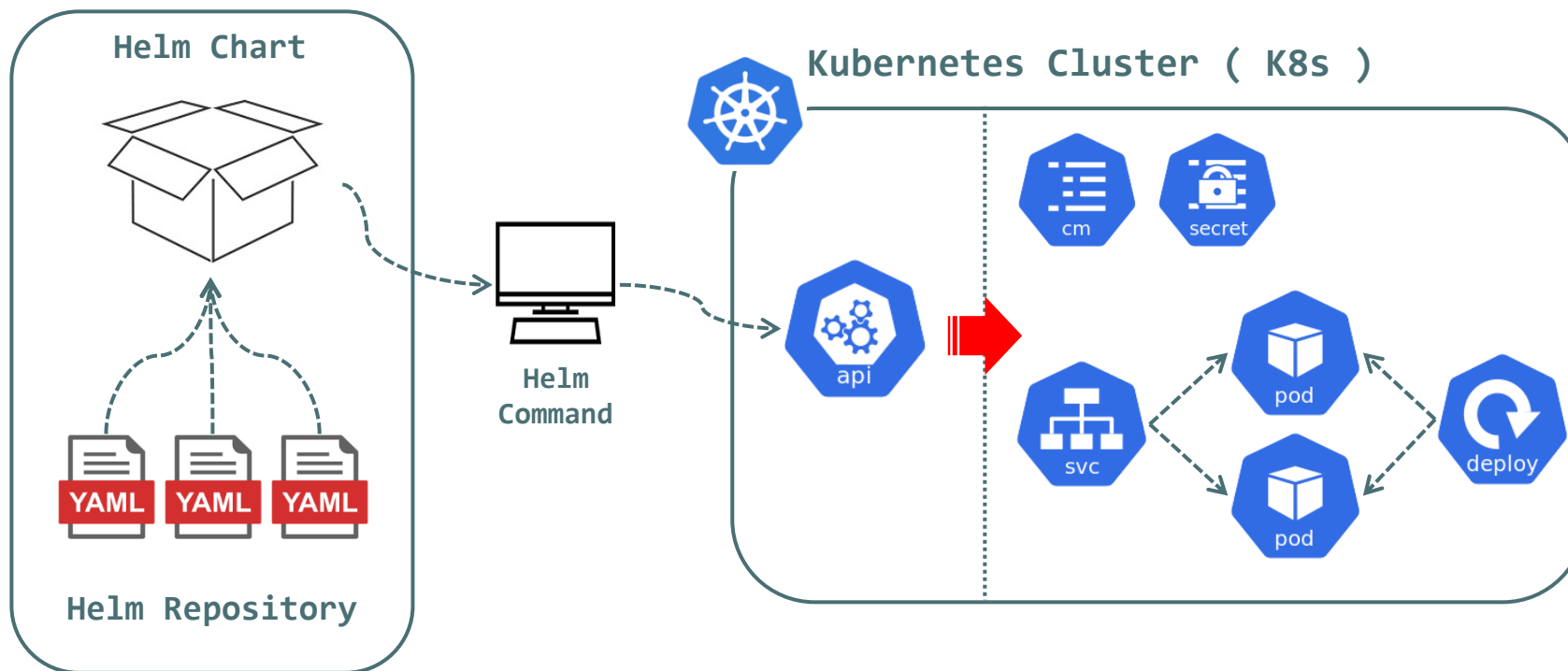
▣ Kubectl & kubens

- ▶ kubectl : kubectl의 Context 전환을 쉽게 할 수 있도록 도움을 주는 도구 (Kubernetes Cluster 환경 전환)
- ▶ kubens : Kubernetes Cluster 내부의 Namespace 간 전환을 쉽게 할 수 있도록 도움을 주는 도구 (작업 Namespace 전환)
- ▶ kubectl & kubens URL: <https://github.com/ahmetb/kubectl>

Kubernetes Related Tools (Helm)

© Kubernetes Package Manager HELM

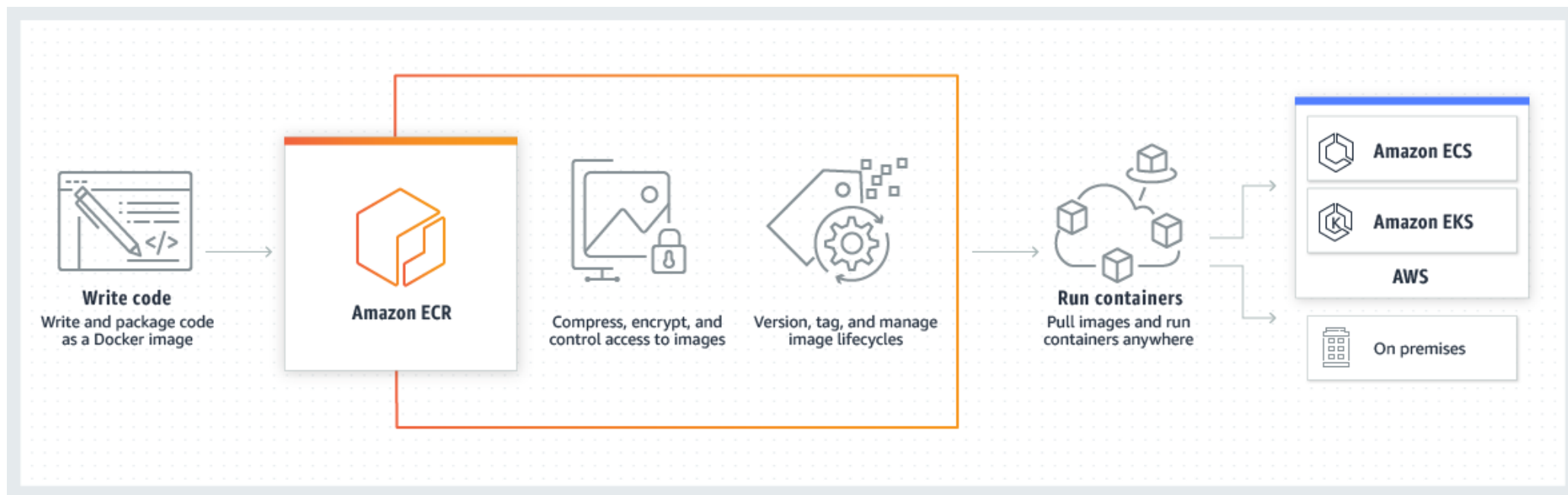
- ▶ Helm : Kubernetes Package Manager (Linux OS: Package Manager와 동일한 역할)
- ▶ Helm은 Kubernetes Cluster에서 배포되는 Application을 **Chart**(패키지) 형태로 제공한다.
- ▶ Chart 내부에는 Application이 가동되기 위한 모든 Resource를 포함하고 있다. (Deploy / ConfigMAP / Secret / Service)
- ▶ Helm은 Revision 관리를 진행하며 동일한 Chart를 여러번 설치가 가능하며, 각 설치 작업을 Revision 정보로 관리한다.



Kubernetes Related Tools (AWS ECR)

◎ AWS ECR (Elastic Container Registry)

- ▶ AWS에서 제공하는 완전 관리형 Container Registry 서비스 (AWS EKS 환경을 사용하는 경우 대부분 ECR 서비스를 함께 사용)
- ▶ AWS ECR은 각 계정 별 Registry 공간을 부여받게 되며 해당 Registry에 실제 이미지를 저장 할 수 있는 Repository를 구성한다.
- ▶ AWS ECR은 이미지를 사용하는 사용자를 식별하기 위한 사용자 권한 토큰을 사용한다. (Image Push, Pull 권한)
- ▶ AWS ECR은 이미지의 수명주기를 관리하고, 외부 Public Registry를 Cache 할 수 있으며, 저장 된 이미지의 SW 취약점을 검사한다.
- ▶ Amazon ECR Public Gallery : <https://gallery.ecr.aws/> (ECR 공개 저장소로 다양한 이미지를 내려받아 사용가능)



Kubernetes Related Tools (Karpenter)

◎ Karpenter

- ▶ Karpenter : AWS에서 개발한 Kubernetes의 Worker Node 자동 확장 기능을 수행하는 오픈소스 프로젝트 (JIT: Just In-Time 배포)
 - ▶ Karpenter는 AWS Resource 의존성 없이 JIT 배포를 수행하므로 기존 Cluster AutoScaler 보다 빠른 확장성을 제공한다.
1. HPA에 의한 수평확장이 최대치에 도달하면 Pod는 배포 Node를 배정 받지 못한 Pending (Unscheduled Pod) 상태가 된다.
 2. Karpenter는 이러한 Pod들의 상태를 지속 감시하면서 Pending 상태가 유지 될 경우 직접 새로운 Node를 추가한다.
 3. Worker Node의 모든 배포 준비가 완료 될 경우 "kube-scheduler" 대신 Karpenter가 새롭게 추가 된 Node에 Pod를 할당한다.

